Entità e Occorrenze

Entità:

Un'**entità** è una categoria (o classe) di oggetti che condividono le stesse caratteristiche.

I tre punti chiave da ricordare:

- **Natura:** Può essere fisica (es. *Impiegato*, *Automobile*) o astratta (es. *Banca*, *Corso*).
- **Rilevanza:** Deve essere un concetto utile per il database che si sta progettando.

Occorrenza di Entità:

Uno specifico oggetto reale, distinto e autonomo, appartenente alla classe definita dall'entità (ad esempio, *"Napoli"* è un'occorrenza dell'entità *Città*). Un'occorrenza non è il valore o il dato che identifica l'oggetto, bensì l'oggetto stesso nella sua interezza.

Associazioni (o Relazioni)

Associazione:

Un legame logico e concettuale che stabilisce una connessione tra due o più entità, reputato rilevante all'interno del dominio di interesse analizzato (ad esempio, l'associazione *Afferenza* tra l'entità *Impiegato* e l'entità *Dipartimento*).

- **Occorrenza di Associazione** È il legame "pratico" tra gli oggetti reali. Se l'associazione è il concetto generale "Lavora in", l'occorrenza è la coppia esatta: *Mario Rossi lavora in Amministrazione*. In un'associazione binaria l'occorrenza è una coppia; in un'associazione n-aria è una n-upla. **Esempio pratico:** L'associazione "Prestito" che collega *Studente*, *Libro* e *Biblioteca*. **L'occorrenza (la tripla):** (Alessandro, Libro di Informatica, Biblioteca di Ingegneria). All'interno di una medesima associazione non sono ammesse occorrenze duplicate.
- **Associazione N-aria** Un legame concettuale che coinvolge contemporaneamente tre o più entità. Un esempio è l'associazione *Fornitura*, che lega contestualmente le entità *Fornitore*, *Prodotto* e *Dipartimento*.
- **Associazione Ricorsiva** Un'associazione che stabilisce un legame tra un'entità e se stessa (ad esempio, l'associazione *Collega* che unisce coppie di occorrenze appartenenti alla stessa entità *Impiegato*).
- **Ruolo** La specifica della funzione o della direzione che un'entità assume quando partecipa a un'associazione. La definizione dei ruoli è indispensabile nelle associazioni ricorsive non simmetriche per eliminare le ambiguità (ad esempio,

nell'associazione *Successione* legata all'entità *Pontefice*, i ruoli distinguono il "Predecessore" dal "Successore").

Attributi

Attributo:

Una **caratteristica** o un **dettaglio** specifico di un'entità o di un'associazione che descrive una caratteristica di interesse ai fini dell'applicazione software o del sistema informativo.

- **Attributo Semplice** Un attributo atomico non ulteriormente scomponibi.
- **Attributo Composto** Un insieme di attributi strettamente correlati dal punto di vista semantico o logico, raggruppati per esprimere un concetto unitario (ad esempio, l'attributo composto *Indirizzo* costituito dai sotto-attributi *Via*, *Numero Civico* e *CAP*).
- **Attributo Multivalore** Un attributo che può assumere un insieme di valori distinti per una singola occorrenza di un'entità. Viene formalizzato mediante una coppia di valori che ne descrivono la cardinalità.
- **Attributo Opzionale** Un attributo la cui cardinalità minima è pari a 0 (0,1). Indica che il valore può non essere presente o non essere specificato per alcune occorrenze, senza che ciò comprometta la validità e la significatività dell'istanza.

Vincoli di Cardinalità

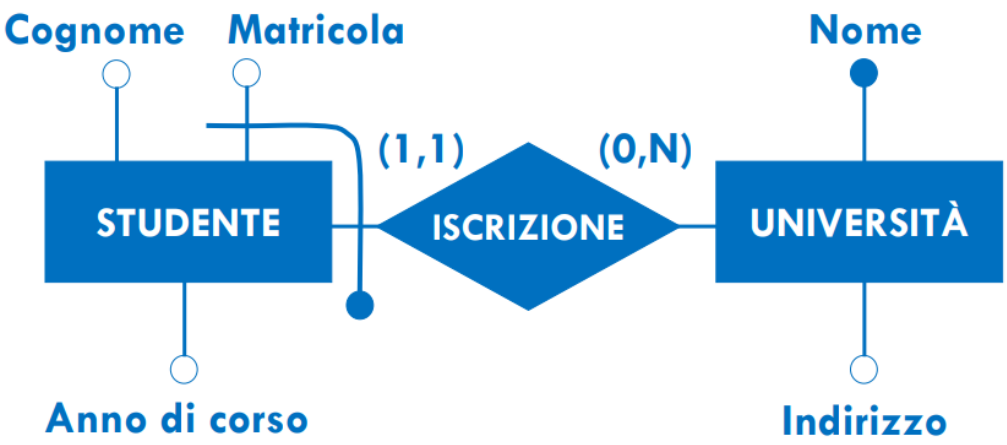
- **Cardinalità di un'Associazione** Una coppia di valori (min, max) associata a ciascuna entità che partecipa a un legame. Specifica il numero minimo e il numero massimo di occorrenze dell'associazione a cui una singola occorrenza dell'entità può prendere parte. I valori standard utilizzati per la modellazione sono:
    - **Cardinalità minima = 0**: Partecipazione opzionale dell'entità.
    - **Cardinalità minima = 1**: Partecipazione obbligatoria dell'entità.
    - **Cardinalità massima = 1**: L'occorrenza si lega al massimo a un'istanza.
    - **Cardinalità massima = N**: Non viene posto alcun limite superiore al numero di legami.
- **Rapporto di Cardinalità (per Associazioni Binarie)** Classificazione sintetica basata sui limiti massimi di partecipazione delle due entità coinvolte. Si distinguono in tre configurazioni principali:

1. **Uno a uno (1:1)**
2. **Uno a molti (1:M)** (richiede rigorosa attenzione alla direzione del verso logico della relazione)
3. **Molti a molti (M:M)** *Nota d'esame: Le relazioni di tipo obbligatoria-obbligatoria (1:1 con minimi a 1) sono estremamente rare nella pratica progettuale.*

- **Cardinalità di un Attributo** Una coppia di valori (min, max) che definisce il numero minimo e massimo di istanze di un attributo associabili a una singola occorrenza dell'entità di appartenenza.

5. Identificatori Univoci (Chiavi)

- **Identificatore di un'Entità** Lo strumento concettuale deputato all'identificazione univoca di ogni singola occorrenza di un'entità. Ogni entità deve possedere necessariamente almeno un identificatore (ma può averne molteplici). Graficamente si denota con un cerchietto pieno.
- **Identificatore Interno** Un identificatore composto esclusivamente da uno o più attributi interni dell'entità stessa (es. *CodiceFiscale* per l'entità *Persona*).
- **Identificatore Esterno** Un identificatore che, per garantire l'univocità, deve combinare uno o più attributi interni con entità esterne collegate tramite un'associazione. Un identificatore esterno può essere validamente costituito solo se la cardinalità dell'associazione tra l'entità in esame e l'entità esterna di supporto è strettamente pari a **(1,1)**. Graficamente si rappresenta con una linea che attraversa sia gli attributi interni facenti parte della chiave sia il segmento dell'associazione.



Due Studenti iscritti a due Università diverse possono avere lo stesso Numero di Matricola.

Quindi per identificare univocamente uno Studente è necessario il Numero di Matricola e la relativa Università.

Generalizzazioni (o Gerarchie)

Generalizzazione:

Un legame logico che unisce un'entità più generica, detta **Padre** (o *superclasse*), a una o più entità specifiche, dette **Figlie** (o *sottoclassi*), che ne rappresentano dei casi particolari.

- *Esempio pratico:* **Padre:** Veicolo; **Figlie:** Automobile, Camion
- *Nota:* Una stessa entità può essere coinvolta in più generalizzazioni contemporaneamente.

- **Sottoinsieme** È il caso più semplice di gerarchia: si ha quando l'entità Padre ha **una sola** entità Figlia (invece di due o più).
- **Ereditarietà (Regola fondamentale)** È il principio cardine delle gerarchie e comporta due regole ferree:

1. Ogni oggetto dell'entità Figlia è automaticamente anche un oggetto dell'entità Padre.
2. Le entità Figlie "ereditano" automaticamente tutti gli attributi (e i legami) dell'entità Padre.

- *Regola per il disegno E-R:* Proprio per via di questa ereditarietà, nello schema concettuale gli attributi del Padre **NON vanno mai ridisegnati** sulle entità Figlie, altrimenti faresti un errore di ridondanza.

Vincoli delle Gerarchie

I vincoli stabiliscono le regole di appartenenza degli oggetti tra il Padre e le Figlie. Si dividono in due grandi categorie:

A. Vincolo di Copertura (Riguarda il Padre)

Determina se le entità Figlie "coprono" o meno tutti i casi possibili dell'entità Padre.

- **Totale:** Tutte le occorrenze del Padre devono appartenere per forza ad almeno una Figlia.
  - *Esempio:* L'entità *Persona* ha come figlie *Uomo* e *Donna*. È totale perché non esiste una persona che non appartenga a una di queste due figlie.
- **Parziale:** Esistono occorrenze del Padre che NON appartengono a nessuna delle entità Figlie disegnate.

    - *Esempio:* L'entità *Veicolo* ha come figlie *Automobile* e *Bicicletta*. È parziale perché esistono veicoli (come i camion o le moto) che rimangono solo nel Padre senza scendere nelle Figlie.

B. Vincolo di Disgiunzione (Riguarda le Figlie)

Determina se le entità Figlie si escludono a vicenda oppure no.

- **Disgiunta (o Esclusiva):** Un'occorrenza del Padre può appartenere al massimo a **una sola** entità Figlia. Le categorie non possono mischiarsi.
    - *Esempio:* Una *Persona* è o *Uomo* o *Donna*. Non può stare in entrambe le entità figlie.
- **Sovrapposta:** Un'occorrenza del Padre può appartenere **contemporaneamente** a più entità Figlie.
    - *Esempio:* L'entità *Persona* ha come figlie *Studente* e *Lavoratore*. È sovrapposta perché una persona può far parte contemporaneamente di entrambe le entità figlie.

Risoluzione delle Sovrapposizioni (Trucco di Modellazione)

- **Da Sovrapposta a Disgiunta:** Una gerarchia sovrapposta può essere trasformata in una disgiunta semplicemente creando una nuova entità Figlia che rappresenti "l'incrocio" delle due.
    - *Esempio:* Invece di avere *Studente* e *Lavoratore* che si accavallano, aggiungi la terza entità figlia **Studente-Lavoratore**. A quel punto, le tre figlie diventano perfettamente separate ed esclusive tra loro.

Relazione (Concetto Teorico)

Deriva dalla teoria matematica degli insiemi. Formalmente, è un sottoinsieme del prodotto cartesiano di n domini (cioè insiemi di valori).

- **Il valore di *n* è finito:** Questo significa che una relazione deve avere un numero limitato e ben preciso di attributi (colonne). Questo garantisce una rappresentazione finita e strutturata delle informazioni.

Tabella (Concetto Pratico)

È la rappresentazione grafica e intuitiva della Relazione.

La regola d'oro: Nel modello relazionale, ogni riga è un oggetto e ogni colonna è una sua caratteristica. I dati contenuti nelle celle devono essere "atomici" (cioè non ulteriormente scomponibili).

Terminologia fondamentale da esame:

- **Grado:** È il numero n di colonne (attributi) che compongono la tabella.
- **Cardinalità della Tabella:** È il numero totale di righe (tuple) materialmente presenti in quel momento nella tabella.
- **Tupla (o n-upla):** È la singola riga della tabella. Rappresenta una specifica occorrenza (es. i dati completi di un singolo studente).
    - *Nota logica:* Oltre alle tuple registrate, a livello teorico è utile considerare anche l'esistenza potenziale di n-uple valide che al momento non sono presenti fisicamente nella relazione (es. lo studente perfetto che prenderà tutti 30L esiste come combinazione logica, anche se non si è ancora immatricolato).
- **Dominio:** L'insieme di tutti i valori validi e consentiti che una colonna può assumere (es. per l'attributo *Voto*, il dominio è un numero intero da 18 a 30).
    - **Cardinalità del dominio:** A differenza della tabella, il dominio può essere considerato di grandezza **infinita** (es. l'insieme di tutte le possibili stringhe di testo per il campo *Cognome* o l'insieme dei numeri reali).
    - **Tipologia dei domini:** All'interno di una tabella, le colonne possono pescare valori da domini **tutti di tipo diverso** (es. Testo per il *Nome*, Interi per l'*Età*), oppure possono condividere **lo stesso tipo** (es. le colonne *Data_Immatricolazione* e *Data_Laurea* pescano entrambe dal medesimo dominio temporale).

Le Proprietà di una Relazione (Tabella valida)

Affinché una tabella sia considerata una vera "Relazione", deve rispettare regole ferree:

1. **Nessun ordine tra le righe:** Scambiare di posto due tuple non cambia il significato della tabella.

2. **Tuple tutte distinte:** Non possono assolutamente esistere due righe identiche in ogni singola colonna (nessun duplicato).
3. **Nessun ordine tra le colonne (Superamento della rigidità posizionale):** A differenza della matematica pura, nei database l'ordine delle colonne non conta.

- *Come è possibile matematicamente?* Noi eliminiamo la corrispondenza posizionale associando ad ogni singolo valore (v_i) il nome della sua intestazione, ovvero l'Attributo (A_i).
- *La definizione formale:* Di conseguenza, una tupla non è più una semplice lista ordinata di valori, ma diventa un insieme di coppie "Valore-Attributo":
  $t = \{\langle v_1, A_1\rangle, \langle v_2, A_2\rangle, \ldots, \langle v_n, A_n\rangle\}$
- *Esempio pratico:* L'insieme $\{\langle \text{Mario, Nome}\rangle, \langle \text{Rossi, Cognome}\rangle\}$ è esattamente identico a $\{\langle \text{Rossi, Cognome}\rangle, \langle \text{Mario, Nome}\rangle\}$. Ogni dato sa già a chi appartiene grazie alla sua etichetta, rendendo totalmente irrilevante l'ordinamento all'interno della tupla.

Dalla Tabella allo Schema: Formalizzazione e Istanze

Tabella vs Relazione (Attenzione all'esame):

Una relazione può essere intuitivamente rappresentata tramite una tabella, in cui ogni riga equivale a una tupla e ogni colonna contiene i valori relativi a uno specifico attributo (l'intestazione). *Regola fondamentale:* **Non tutte le tabelle sono relazioni**. Una generica tabella è considerata una vera "Relazione" nel database *se e solo se* soddisfa rigorosamente la Proprietà 1 (nessun ordine logico tra le righe) e la Proprietà 2 (totale assenza di tuple identiche/duplicate).

Schema di Relazione (Componente Intensionale):
È l'impalcatura fissa, strutturale e teorica della tabella. Formalmente, dato un insieme di nomi di attributi $X = \{A_1, A_2, \ldots, A_n\}$, lo schema di relazione è definito da un nome R (il nome della tabella) seguito dal suo set di attributi:
$R(X) = R(A_1, A_2, \ldots, A_n)$
Esempi pratici: `AUTORI(Codice, Nome, Cognome, Nazionalità)` oppure `STUDENTI(Matricola, Nome, Cognome)`.

Istanza di Relazione (Componente Estensionale): È il contenuto reale del database, ovvero l'insieme dei dati "vivi" (le tuple) fisicamente presenti nella tabella in un preciso istante di tempo. Mentre l'istanza è dinamica e muta continuamente (man mano che aggiungi o elimini righe), lo schema di base che la contiene resta identico e immutabile.

Il Valore NULL

Nel modello relazionale, i dati non possono avere "buchi" non gestiti. Quando un'informazione manca, si usa il valore speciale **NULL**. Il NULL non significa "zero" o "spazio vuoto", ma rappresenta **l'assenza di informazione**. Può indicare tre situazioni ben distinte:

1. **Valore Sconosciuto:** Il dato esiste nella realtà, ma non lo conosciamo (es. uno studente ha il cellulare, ma non ce lo ha comunicato).
2. **Valore Inesistente:** Il dato non è applicabile a quella tupla (es. il "Telefono Ufficio" per un normale studente non esiste proprio).
3. **Senza Informazione:** Non sappiamo se il dato esista o meno (siamo totalmente all'oscuro).

Vincoli di Integrità

Sono regole (IC, *Integrity Constraints*) che i dati devono obbligatoriamente rispettare per far sì che il database abbia senso e rispecchi il mondo reale. Si dividono in due grandi famiglie:

A. Vincoli Intra-relazionali (All'interno della stessa tabella)

- **Vincolo di Dominio:** Controlla i valori di una singola colonna (es. L'attributo *Età* non può essere minore di 0 o maggiore di 100).
- **Vincolo di Tupla:** Controlla più colonne della stessa riga incrociando i dati (es. In una riga di esame, l'attributo *Lode* può valere 'SI' *solo se* l'attributo *Voto* vale '30').

B. I Vincoli di Identificazione (Le Chiavi)

- **Superchiave:** Un insieme di attributi (anche sovrabbondante) che riesce a identificare univocamente ogni singola riga della tabella. (*Esempio:* Matricola + Nome + Cognome).
- **Chiave (o Superchiave Minimale):** È una superchiave a cui non puoi togliere nemmeno un attributo, altrimenti perde il suo potere identificativo. (*Esempio:* La sola Matricola).
- **Chiave Primaria (Primary Key):** Tra le varie chiavi possibili in una tabella, è quella scelta ufficialmente dal progettista per identificare le righe.
    - **Integrità dell'Entità (Regola Sacra):** Nessun valore che compone la Chiave Primaria può **mai** essere NULL. Se manca la chiave primaria, l'inserimento della riga viene bloccato.

C. Vincoli Inter-relazionali (Tra tabelle diverse)

Servono a suddividere correttamente le informazioni su più tabelle per evitare ridondanze (doppioni), mantenendo però ben saldi i collegamenti logici tra i dati.

- **Chiave Esterna (Foreign Key - FK)** È un attributo (o gruppo di attributi) all'interno di una tabella che fa da "ponte", riferendosi direttamente alla Chiave Primaria (PK) di un'altra tabella.
    - *Regola di base:* Poiché rappresentano la stessa informazione, la Chiave Esterna e la Chiave Primaria a cui punta devono essere definite esattamente sullo **stesso dominio** (es. se la chiave *Matricola* in STUDENTI è un numero a 6 cifre, anche la chiave esterna *MatStudente* in CARRIERE deve esserlo) (**CARRIERE**(**MatStudente**: STUDENTI, **CodiceCorso**: CORSI, Data, Voto)).
- **Vincolo di Integrità Referenziale** È la regola di sicurezza che garantisce che i "ponti" tra le tabelle non crollino. In parole povere: ogni valore inserito in una Chiave Esterna **deve esistere obbligatoriamente** come valore nella Chiave Primaria della tabella di destinazione. Non puoi far riferimento a qualcosa che non esiste.
    - **Definizione Formale (Da esame):**
      Date due relazioni r_1 ed r_2, con r_1 dotata di chiave esterna FK che punta alla chiave primaria PK di r_2, sussiste un vincolo di integrità referenziale se per ogni tupla $t_1 \in r_1$ si verifica una di queste due condizioni:

      1. Il valore $t_1[FK]$ è **NULL** (se l'assenza di collegamento è ammissibile).
      2. $\exists t_2 \in r_2 \mid t_2[PK] = t_1[FK]$ *(Ovvero: esiste una tupla t_2 nella seconda tabella in cui la chiave primaria è esattamente uguale alla chiave esterna della prima tabella).*

Progettazione Logica

È la fase in cui prendi lo schema concettuale (il tuo diagramma E-R fatto di rettangoli e rombi) e lo traduci in un modello logico dei dati (nel nostro caso, il Modello Relazionale, cioè le **tabelle**).

- **A cosa serve:** Fa da ponte tra l'idea astratta e la realizzazione pratica, definendo la struttura dei dati indipendentemente da come verranno salvati fisicamente sull'hard disk.

La Fase di Ristrutturazione

La regola d'oro: Non tutto quello che puoi disegnare in uno schema E-R può essere trasformato direttamente in una tabella. Prima di tradurre, devi "pulire" lo schema tramite la Ristrutturazione.

Eliminazione degli Attributi Complessi

I database relazionali vogliono dati semplici e "atomici" (un solo valore per cella).

- **Attributi Composti:** Si "spacchettano". L'attributo composto viene eliminato e sostituito direttamente dai suoi componenti semplici.
    - *Esempio:* Cancelli l'attributo *Indirizzo* e al suo posto metti *Via*, *Numero Civico* e *CAP*.
- **Attributi Multivalore:** I database odiano le celle con più valori dentro. Per risolvere, l'attributo multivalore viene cancellato dall'entità originale e diventa **una nuova Entità a sé stante**, collegata a quella di partenza con un'associazione 1 a Molti (1:N).

La Scelta degli Identificatori (La Chiave Primaria)

Per poter collegare le tabelle tra loro (tramite le Chiavi Esterne) e per far sì che il database trovi i dati velocemente, devi scegliere un **Identificatore Principale (Chiave Primaria)** per ogni entità.

Se un'entità ha più identificatori candidati, come scegli quello giusto? Usi tre criteri:

Criterio di Unicità (Obbligatorio)

- L'identificatore scelto non deve **mai** ammettere valori NULL. Se un attributo può essere lasciato vuoto, scartalo subito: non può fare da chiave.

Criteri di Efficienza (Per ottimizzare il DB)

1. **Scegli il più "piccolo":** È sempre da preferire l'identificatore composto dal minor numero di attributi (es. meglio un singolo codice numerico che la combinazione

Nome + Cognome + Data di Nascita). Occupa meno memoria ed è più veloce da leggere per il sistema.

2. **Scegli il più usato:** È da preferire l'identificatore che verrà utilizzato più spesso dagli utenti per fare ricerche o operazioni.

E se nessun attributo va bene? (Identificatori Aggiunti)

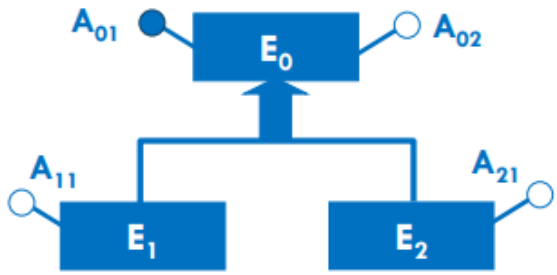
Se l'entità non ha attributi adatti a fare da chiave primaria perfetta, **te ne inventi uno tu**. Si aggiunge un campo artificiale (surrogato) al solo scopo di identificare la riga:

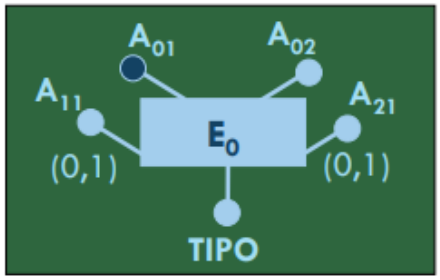
- **ID Numerico:** Un contatore generato automaticamente dal database (es. Utente 1, Utente 2, Utente 3).
- **Codice Mnemonico/Parlante:** Un codice alfanumerico generato fondendo altri dati (es. il *Codice Fiscale*, la *Matricola Universitaria* o il *Codice Articolo*).

Risoluzione delle Gerarchie (Specializzazioni/Generalizzazioni)

Soluzione 1: Accorpamento verso l'Alto (Tutto nel Padre)

- **Cosa fai:** Elimini le entità Figlie. L'entità Padre "ingloba" tutti gli attributi delle figlie. Aggiungi un nuovo attributo inventato (es. Tipo) per distinguere di che categoria fa parte ogni riga.
- **Quando usarla:** Quando le operazioni che farai sul database riguardano quasi sempre l'entità generale, senza fare troppe distinzioni tra le categorie figlie.
- **Pro (Efficienza):** Velocità. Con una sola tabella, il sistema fa meno "salti" (accessi) per trovare i dati.
- **Contro (Spazio):** Spreco di memoria. Genera un sacco di campi vuoti (NULL) perché gli attributi specifici di una figlia rimarranno vuoti per tutte le altre.

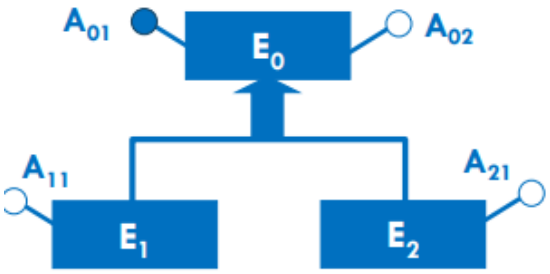


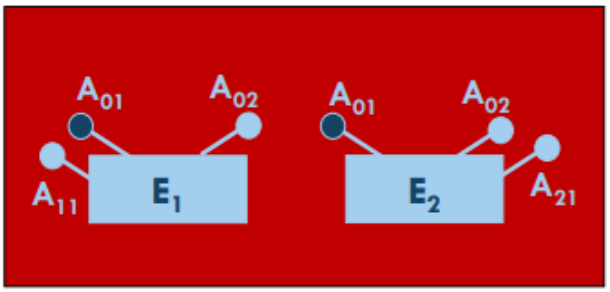


Soluzione 2: Accorpamento verso il Basso (Tutto nelle Figlie)

- **Cosa fai:** Elimini l'entità Padre. Le entità Figlie sopravvivono e "ereditano" (copiano su di sé) tutti gli attributi che aveva il padre. La generalizzazione scompare e ti restano solo tabelle separate per ogni categoria specifica.

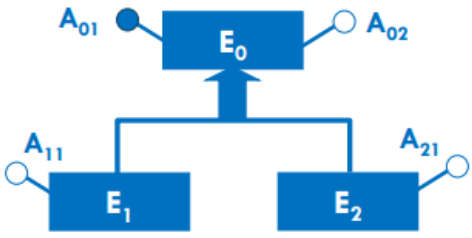
- **Quando usarla: SOLO in presenza di generalizzazioni Totali** (dove ogni padre è per forza almeno una figlia) e quando le operazioni del database lavorano quasi sempre sulle singole categorie specifiche in modo separato.
- **Pro (Spazio):** Ottimizza la memoria. Niente celle vuote (NULL) e non serve inventarsi l'attributo Tipo.
- **Contro (Vincolo letale):** Se la generalizzazione è Parziale, **non puoi usare questa soluzione**. Se lo fai, tutte le occorrenze del padre che non appartengono a nessuna figlia verrebbero letteralmente cancellate dal database!

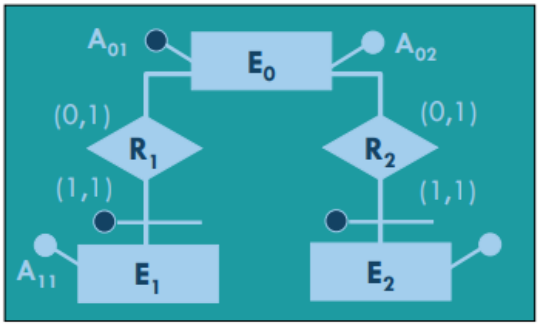




Soluzione 3: Mantenimento con Associazioni (Tabelle Separate)

- **Cosa fai:** Non elimini nulla. Tieni in vita sia l'entità Padre che le entità Figlie. Per non perdere il legame, le colleghi creando delle associazioni (relazioni 1:1) tra il padre e ciascuna figlia.
- **Quando usarla:** È la soluzione universale, perfetta per le **generalizzazioni Parziali** o quando hai operazioni molto diverse: alcune che interrogano solo il padre (es. contare tutte le persone) e altre che interrogano solo le figlie (es. calcolare lo stipendio solo per i lavoratori).
- **Pro (Spazio e Logica):** Niente valori NULL, niente campi Tipo inventati, e nessuna perdita di dati. Rappresenta la realtà in modo pulito.
- **Contro (Efficienza):** È la più lenta. Per avere le informazioni complete di un'entità figlia, il database deve fare un doppio accesso (leggere la tabella della figlia + leggere la tabella del padre collegata).





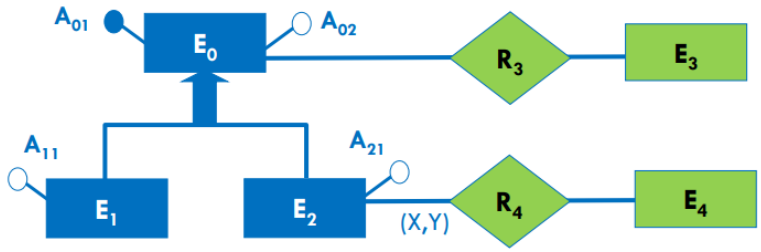
Risoluzione Avanzata delle Gerarchie

A. Gerarchie a Più Livelli (Matrioske)

Quando ti trovi davanti a una generalizzazione dentro l'altra (es. *Veicolo* -> *Automobile* -> *Utilitaria / SUV*):

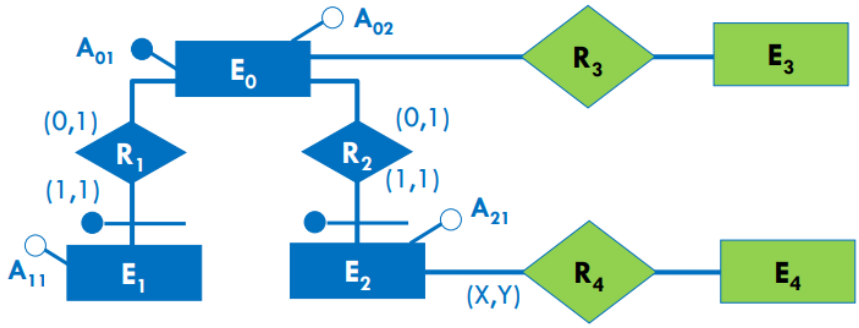
- **Regola d'esame:** Si procede analizzando e smontando una generalizzazione alla volta.
- **Da dove si parte?** Sempre dal **basso verso l'alto** (dai livelli più bassi delle figlie per poi risalire verso il padre generale).

B. Casi Complessi (Gerarchie con Associazioni Esterne)



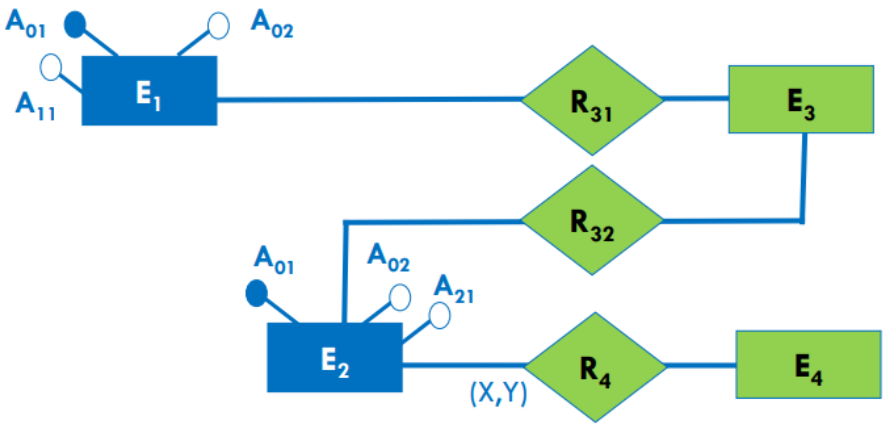
Cosa succede quando il Padre o le Figlie sono collegate ad *altre* entità del database? I tre metodi visti prima si adattano in questo modo:

- **Mantenimento di tutte le entità** *(Nelle nuove slide: Soluzione 1)*
  - Non elimini nulla. Crei le famose associazioni 1:1 tra Padre e Figlie.
  - **Sulle relazioni esterne:** Tutto resta esattamente dov'è. Il Padre mantiene le sue associazioni, e le Figlie mantengono le loro. È la soluzione più sicura ed è **sempre applicabile**.

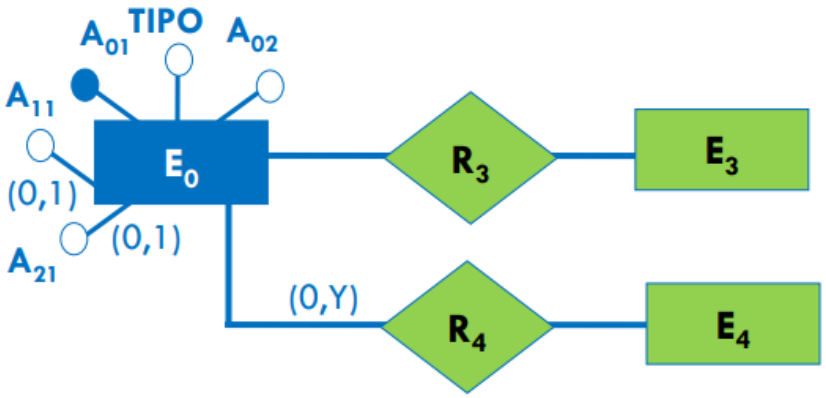


- **Accorpamento verso il Basso** *(Nelle nuove slide: Soluzione 2)*
  - Elimini il Padre. Le Figlie sopravvivono.
  - **Sulle relazioni esterne:** Le Figlie non ereditano solo gli attributi del Padre, ma **ereditano anche le sue associazioni**. Se il Padre era collegato all'entità *Reparto*, ora dovrai disegnare due associazioni separate: una che collega la

prima Figlia al *Reparto* e una che collega la seconda Figlia al *Reparto*. (Consigliabile per gerarchie Totali e Disgiunte).



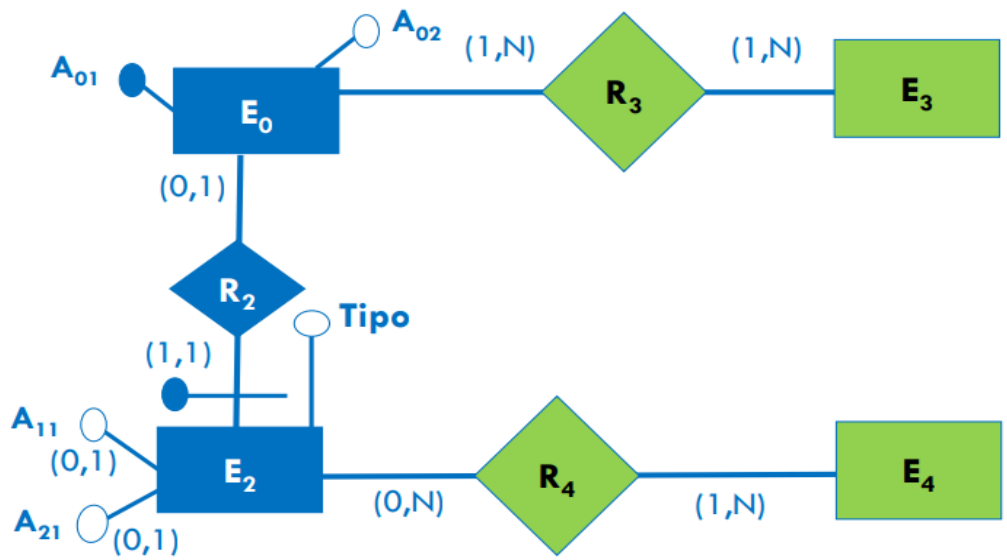
- **Accorpamento verso l'Alto** *(Nelle nuove slide: Soluzione 3)*
  - Elimini le Figlie. Il Padre sopravvive.
  - **Sulle relazioni esterne:** Il Padre "assorbe" le associazioni che prima appartenevano alle singole Figlie.
  - **Attenzione al trabocchetto (Vincolo di Cardinalità):** Poiché il Padre assorbe un'associazione che in realtà riguardava solo una specifica categoria di figlie, la **cardinalità minima** di questa nuova associazione attaccata al Padre **deve per forza diventare 0**. (Adatta per gerarchie Parziali e Sovrapposte).



C. La Soluzione 4: L'Approccio "Misto"

Questa è una soluzione ibrida molto elegante, perfetta per le **gerarchie Parziali e Sovrapposte**, pensata per non riempire il Padre di valori nulli ma senza creare mille tabelle diverse per le Figlie.

- **Come funziona:** 1. Mantieni l'entità Padre originale. 2. Cancelli tutte le diverse entità Figlie e **le fondi in un'UNICA grande entità "Sottoclasse"** (un super-contenitore delle figlie). 3. Colleghi il Padre a questa Sottoclasse unica con un'associazione (1,1).
- **Cosa c'è dentro la Sottoclasse Unica?**
    - Avrà un nuovo attributo inventato **"Tipo"** (per capire di che ex-figlia stiamo parlando).
    - Conterrà **tutti** gli attributi specifici di tutte le vecchie figlie.
    - Assorbirà **tutte le associazioni esterne** a cui partecipavano le vecchie figlie (sempre con cardinalità minima a 0, visto che non tutte le righe parteciperanno a tutti i legami).



Traduzione verso il Modello Relazionale (Da E-R a Tabelle)

Una volta "pulito" lo schema E-R (tramite la ristrutturazione vista prima), sei pronto per tradurlo. La regola generale è semplice:

1. **Le Entità diventano Tabelle.**
    - **La Tabella:** Prende il nome dell'entità, ma **messo al plurale**. (es. Entità *Impiegato* -> Tabella *IMPIEGATI*).
    - **Le Colonne:** Diventano gli attributi che l'entità aveva nello schema.
    - **La Chiave Primaria:** Diventa l'identificatore principale scelto durante la ristrutturazione.
2. **Le Associazioni diventano Tabelle** (ma con regole precise in base alla cardinalità).
3. Si creano **Vincoli di Integrità Referenziale** (le chiavi esterne) per non perdere i collegamenti.

Traduzione delle Associazioni: Regole per Cardinalità

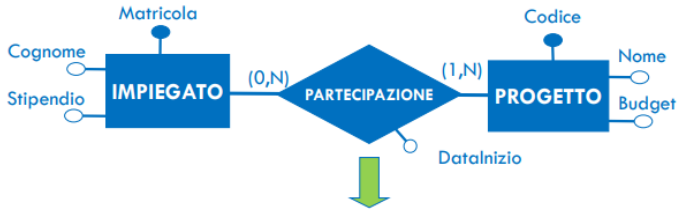
Il modo in cui un'associazione si trasforma in tabella dipende **esclusivamente** dal suo rapporto di cardinalità (1:1, 1:M o M:M). Iniziamo dal caso più classico e sicuro.

Caso A: Le Associazioni Molti a Molti (M:M)

Un'associazione (o relazione) Molti a Molti si traduce **SEMPRE generando una nuova tabella a sé stante**.

- **Nome Tabella:** Il nome dell'associazione al plurale (es. associazione *Partecipa* -> Tabella *PARTECIPAZIONI*).
- **Attributi (Colonne):** 1. Gli identificatori delle entità che collega (che diventano automaticamente Chiavi Esterne). 2. Eventuali attributi propri dell'associazione (es. la *Data* o il *Voto* dell'esame).
- **Chiave Primaria:** È formata dalla **combinazione** degli identificatori delle entità coinvolte.

Esempio Pratico (Impiegati e Progetti):



Lo schema logico tradotto sarà composto da 3 tabelle:

1. IMPIEGATI (Matricola, Cognome, Stipendio)
2. PROGETTI (Codice, Nome, Budget)
3. PARTECIPAZIONI (Matricola_Imp: IMPIEGATI, Codice_Prog: PROGETTI, DataInizio) *(Nota: la chiave primaria di PARTECIPAZIONI è la coppia: Matricola_Imp + Codice_Prog).*

3. Ridenominazione degli Attributi (Evitare Confusioni)

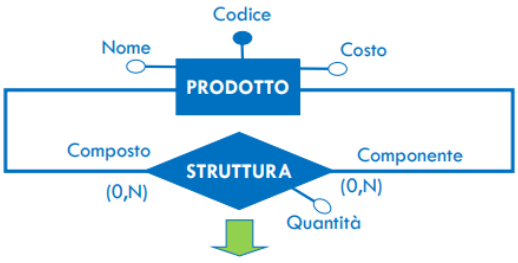
Nella tabella derivata dall'associazione, è **caldamente consigliato** (e a volte obbligatorio) cambiare il nome degli identificatori "importati" per rendere la tabella più leggibile.

- *Come si fa:* Invece di chiamare la colonna semplicemente Matricola, la chiami Impiegato specificando che è una chiave esterna: (Impiegato: IMPIEGATI).

Quando la ridenominazione è OBBLIGATORIA

È assolutamente obbligatoria nelle **Associazioni Ricorsive** (quando un'entità si collega a se stessa). Se non lo facessi, avresti una tabella con due colonne che si chiamano allo stesso modo (errore fatale nei database).

- **Esempio (Distinta Base dei Prodotti):**
  1. PRODOTTI (Codice, Nome, Costo)
  2. STRUTTURE (Composto: PRODOTTI, Componente: PRODOTTI, Quantità)

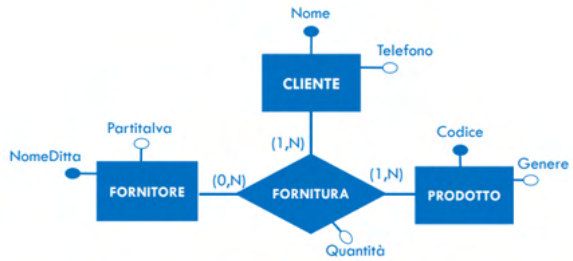


(Se non le avessimo ridenominate in "Composto" e "Componente", avremmo avuto la colonna "Codice" ripetuta due volte!).

4. Associazioni N-arie (Più di due entità)

Le associazioni che collegano tre o più entità contemporaneamente (N-arie) si comportano **esattamente come le Molti a Molti (M:M)**. Si crea una tabella a sé stante.

Esempio (Fornitura): Un *Fornitore* fornisce un *Prodotto* a un *Cliente* in una certa *Quantità*.



- FORNITORI (Partitalva, NomeDitta)
- PRODOTTI (Codice, Genere)
- CLIENTI (Nome, Telefono)
- FORNITURE (Fornitore: FORNITORI, Cliente: CLIENTI, Prodotto: PRODOTTI, Quantità)

(La chiave primaria di FORNITURE è la tripla: Fornitore + Cliente + Prodotto).

Caso B: Le Associazioni Uno a Molti (1:M)

In un'associazione 1:M (es. Un *Giocatore* gioca in una *Squadra*, ma una *Squadra* ha molti *Giocatori*), la teoria direbbe di creare tre tabelle. Tuttavia, nella pratica dei database, si fa un'**ottimizzazione obbligatoria** per non sprecare spazio.

- **La Regola della Fusione:** L'associazione **non** diventa una nuova tabella. Invece, viene "assorbita" (inglobata) dalla tabella dell'entità che ha la cardinalità massima pari a 1 (cioè il lato "Uno" della relazione, che nell'E-R solitamente ha la cardinalità (0,1) o (1,1)).
- **Come si fa (Il lato che "assorbe"):**
    1. La tabella dell'entità dal lato 1 si prende tutti gli attributi dell'associazione (se ci sono).
    2. Si prende la Chiave Primaria dell'entità "forte" (dal lato N) e la inserisce al proprio interno come **Chiave Esterna**.

Esempio Pratico (Giocatori e Squadre):

L'associazione *Contratto* (con attributo *Ingaggio*) lega un *Giocatore* (lato 1) a una *Squadra* (lato M). Invece di creare la tabella CONTRATTI, si fonde tutto dentro la tabella dei giocatori:

1. SQUADRE (Nome_Squadra, Città)
2. GIOCATORI (CodiceFiscale, Nome, Ruolo, Squadra: SQUADRE, Ingaggio) *(Nota: Il giocatore "assorbe" il nome della squadra in cui gioca e il valore del suo ingaggio).*

- **Il problema dei Valori Nulli:** Attenzione! Se la cardinalità del lato che assorbe è **(0,1)** (es. un giocatore potrebbe essere momentaneamente svincolato e non avere squadra), gli attributi importati (*Squadra* e *Ingaggio*) conterranno dei valori NULL. È del tutto normale e accettato.
- **Identificatori Esterni:** Se il lato 1 era un'entità debole che usava già l'identificatore esterno della squadra per riconoscersi, non cambia assolutamente nulla nella traduzione: l'attributo importato funge semplicemente da parte della chiave primaria.

Caso C: Le Associazioni Uno a Uno (1:1)

Le associazioni 1:1 sono le più flessibili, perché si comportano come le 1:M, ma **puoi scegliere tu quale delle due entità assorbirà l'associazione**. La scelta si basa interamente sul cercare di **evitare i valori NULL**.

Esempio Pratico (Direzione Dipartimenti):

La Scelta (Dove metto la chiave esterna e l'Anno?):

- *Soluzione A (Ingloba il Dipendente):* DIPENDENTI (Codice, Nome, Dipartimento: DIPARTIMENTI, Anno)
- *Soluzione B (Ingloba il Dipartimento):* DIPARTIMENTI (Nome, Sede, Direttore: DIPENDENTI, Anno)

Partecipazione Obbligatoria (1,1) da entrambi i lati: Scegli a piacere, le due soluzioni (A o B) sono perfettamente identiche ed efficienti.

Caso Misto: (0,1) da un lato e (1,1) dall'altro (Il più frequente)

- **Regola:** L'associazione viene **sempre** assorbita dall'entità con partecipazione **obbligatoria (1,1)**.
- **Il Motivo:** Inserendo la chiave esterna nell'entità obbligatoria, hai la certezza matematica che quel campo sarà sempre valorizzato. Se la mettessi nell'entità opzionale, creeresti colonne piene di campi vuoti (NULL).

Caso Estremo: (0,1) da entrambi i lati

- **Regola: Nessuna fusione.** Devi creare una **terza tabella** a sé stante per l'associazione (esattamente come fai per le M:M).

- **Il Motivo:** Poiché la partecipazione è opzionale per entrambe, fondere l'associazione da una parte o dall'altra genererebbe in ogni caso dei valori NULL. Creando una terza tabella, registri le righe *solo* quando il legame esiste effettivamente, mantenendo le tabelle principali pulitissime.

Operazioni Relazionali e SQL Base

L'Algebra Relazionale è l'insieme delle operazioni matematiche che si possono fare sulle tabelle per estrarre o combinare i dati. L'SQL è semplicemente il linguaggio di programmazione che mette in pratica queste operazioni.

1. Operazioni Insiemistiche Base

Queste operazioni si possono fare **solo se le due tabelle hanno esattamente le stesse colonne** (stesso numero e stesso dominio).

- **Unione:** Prende le righe della Tabella A e le unisce a quelle della Tabella B, creando un'unica grande tabella.
- **Intersezione:** Restituisce **solo** le righe che sono presenti contemporaneamente sia nella Tabella A che nella Tabella B.
- **Differenza:** Restituisce le righe della Tabella A, *togliendo* però quelle che sono presenti anche nella Tabella B.

2. Operazioni di Modifica (Stato del DB)

Sono le operazioni che alterano i dati scritti nel database (in SQL si chiamano operazioni DML). Attenzione all'esame: ogni modifica rischia di violare i famosi Vincoli di Integrità!

- **Insert (Inserimento):** Aggiunge una nuova tupla (riga).
    - *Rischi:* Può violare i vincoli di dominio (es. inserisci lettere al posto di numeri), di chiave primaria (es. inserisci una matricola già esistente) o di integrità referenziale (es. assegni uno studente a un corso che non esiste).
- **Delete (Cancellazione):** Elimina una tupla.
    - *Rischi:* Può violare il vincolo di integrità referenziale (es. cancelli un Dipartimento, ma ci sono ancora impiegati assegnati a quel dipartimento).
- **Update (Modifica):** Cambia i valori di una riga già esistente.
    - *Rischi:* Come per la cancellazione, attenzione ai vincoli referenziali se modifichi le chiavi.

3. Le Operazioni Relazionali

Queste sono le vere operazioni usate per interrogare il database (Query).

A. Proiezione (π) - *Il taglio Verticale*

Sceglie solo **alcune colonne** (attributi) della tabella, scartando le altre.

- *Nota fondamentale per l'esame:* Se, tagliando le colonne, ti rimangono due righe identiche (perché hai tagliato via la chiave primaria che le distingueva), vengono eliminati i doppioni e le **fonde in un'unica tupla**.

B. Selezione (σ) - *Il taglio Orizzontale*

Sceglie solo **alcune righe** (tuple) che soddisfano una precisa condizione logica.

- *Esempio:* Prendi solo le righe in cui l'attributo Età > 18 oppure Città = 'Napoli'.
- **La Query SQL perfetta:** In SQL, una classica interrogazione è formata proprio dall'unione di Proiezione e Selezione: $\pi(_{\sigma_\chi}$ (r)) (Seleziona le righe che rispettano la condizione, e di quelle righe mostrami solo le colonne che mi interessano).

C. Prodotto Cartesiano (X)

Prende due tabelle e combina **ogni riga della prima con ogni riga della seconda**.

- *Il problema dell'omonimia:* Se entrambe le tabelle hanno una colonna chiamata "Nome", si crea confusione. Per risolverla si usa la **Dot Notation** (Notazione col punto): si scrive NomeTabella.NomeAttributo (es. Studenti.Nome vs Professori.Nome).

4. L'Operazione di JOIN (Unire le tabelle in modo intelligente)

Il Prodotto Cartesiano puro genera un sacco di combinazioni inutili. Il JOIN serve per unire due tabelle *solo dove c'è un legame logico.*

- **Theta Join ed Equi Join:** È un prodotto cartesiano seguito da una selezione. Unisce le righe di due tabelle solo se rispettano una condizione (Theta). Se la condizione è di **uguaglianza** (es. Matricola = MatStudente), si chiama specificamente **Equi Join**.
    - **SQL Implicito:** SELECT * FROM r1, r2 WHERE r1.Codice = r2.Codice
    - **SQL Esplicito:** SELECT * FROM r1 JOIN r2 ON r1.Codice = r2.Codice
    - **Join Naturale:** È un Equi Join "pigro". Fa l'uguaglianza in automatico su tutte le colonne delle due tabelle che hanno esattamente lo stesso nome. (Scrivi semplicemente SELECT * FROM Tabella1 NATURAL JOIN Tabella2. Fa tutto da solo: **NON devi** (e non puoi) usare la clausola ON t1.a = t2.b).

Il Problema delle "Tuple Dondolanti" (Dangling Tuples)

Cosa succede se fai un Join tra *Impiegati* e *Mansioni*, ma c'è un impiegato a cui non è stata ancora assegnata nessuna mansione? Quella riga, non trovando corrispondenza nell'altra tabella, viene **scartata e sparisce dal risultato finale**. Questa riga "orfana" viene chiamata tupla dondolante.

ANAGRAFICA_DIPENDENTI

Nome	Cognome	Mansione
carlo	rossi	001
mirko	verdi	001
paolo	paolone	002

MANSIONI

Codice	Qualifica
001	impiegato
002	dirigente
003	amministratore

La Soluzione: Il JOIN ESTERNO (Outer Join)

Se non vuoi perdere le informazioni delle tuple dondolanti, usi il Join Esterno. Il database forza l'unione e riempie i buchi con dei valori **NULL**.

- **Left Join (Sinistro):** Salva e mostra tutte le tuple dondolanti della tabella di *sinistra*.
- **Right Join (Destro):** Salva e mostra tutte le tuple dondolanti della tabella di *destra*.
- **Full Join (Completo):** Salva assolutamente tutto, sia a sinistra che a destra.

5. Ridenominazione (ρ Alias)

Serve a cambiare "al volo" il nome di una tabella o di una colonna, per rendere il risultato più leggibile o per abbreviare query molto lunghe. In SQL si fa usando la parolina **AS**.

- **Ridenominare le Colonne:**

```
SELECT Nome AS Name, Cognome AS Surname FROM ANAGRAFICA_DIPENDENTI
```

- **Ridenominare le Tabelle (comodissimo per la dot notation):**

```
SELECT T1.Nome, T2.Qualifica FROM ANAGRAFICA_DIPENDENTI AS T1, MANSIONI AS T2
WHERE T1.Mansione = T2.Codice
```

SQL Base: La Definizione dei Dati (DDL)

1. Caratteristiche Generali di SQL

- **SQL (Structured Query Language):** È il linguaggio standard universale per dialogare con i database relazionali.
- **È un linguaggio Dichiarativo:** Questo è fondamentale all'esame. Significa che in SQL **tu dici al computer *cosa* vuoi ottenere, non *come* deve farlo**. Non devi scrivere cicli "for" o "if" per cercare i dati riga per riga; scrivi solo il risultato desiderato, e il DBMS trova da solo la strada più efficiente per recuperare le informazioni.
- **Le 3 anime di SQL:** 1. **DDL** (Data Definition Language): Crea e modifica le tabelle (struttura). 2. **DML** (Data Manipulation Language): Inserisce, modifica e cancella i dati (le righe). 3. **DCL** (Data Control Language): Gestisce i permessi e la sicurezza degli utenti.

2. Creare una Tabella (CREATE TABLE)

È il comando principe del DDL. Prende lo Schema Logico che hai fatto su carta e lo fa nascere nel database (creando però una tabella completamente vuota, senza righe).

```
CREATE TABLE NomeTabella (
    NomeAttributo1 Dominio [ValoreDiDefault] [Vincoli_di_Attributo],
    NomeAttributo2 Dominio [ValoreDiDefault] [Vincoli_di_Attributo],
    ...
    [Altri_Vincoli_di_Tabella]
);
```

Le Regole per gli Attributi (Le Colonne)

Quando definisci una colonna, oltre al nome e al tipo di dato (dominio, es. int o char(10)), puoi aggiungere delle regole per controllarne il contenuto:

- **Vietare il vuoto (NOT NULL):** Se per te un dato è obbligatorio, aggiungi questo vincolo. Se un utente proverà a inserire una riga lasciando quel campo vuoto, il sistema bloccherà l'operazione.
    - *Esempio:* CodiceFiscale char(16) NOT NULL
- **Valore predefinito (DEFAULT):** Se l'utente non inserisce nulla, il database compila la cella in automatico con un valore scelto da te.
    - *Esempio:* Ruolo varchar(20) DEFAULT 'Programmatore'

3. Dichiarazione delle Chiavi (Il fulcro della tabella)

A. Chiave Primaria (PRIMARY KEY)

Identifica univocamente ogni riga della tabella.

- *Regola:* Ce ne può essere al massimo UNA per tabella. Non è obbligatorio scriverla, ma nella pratica si fa *sempre*.
- *Nota tecnica:* Non devi aggiungere NOT NULL vicino alla chiave primaria, perché la Primary Key lo include già automaticamente per natura!
- **Come si scrive:**
    - *In linea (se è una sola colonna):* Matricola char(6) PRIMARY KEY
    - *A fine tabella (se è composta da più colonne):* PRIMARY KEY (CodProg, Città)

B. Chiavi Alternative (UNIQUE)

Serve per indicare attributi che devono essere assolutamente univoci (nessun doppione ammissibile), ma che non hai scelto come Chiave Primaria ufficiale.

- *Attenzione all'esame (Trabocchetto Logico):*
    - Scrivere UNIQUE (Cognome, Nome) a fine tabella significa che non possono esistere due "Mario Rossi" identici.
    - Scrivere UNIQUE(Cognome), UNIQUE(Nome) in linea su ogni attributo è molto più restrittivo: significa che può esserci un solo "Rossi" in tutta l'azienda, e un solo "Mario" in tutta l'azienda!

C. Chiavi Esterne (FOREIGN KEY)

Creano i famosi ponti tra le tabelle per l'integrità referenziale. Si usa la parolina magica **REFERENCES** seguita dal nome della tabella di destinazione e (tra parentesi) dal nome della colonna a cui punta.

- **Come si scrive:**
    - *In linea:* Sede Lavoro char(3) REFERENCES Sedi(CodiceSede)
    - *A fine tabella:* FOREIGN KEY (Sede_Lavoro) REFERENCES Sedi(CodiceSede)
- *Scorciatoia:* Se ometti il nome della colonna di destinazione tra le parentesi, il database darà per scontato che tu stia puntando alla Chiave Primaria di quella tabella.

4. Vincoli Arbitrari (CHECK constraint)

A volte i vincoli standard (NOT NULL o UNIQUE) non bastano. Se vuoi mettere una regola tutta tua (es. "Lo stipendio deve essere positivo" o "La data di nascita deve essere nel passato"), usi il comando CHECK.

- **Come funziona:** Il vincolo scatta (e blocca l'operazione) solo se la condizione tra parentesi è **falsa**.
- **Come si scrive:**
    - *Su singola colonna:* Stipendio int CHECK (Stipendio > 0) *(Nota: questo vincolo ammette valori NULL, perché NULL > 0 non dà come risultato "Falso", ma "Sconosciuto", e il CHECK blocca solo il Falso).*
    - *Su più colonne contemporaneamente (si mette a fine tabella):* CHECK (ImportoLordo = Netto + Ritenute)

5. Vincoli con Nome (CONSTRAINT) - *Best Practice*

Quando il database dà un errore perché un utente ha violato una regola, restituisce un codice di errore incomprensibile. Per capire subito dove sta il problema, è buona pratica **dare un nome** ai tuoi vincoli (soprattutto i CHECK e le FOREIGN KEY).

- **Come si fa:** Metti la parolina CONSTRAINT seguita dal nome inventato da te, subito prima del vincolo.
    - *Esempio:* Stipendio int CONSTRAINT StipendioPositivo CHECK (Stipendio > 0)
    - Se l'utente inserirà "-500", a schermo comparirà: *"Errore: Violazione del vincolo 'StipendioPositivo'"*. Capirai subito l'errore senza impazzire!

6. Modificare una Tabella Esistente (ALTER TABLE)

Se hai già creato una tabella ma ti accorgi che manca qualcosa, non devi per forza cancellarla. Usi il comando ALTER TABLE.

- **Cosa puoi fare:** Aggiungere/Rimuovere colonne (ADD COLUMN, DROP COLUMN) oppure Aggiungere/Rimuovere vincoli (ADD CONSTRAINT, DROP CONSTRAINT).
- **Regola d'oro per l'esame (Il trabocchetto del NOT NULL):** Se aggiungi una nuova colonna a una tabella che contiene già dei dati e le metti il vincolo NOT NULL, il database andrà in errore. Perché? Perché le righe vecchie avranno quello spazio vuoto!
    - *Soluzione:* Devi obbligatoriamente fornire un valore predefinito.

- *Esempio corretto:* ALTER TABLE Imp ADD COLUMN Istruzione varchar(10) NOT NULL DEFAULT 'Laurea';

7. Il Data Manipulation Language (DML)

Il DML è l'insieme dei comandi per gestire le "righe" vere e proprie. I comandi base sono quattro:

1. **INSERT**: Aggiunge nuove tuple.
2. **DELETE**: Cancella tuple esistenti (puoi usare la clausola WHERE per cancellare solo righe specifiche).
3. **UPDATE**: Modifica i valori di tuple già esistenti (sempre usando WHERE).
4. **SELECT**: Interroga il database (il comando più importante in assoluto).

8. L'istruzione SELECT (Come interrogare il DB)

La sintassi base di ogni query SQL è formata da tre blocchi fondamentali. Corrisponde all'unione della Proiezione (π) e della Selezione (σ) dell'algebra relazionale.

```
SELECT Attributo1, Attributo2   -- (TARGET LIST: Quali colonne voglio vedere? -> Proiezione)
FROM Tabella1, Tabella2         -- (Da quali tabelle pesco i dati?)
WHERE Condizione;               -- (Quali righe devono essere scelte? -> Selezione)
```

Le Regole della SELECT

- **Selezionare tutto (*):** Se al posto dei nomi delle colonne scrivi SELECT *, il database ti mostrerà tutte le colonne della tabella.
- **Righe Duplicate (Differenza Teorica):** Nell'algebra relazionale i duplicati non esistono. In SQL, invece, una SELECT standard **mostra i duplicati**. Se vuoi eliminarli, devi scrivere esplicitamente SELECT DISTINCT.
- **Espressioni Matematiche:** Nella SELECT puoi fare calcoli al volo (es. SELECT Stipendio * 12).
- **Gli Alias (AS):** Puoi rinominare al volo le colonne nel risultato per renderle più leggibili: SELECT Stipendio * 12 AS StipendioAnnuo.

L'uso degli Pseudonimi (Alias di Tabella)

Quando scrivi query complesse, riscrivere ogni volta il nome intero della tabella è noioso. Puoi assegnare una "lettera" o un nome breve alla tabella direttamente nel FROM (es. FROM Impiegati I). A quel punto userai la notazione col punto: I.CodImp, I.Nome.

9. Gli Operatori Speciali della clausola WHERE

Oltre ai classici =, <, >, SQL ti offre operatori potenti per filtrare le righe:

- **LIKE (Ricerca di pattern nelle stringhe):**
  - _ (Underscore) = Sostituisce un singolo carattere esatto.
  - % (Percentuale) = Sostituisce una stringa di lunghezza qualsiasi (anche vuota).
  - *Esempio:* WHERE Nome LIKE '_i%i' (Prende i nomi che hanno una 'i' come seconda lettera e finiscono per 'i', tipo "G**i**ovann**i**").
- **IN (Appartenenza a un gruppo):** Evita di farti scrivere mille OR.
  - *Invece di:* WHERE Sede = 'S02' OR Sede = 'S03'
  - *Scrivi:* WHERE Sede IN ('S02', 'S03')
- **BETWEEN (Intervalli inclusivi):**
  - *Esempio:* WHERE Stipendio BETWEEN 1300 AND 2000 (Prende chi guadagna tra 1300 e 2000, inclusi).
- **Gestione dei Valori NULL (Logica a 3 Valori):**
  - Non puoi *mai* usare l'uguale con i valori nulli (es. Stipendio = NULL è un errore logico). Per verificare se un campo è vuoto devi usare obbligatoriamente **IS NULL** oppure **IS NOT NULL**.

10. Ordinare i Risultati (ORDER BY)

Mettendo alla fine della query la clausola ORDER BY NomeColonna, il risultato viene messo in ordine alfabetico o numerico.

- **ASC** (Ascendente): È il valore di default, dal più piccolo al più grande.
- **DESC** (Discendente): Ordine inverso, dal più grande al più piccolo.

11. Interrogazioni su Più Tabelle (I JOIN in SQL)

Quando i dati che ti servono sono "sparpagliati" in tabelle diverse, devi usare l'operazione di JOIN per ricomporli. Nei JOIN si usano quasi sempre gli **Alias di tabella** (es. studente s, esame e) per rendere il codice più breve ed evitare ambiguità quando due tabelle hanno colonne con lo stesso nome (es. s.id e e.id).

A. Inner Join (L'intersezione perfetta)

È il Join più usato. Restituisce **solo** le righe che trovano un compagno in *entrambe* le tabelle collegate. Se una riga è "orfana", viene scartata.

```sql
-- Es: Mostra solo gli studenti che hanno sostenuto almeno un esame
SELECT s.nome, s.cognome, c.nome AS corso, e.voto
FROM esame e
  INNER JOIN studente s ON e.studente_id = s.id
  INNER JOIN corso c ON e.corso_id = c.id
ORDER BY s.cognome, c.nome;
```

B. Left Join (Il recupero delle orfane)

Restituisce **tutte** le righe della tabella di sinistra (quella scritta dopo il FROM), anche se non trovano una corrispondenza nella tabella di destra. I buchi della tabella di destra vengono semplicemente riempiti con NULL.

```sql
-- Es: Mostra TUTTI gli studenti e il numero di esami (anche chi ne ha fatti 0)
SELECT s.nome, s.cognome, COUNT(e.id) AS num_esami
FROM studente s
  LEFT JOIN esame e ON e.studente_id = s.id
GROUP BY s.id, s.nome, s.cognome;
```

- **Il trucco dell'Anti-Join (Domanda da Esame):** Unendo il LEFT JOIN con un WHERE ... IS NULL, trovi tutte le righe della tabella di sinistra che **NON** hanno corrispondenza a destra.

```sql
-- Es: Trova gli studenti "fantasma" che NON hanno mai fatto esami
SELECT s.nome, s.cognome
FROM studente s
  LEFT JOIN esame e ON e.studente_id = s.id
WHERE e.id IS NULL;
```

C. Right Join e Full Outer Join

- **Right Join:** È esattamente speculare al Left Join (salva le orfane della tabella di destra). Nella pratica si usa pochissimo: basta invertire l'ordine delle tabelle e usare un più comodo Left Join.
- **Full Outer Join:** È l'unione totale. Salva tutte le orfane sia di sinistra che di destra. *Nota tecnica:* MySQL non lo supporta nativamente; per simularlo bisogna fare un LEFT JOIN, scrivere UNION, e poi attaccarci un RIGHT JOIN.

D. Self Join (La tabella che unisce se stessa)

Si verifica quando una tabella fa riferimento a se stessa (Associazioni Ricorsive). In questo caso, l'uso degli **Alias è obbligatorio** per sdoppiare la tabella e far capire al sistema quale ruolo sta giocando ogni riga.

```sql
-- Es: Tabella Dipendenti, dove alcuni sono Manager di altri. Trova il capo di ciascuno.
SELECT d.nome AS dipendente, m.nome AS manager
FROM dipendente d
  LEFT JOIN dipendente m ON d.manager_id = m.id;
```

12. Il Data Manipulation Language (DML): Inserire, Modificare e Cancellare

Il DML comprende i comandi che alterano le righe dentro le tabelle (non la struttura, ma i dati veri e propri). *Nota d'esame:* Ogni singola operazione di INSERT, DELETE o UPDATE agisce **sempre e solo su una tabella alla volta**. Non puoi aggiornare o cancellare dati da due tabelle contemporaneamente con un solo comando.

A. Inserimento di Tuple (INSERT)

Aggiunge nuove righe alla tabella. Puoi farlo in due modi:

1. **Inserimento Singolo (Valori espliciti):** Specifichi le colonne che vuoi riempire e, subito dopo, i valori esatti.

```sql
INSERT INTO Sedi (Sede, Responsabile, Citta)
VALUES ('S04', 'Bruni', 'Firenze');
```

   - *Regola 1:* Ci deve essere corrispondenza perfetta (1 a 1) tra l'elenco delle colonne e l'elenco dei valori.
   - *Regola 2:* Se ometti l'elenco delle colonne (es. INSERT INTO Sedi VALUES(...)), devi inserire i valori rispettando l'ordine esatto con cui la tabella è stata creata.
   - *Regola 3:* Se scrivi solo alcune colonne, quelle omesse verranno riempite in automatico con NULL (se il vincolo lo permette) o con il valore DEFAULT (se lo avevi impostato).

2. **Inserimento Multiplo (Da una Query):** Prende i risultati di una SELECT e li "travasa" in un'altra tabella. È comodissimo per creare copie o archivi.

```sql
INSERT INTO SediBologna (SedeBO, Resp)
SELECT Sede, Responsabile FROM Sedi WHERE Citta = 'Bologna';
```

   - *Regola:* I nomi delle colonne tra le due tabelle possono essere diversi, l'importante è che i **tipi di dato** (domini) siano compatibili (es. non puoi travasare un testo in una colonna numerica).

B. Cancellazione di Tuple (DELETE)

Elimina intere righe dalla tabella.

- Si usa **sempre** insieme alla clausola WHERE per specificare *quali* righe cancellare.
- *Attenzione al disastro:* Se scrivi DELETE FROM Sedi; senza il WHERE, il database svuoterà completamente la tabella!

```
-- Cancella solo le sedi di Bologna
DELETE FROM Sedi WHERE Citta = 'Bologna';
```

C. Modifica di Tuple (UPDATE)

Cambia i valori delle celle di una o più righe già esistenti. Usa la parola chiave SET per impostare i nuovi valori.

- Anche l'UPDATE va usato con il WHERE, altrimenti modificherai tutte le righe della tabella in un colpo solo.

```
-- Esempio 1: Cambia nome e città per una sede specifica
UPDATE Sedi SET Responsabile = 'Bruni', Citta = 'Firenze' WHERE Sede = 'S01';

-- Esempio 2: Aumenta lo stipendio del 10% a tutti i programmatori
UPDATE Imp SET Stipendio = 1.1 * Stipendio WHERE Ruolo = 'Programmatore';
```

13. Le Politiche di Reazione (ON DELETE / ON UPDATE)

Questo è un concetto fondamentale per l'esame. Cosa succede se faccio un DELETE su un dipartimento, ma c'erano ancora degli impiegati assegnati a quel dipartimento? Il **Vincolo di Integrità Referenziale** viene violato!

Per evitare che il sistema vada in panico o che il programmatore debba fare mille controlli manuali, si impostano le **Politiche di Reazione** direttamente durante il CREATE TABLE, quando si definisce la chiave esterna (FOREIGN KEY).

Dicono al database: *"Se qualcuno cancella o modifica la riga Padre a cui sono collegato, tu comportati così:"*

Le 4 Politiche (Come reagisce il database)

1. **NO ACTION (o RESTRICT) - Il blocco di sicurezza** È il comportamento di default. Se provi a cancellare il dipartimento, il database ti lancia un errore e blocca l'operazione. Prima devi spostare a mano gli impiegati, poi potrai cancellare il dipartimento.

2. **CASCADE (La reazione a catena)** Molto potente ma pericolosa. Se cancelli il dipartimento Padre, il database **cancella in automatico** tutti gli impiegati collegati a quel dipartimento. Se aggiorni il codice del dipartimento, si aggiornerà in automatico anche la chiave esterna di tutti gli impiegati.
3. **SET NULL (Lo scollegamento)** Se cancelli il dipartimento Padre, gli impiegati non vengono cancellati, ma il loro campo "Dipartimento" viene svuotato e impostato a NULL (diventano "svincolati"). Ovviamente, la colonna non deve avere il vincolo NOT NULL.
4. **SET DEFAULT (Il paracadute)** Se cancelli il dipartimento Padre, gli impiegati vengono riassegnati automaticamente a un dipartimento di "default" (es. "In Attesa di Assegnazione") che hai specificato quando hai creato la colonna.

Come si scrive in SQL:

```
CREATE TABLE Impiegati (
    CodImp char(4) PRIMARY KEY,
    Sede_Lavoro char(3),
    -- Definizione del ponte e delle reazioni:
    FOREIGN KEY (Sede_Lavoro) REFERENCES Sedi(Sede)
    ON DELETE SET NULL     -- Se cancellano la sede, metti a NULL
    ON UPDATE CASCADE      -- Se cambiano il codice sede, aggiornalo anche qui
);
```

14. Le Funzioni Aggregate (o "di Colonna")

A differenza delle normali operazioni SQL che lavorano riga per riga, le funzioni aggregate prendono un'intera colonna (o un gruppo di righe), le "macinano" tutte insieme e **restituiscono un unico valore finale**.

Le 7 funzioni standard:

- MIN() e MAX(): Valore minimo e massimo.
- SUM(): Somma totale.
- AVG(): Media aritmetica.
- STDEV() e VARIANCE(): Deviazione standard e Varianza (statistica).
- COUNT(): Conteggio di quante righe ci sono.

Regole d'oro per l'esame sulle Funzioni Aggregate

1. **Ignorano i valori vuoti (NULL):** Tutte queste funzioni (tranne una eccezione che vediamo dopo) scartano in automatico le celle vuote prima di fare il calcolo. Se l'intera colonna è vuota, il risultato finale sarà NULL.

2. **Possono fare calcoli al volo:** Dentro le parentesi puoi mettere espressioni matematiche (es. SUM(Stipendio * 12)), ma **NON puoi mai** mettere un'altra funzione aggregata dentro un'altra (es. MAX(SUM(Stipendio)) è un errore fatale).
3. **Uso di DISTINCT:** Se vuoi, ad esempio, sommare gli stipendi ma contando una sola volta gli stipendi identici, usi la parolina magica: SUM(DISTINCT Stipendio).
4. **Il trucco del Casting:** A volte la media (AVG) di numeri interi restituisce un numero intero tagliando i decimali. Per avere la precisione, si forza il tipo di dato con un CAST: SELECT AVG(CAST(Stipendio AS Decimal(6,2))).

Il Trabocchetto del COUNT (Domanda classica)

C'è una differenza abissale tra usare l'asterisco o il nome della colonna:

- **COUNT(*)**: Conta fisicamente **tutte le righe** (tuple) della tabella, comprese quelle che hanno valori NULL. È il contatore assoluto.
- **COUNT(NomeColonna)**: Conta **solo le righe in cui quella specifica colonna NON è vuota**.
    - *Esempio d'esame:* Se hai 10 impiegati, ma solo 8 hanno il cellulare registrato: COUNT(*) ti darà 10, mentre COUNT(Cellulare) ti darà 8.

🚨 **L'Errore Sintattico più grave in assoluto**

Se usi una funzione aggregata nella SELECT, **NON PUOI** inserire nomi di colonne normali accanto ad essa.

- **ERRORE:** SELECT Nome, MIN(Stipendio) FROM Impiegati;
    - *Perché è sbagliato?* Il DB va in tilt: la colonna Nome genera un elenco di 100 nomi (100 righe), mentre MIN(Stipendio) genera un singolo numero (1 riga). Il sistema non sa come allinearli in una tabella.
- **CORRETTO:** O metti solo funzioni aggregate (SELECT MIN(Stipendio), MAX(Stipendio)...), oppure usi il **GROUP BY**.

15. Raggruppamento e Filtro dei Gruppi (GROUP BY e HAVING)

Quando usi le funzioni aggregate (SUM, AVG, COUNT), spesso non vuoi un unico risultato globale, ma vuoi calcolarlo per **sotto-categorie** (es. lo stipendio medio *per ogni sede* o *per ogni ruolo*). Qui entra in gioco il GROUP BY.

A. Come ragiona il Database (Ordine di Esecuzione)

Per non sbagliare mai una query all'esame, devi pensare esattamente come pensa il motore SQL. L'ordine in cui scrivi le clausole è fisso, e l'ordine in cui il DB le esegue è questo:

1. **FROM / JOIN:** Prende le tabelle e le unisce.
2. **WHERE (Filtro sulle righe):** Scarta le singole tuple che non rispettano la condizione.
3. **GROUP BY (Creazione dei secchielli):** Prende le righe sopravvissute e le raggruppa dividendole per valori uguali (es. crea un gruppo per "Milano" e uno per "Roma").
4. **HAVING (Filtro sui secchielli):** *Novità!* Scarta interi gruppi se non rispettano una determinata condizione (es. butta via il gruppo "Roma" se ha meno di 3 impiegati).
5. **SELECT (Sintesi e Calcolo):** Applica la funzione aggregata (es. calcola la media) per ogni gruppo sopravvissuto.
6. **ORDER BY (Ordinamento):** Mette in ordine alfabetico o numerico il risultato finale.

B. La clausola HAVING: Differenza cruciale con WHERE

Questo è il classico **trabocchetto da esame**. Entrambe servono a filtrare, ma lavorano in momenti diversi e su "oggetti" diversi:

- **WHERE filtra le singole righe *PRIMA* del raggruppamento.** (Non può **mai** contenere funzioni aggregate come COUNT o SUM).
- **HAVING filtra gli interi gruppi *DOPO* il raggruppamento.** (Si usa quasi esclusivamente per fare test sulle funzioni aggregate).

Esempio di differenza logica: Se voglio analizzare solo le sedi che hanno *più di 2 impiegati*:

- ❌ *Sbagliato:* WHERE COUNT(*) > 2 (Il DB va in errore, non ha ancora raggruppato nulla!)
- ✅ *Corretto:* HAVING COUNT(*) > 2 (Il DB prima raggruppa le sedi, poi conta chi c'è dentro, e infine scarta i gruppi troppo piccoli).

Nota tecnica: Puoi mettere nell'HAVING anche condizioni normali sulle colonne di raggruppamento (es. HAVING Sede <> 'S01'), ma per efficienza è sempre meglio metterle nel WHERE (WHERE Sede <> 'S01'), così il database scarta la sede 'S01' subito e non spreca tempo a raggrupparla.

C. Livelli di Raggruppamento (Esempi Pratici)

Puoi raggruppare per una o più colonne. Più colonne aggiungi al GROUP BY, più i "secchielli" diventano piccoli e specifici.

1. Raggruppamento Singolo (Per Ruolo): Voglio lo stipendio medio a Milano, diviso per Ruolo.

```
SELECT I.Ruolo, AVG(I.Stipendio) AS MediaStip
FROM Imp I INNER JOIN Sedi S ON (I.Sede = S.Sede)
WHERE S.Citta = 'Milano'
GROUP BY I.Ruolo;
```

2. Raggruppamento Multiplo (Per Sede E per Ruolo): Voglio lo stipendio medio a Milano, diviso per singola Sede e, all'interno di ogni sede, diviso per Ruolo.

```
SELECT I.Sede, I.Ruolo, AVG(I.Stipendio) AS MediaStip
FROM Imp I INNER JOIN Sedi S ON (I.Sede = S.Sede)
WHERE S.Citta = 'Milano'
GROUP BY I.Sede, I.Ruolo;
```

D. Il trucco GROUP BY = DISTINCT

Se scrivi una query con il GROUP BY, ma nella SELECT ti limiti a chiedere solo il nome del gruppo senza applicare nessuna funzione aggregata (senza COUNT o AVG), ottieni semplicemente un elenco senza doppioni.

```
--In pratica, scrivere:
SELECT Sede
FROM Imp
GROUP BY Sede;
-- è l'esatto equivalente logico di scrivere
SELECT DISTINCT Sede FROM Imp;
```

16. Subquery (Interrogazioni Innestate)

Una subquery non è altro che una SELECT scritta all'interno di un'altra SELECT (di solito dentro la clausola WHERE). Serve a calcolare dei valori "al volo" da usare come filtro per la query principale.

Esistono **tre grandi famiglie** di subquery, e per ognuna devi usare operatori ben precisi.

A. Subquery Scalari (Restituiscono 1 solo valore)

Si hanno quando sei **matematicamente certo** che la subquery restituirà al massimo una singola cella (una riga e una colonna).

- **Operatori ammessi:** Puoi usare i classici operatori matematici: =, <, >, <=, >=.
- **Quando si usano:** Spesso in combinazione con le funzioni aggregate o con la Chiave Primaria.
- **Esempio classico (Chi guadagna meno di tutti?):**

```
SELECT CodImp FROM Imp
WHERE Stipendio = (SELECT MIN(Stipendio) FROM Imp);
```

Errore fatale d'esame: Se usi il simbolo = ma la subquery dovesse per caso restituire due o più righe, il database va in crash!

B. Subquery Multi-valore (Caso Generale)

Se la subquery restituisce una colonna con **più di un valore** (una lista), non puoi usare il semplice uguale. Devi usare questi operatori speciali:

- **IN**: Verifica se il valore esterno è presente nella lista restituita dalla subquery.
- **ANY**: La condizione deve essere vera per *almeno uno* dei valori della lista. *(Nota: Scrivere = ANY è l'esatto equivalente logico di scrivere IN).*
- **ALL**: La condizione deve essere vera per *tutti* i valori della lista.
    - *Esempio (Chi ha lo stipendio più basso? Alternativa ad aggregazione):* WHERE Stipendio <= ALL (SELECT Stipendio FROM Imp) -> *(Traduzione: "Il mio stipendio è minore o uguale a quello di tutti gli altri").*

C. L'Operatore EXISTS (Il Test di Esistenza)

A EXISTS non interessa quali dati vengono restituiti, gli interessa solo **se la subquery produce o meno delle righe**.

- **EXISTS**: È vero se la subquery restituisce *almeno una* tupla.
- **NOT EXISTS**: È vero se la subquery è completamente vuota.
- *Nota sintattica:* Di solito dentro un EXISTS si scrive SELECT * o SELECT 1, tanto al database non importa cosa selezioni, verifica solo l'esistenza della riga.

15. Subquery Correlate vs Non Correlate

Questa è la distinzione teorica più importante.

- **Subquery NON Correlata (Statica):** È completamente indipendente dalla query esterna. Il database calcola la subquery *una sola volta* all'inizio, ottiene la sua lista di risultati e poi esegue la query principale. (Molto veloce).

- **Subquery CORRELATA (Dinamica):** La subquery fa riferimento a una variabile (un Alias) che appartiene alla tabella del blocco esterno.
    - *Come ragiona il DB:* Prende la prima riga della query esterna, "passa" il valore alla subquery, esegue la subquery e vede se è vero. Poi passa alla seconda riga esterna e **riesegue la subquery da capo**. Lo fa per ogni singola tupla! (Lento e costoso).

Esempio di Subquery Correlata (con EXISTS):

```
SELECT Sede FROM Sedi S
WHERE EXISTS (
    SELECT * FROM Imp I
    WHERE I.Ruolo = 'Programmatore' AND I.Sede = S.Sede -- <-- Ecco la correlazione!
);
```

16. Unnesting: Trasformare le Subquery in JOIN

Molti professori chiedono di scrivere la stessa query in due modi: prima in forma "innestata" (Subquery) e poi in forma "piatta" (Unnesting, cioè convertendola con dei JOIN).

Regola 1: Da EXISTS / IN a INNER JOIN

Per trovare le sedi che hanno programmatori, la forma innestata usa EXISTS. Per trasformarla in forma piatta, fai un normalissimo INNER JOIN.

- 🚨 **Il trucco del DISTINCT:** Poiché una sede ha tanti programmatori, l'Inner Join duplicherebbe il nome della sede tante volte quanti sono i suoi programmatori. Per avere un risultato identico alla subquery, **devi ricordarti di usare DISTINCT!**

```
SELECT DISTINCT S.Sede
FROM Sedi S INNER JOIN Imp I ON (S.Sede = I.Sede)
WHERE I.Ruolo = 'Programmatore';
```

Regola 2: Da NOT EXISTS a LEFT JOIN (L'Anti-Join)

Questa è la conversione più difficile e a rischio errore. Come trovi le sedi **SENZA** programmatori in forma piatta? Devi usare un LEFT JOIN unito al trucco dell'IS NULL.

- Fai il LEFT OUTER JOIN inserendo le condizioni direttamente nella clausola ON.
- Nel WHERE, metti **solo** il controllo IS NULL sulla chiave primaria della tabella di destra.

```
SELECT DISTINCT S.Sede
FROM Sedi S
     LEFT OUTER JOIN Imp I ON (S.Sede = I.Sede AND I.Ruolo = 'Programmatore')
WHERE I.CodImp IS NULL; -- Se è NULL, vuol dire che l'incastro a destra è fallito (non ha
programmatori)
```

17. Le Viste (View)

Una Vista è una "tabella fittizia". È un'interfaccia comodissima che salva una query complessa e ti permette di usarla come se fosse una tabella vera e propria.

- **Perché si usano?** 1. Per non dover riscrivere ogni volta query chilometriche usate spesso. 2. Per motivi di sicurezza (es. crei una vista per le segretarie che mostra i dati dei dipendenti, ma nasconde la colonna "Stipendio").
- **Il concetto di Virtuale:** I dati di una vista **non sono salvati sul disco**. Ogni volta che interroghi la vista, il database va a pescare "in diretta" i dati dalle tabelle originali (tabelle di base).
    - *Eccezione:* La MATERIALIZED VIEW. In questo caso, il database fa una "fotografia" dei dati e li salva fisicamente sul disco per velocizzare la lettura, ma andranno aggiornati manualmente se le tabelle originali cambiano.
- **Limiti:** Proprio perché non esistono fisicamente, le operazioni di aggiornamento (INSERT, UPDATE, DELETE) fatte "attraverso" una vista sono molto limitate e sconsigliate.

Come si crea:

```
-- Salvo una query complessa dandole il nome "Impiegati_Dieti"
CREATE VIEW IMPIEGATI_DIETI AS
SELECT * FROM ANAGRAFICA WHERE NomeDip = 'DIETI';

-- Ora posso usarla come fosse una tabella normale!
SELECT Nome, Cognome FROM IMPIEGATI_DIETI;
```

Il Data Control Language (DCL): Sicurezza e Permessi

Il DCL è la parte di SQL utilizzata dal **DBA** (Data Base Administrator) per implementare le politiche di sicurezza, decidendo chi può accedere al database e quali operazioni è autorizzato a compiere.

1. Il Livello Base: L'Utente

Il DBMS ha sempre un utente "Amministratore" predefinito (che può fare tutto). Per far entrare altre persone nel sistema, il primissimo passo è creare le loro credenziali di accesso:

```
CREATE USER 'username' IDENTIFIED BY 'password';
```

2. Definizione Teorica di Privilegio (Domanda d'esame)

Una volta creato l'utente, il sistema è chiuso di default: non può fare nulla se non riceve dei permessi specifici.

Dal punto di vista formale, un privilegio P non è un semplice permesso, ma è definito come una **quintupla logica**:

$$P = \langle R, U_1, U_2, A, T \rangle$$

- R **(Risorsa):** L'oggetto del database su cui si applica il permesso (es. una Tabella, una Vista, o un intero Schema).
- U_1 **(Concedente):** L'utente autorizzato che *concede* il privilegio (di solito il DBA).
- U_2 **(Ricevente):** L'utente che *ottiene* il privilegio.
- A **(Azioni):** L'insieme di operazioni permesse su quella specifica risorsa.
- T **(Trasmissibilità / Grant Option):** L'autorizzazione al "passaparola". Se è attiva, l'utente U_2 potrà a sua volta regalare questo privilegio ad altri utenti.

3. L'Arsenale delle Azioni (Cosa può valere A?)

Ecco i principali permessi che il DBA può assegnare:

Permessi sui Dati (Righe):

- SELECT: Leggere i dati.
- INSERT: Aggiungere nuove righe.
- UPDATE: Modificare i valori delle righe esistenti.
- DELETE: Cancellare righe.

Permessi sulla Struttura:

- CREATE: Definire nuove istanze/tabelle.
- DROP: Distruggere/rimuovere istanze.

Macro-Permessi (Scorciatoie):

- CONNECT: Il permesso di base assoluto. Serve semplicemente per potersi connettere al DBMS.
- RESOURCE: Un pacchetto completo che permette di creare, modificare e rimuovere risorse in uno schema.
- ALL PRIVILEGES: Dà letteralmente ogni potere possibile su quella risorsa.

4. Gestione Pratica: GRANT e REVOKE

I due comandi che traducono in codice la formula vista sopra.

A. Concedere un privilegio (GRANT)

Dà i permessi all'utente. Se vuoi attivare la trasmissibilità (la lettera T della formula), devi aggiungere l'opzione finale.

```
GRANT select, update ON NomeRisorsa TO username [WITH GRANT OPTION];
```

B. Ritirare un privilegio (REVOKE)

Toglie i permessi precedentemente dati.

- *Il Trabocchetto Logico d'esame:* Cosa succede se tolgo un permesso a Marco, ma Marco aveva usato la *Grant Option* per darlo anche a Luigi? Qui entrano in gioco le politiche di revoca:

  ```
  REVOKE select, update ON NomeRisorsa FROM username [RESTRICT | CASCADE];
  ```

  CASCADE: Effetto domino. Il sistema revoca il permesso a Marco e, in automatico, lo revoca anche a Luigi e a chiunque altro lo abbia ricevuto tramite Marco.

- **RESTRICT**: Misura di sicurezza. Il sistema blocca la revoca e ti dà errore finché non vai a togliere manualmente i permessi a Luigi.

5. I Ruoli (Role): Semplificarsi la vita

Se devi gestire 100 impiegati che fanno tutti lo stesso lavoro, fare 100 GRANT separate sarebbe folle. In SQL si usa il concetto di **Ruolo (Profilo)**.

- **Come funziona in 3 passi:** 1. Si crea un "contenitore" vuoto (il ruolo). 2. Si caricano tutti i privilegi dentro il contenitore. 3. Si assegna il contenitore agli utenti, che ereditano tutto istantaneamente.

Esempio Pratico e Completo (Giornalisti vs Tifosi):

```
-- 1. Creazione dei Ruoli
CREATE ROLE 'giornalista';
CREATE ROLE 'tifoso';

-- 2. Assegnazione dei privilegi ai Ruoli
GRANT connect, update ON CAMPIONATO.* TO 'giornalista';
GRANT connect, select ON CAMPIONATO.GIOCATORI TO 'tifoso';

-- 3. Assegnazione dei Ruoli agli utenti fisici
-- Paola e Carla ereditano in automatico connect e update
GRANT 'giornalista' TO 'paola';
GRANT 'giornalista' TO 'carla';

-- Vinni e Antonio ereditano solo connect e select
GRANT 'tifoso' TO 'vinni';
GRANT 'tifoso' TO 'antonio';
```

La Normalizzazione e la Qualità del Database

1. Cos'è la Normalizzazione?

La Normalizzazione è una procedura formale che serve a **verificare la qualità** di un database. Si basa sul rispetto delle cosiddette "Forme Normali" (regole matematiche).

- **A cosa serve:** Serve ad assicurarsi che le tabelle non presentino ridondanze (dati ripetuti inutilmente) e che non creino problemi durante l'inserimento, la modifica o la cancellazione dei dati.
- **Cosa NON è (Attenzione all'esame!):** La normalizzazione **non** è un metodo per progettare database partendo da zero. Per quello si usa lo schema E-R. Di solito, se fai un ottimo schema E-R, le tabelle che ne derivano sono *già* in forma normale. La normalizzazione si usa quindi "a posteriori" come **test di controllo qualità** sullo schema logico finale.

2. La Causa di tutti i mali: Tabelle Disomogenee

Il problema principale nasce quando cerchi di "risparmiare spazio" (o progetti male) e **unisci concetti completamente diversi all'interno della stessa tabella**.

PROGETTAZIONI

Impiegato	Stipendio	Progetto	Bilancio	Funzione
Rossi	20	Marte	2	tecnico
Verdi	35	Giove	15	progettista
Verdi	35	Venere	15	progettista
Neri	55	Venere	15	direttore
Neri	55	Giove	15	consulente
Neri	55	Marte	2	consulente
Mori	48	Marte	2	direttore
Mori	48	Venere	15	progettista
Bianchi	48	Venere	15	progettista
Bianchi	48	Giove	15	direttore

In questa unica tabella stai mischiando tre entità che nella realtà sono distinte:

1. Gli **Impiegati** (che hanno un loro stipendio fisso, a prescindere da cosa fanno).
2. I **Progetti** (che hanno un loro bilancio fisso).
3. Le **Partecipazioni** (il ruolo specifico che un impiegato ha in un dato progetto).

Questa "fusione forzata" genera un database malato, affetto da **4 gravi problemi strutturali**.

A. Ridondanza (Spreco di spazio)

È la presenza di dati ripetuti in diverse righe senza che aggiungano alcuna nuova informazione.

- *Esempio pratico:* L'impiegato "Verdi" lavora su due progetti diversi. Nella tabella, il suo stipendio (35) viene scritto due volte. Il progetto "Venere" coinvolge tre persone, quindi il suo bilancio (15) viene scritto inutilmente tre volte. Se un progetto avesse 1000 impiegati, il suo bilancio verrebbe copiato 1000 volte.

B. Anomalia di Aggiornamento (Incoerenza dei dati)

Nasce direttamente dalla ridondanza. Se un dato è ripetuto e cambia nel tempo, devi ricordarti di andarlo a modificare in tutte le righe in cui compare. Se te ne dimentichi una, il database "si rompe" a livello logico.

- *Esempio pratico:* L'impiegato "Neri" riceve un aumento e il suo stipendio passa da 55 a 60. Poiché lavora su 3 progetti, devi andare a cercare e modificare 3 righe diverse. Se per un errore di distrazione aggiorni solo 2 righe, il database dirà contemporaneamente che "Neri" guadagna 60 e che guadagna 55.

C. Anomalia di Cancellazione (Perdita involontaria di dati)

Avviene quando, cancellando un'informazione che non è più vera, cancelli per sbaglio anche altre informazioni che invece sono ancora valide.

- *Esempio pratico:* L'impiegato "Rossi" lavora unicamente al progetto "Marte". Se il progetto "Marte" finisce, tu cancelli la riga di Rossi. Il problema è che in quella riga c'era anche il suo Stipendio! Così facendo, non solo hai tolto Rossi dal progetto, ma hai letteralmente cancellato l'impiegato Rossi e il suo stipendio da tutto il database dell'azienda.

D. Anomalia di Inserimento (Dati bloccati)

Avviene quando sei impossibilitato ad inserire un nuovo dato nel sistema perché ti manca un pezzo di informazione non strettamente correlata (spesso perché parte della chiave primaria non può essere NULL).

- *Esempio pratico:* L'azienda assume un nuovo impiegato ("Gialli", con stipendio 30). Tuttavia, il primo giorno di lavoro non gli è ancora stato assegnato nessun progetto. In questa tabella non puoi registrarlo! Poiché il Progetto è fondamentale per definire la riga (fa parte della chiave), non puoi inserire l'impiegato Gialli nel sistema finché non gli trovi un progetto.

La Dipendenza Funzionale (DF)

1. Cos'è in parole povere?

Una Dipendenza Funzionale è un **vincolo di integrità logico** tra due gruppi di attributi (colonne) all'interno di una tabella.

- **La Notazione:** Si scrive $Y \rightarrow Z$ e si legge *"Y determina Z"* oppure *"Z dipende funzionalmente da Y"*.
- **Cosa significa in pratica?** Significa che se io conosco il valore della colonna Y, conosco in automatico e senza alcuna ambiguità il valore della colonna Z.

2. La Definizione Formale

Consideriamo una relazione r e due gruppi di attributi Y e Z. Esiste una dipendenza funzionale $Y \rightarrow Z$ se e solo se:

$$\forall t_1, t_2 \in r: t_1[Y] = t_2[Y] \Rightarrow t_1[Z] = t_2[Z]$$

Traduzione pratica: Per ogni coppia di tuple (righe) della tabella, se le due righe hanno **lo stesso identico valore nella colonna Y**, allora **devono obbligatoriamente avere lo stesso valore anche nella colonna Z**.

3. Esempi Pratici (Sulla tabella PROGETTAZIONI malata)

Riprendiamo la tabella di prima: PROGETTAZIONI(Impiegato, Stipendio, Progetto, Bilancio, Funzione).

Quali sono i legami logici (le DF) nascosti qui dentro?

1. **Impiegato $\rightarrow$ Stipendio**

Ogni impiegato ha un solo stipendio. Se leggo "Rossi" nella colonna Impiegato, so di per certo che lo stipendio associato sarà sempre e solo "20". Non può essere 20 in una riga e 50 in un'altra.

2. **Progetto $\rightarrow$ Bilancio**

Ogni progetto ha un budget fisso. Se il progetto è "Marte", il suo bilancio è obbligatoriamente "2", a prescindere da chi ci stia lavorando.

3. **Impiegato, Progetto $\rightarrow$ Funzione**

Qui la DF ha due attributi a sinistra. Per sapere che ruolo ha una persona, mi servono due informazioni: chi è e a che progetto lavora. Se so che l'impiegato è "Neri" e il progetto è "Venere", so con certezza assoluta che la sua funzione in quel contesto è "Direttore".

4. Dipendenze Banali vs Non Banali

Quando cerchi le DF in uno schema, devi scartare quelle "Banali", ovvero quelle ovvie che non aggiungono nessuna informazione utile al database.

- **DF Banale:** $Y \rightarrow A$ è banale se l'attributo A fa già parte dell'insieme Y.
    - *Esempio:* $Impiegato, Progetto \rightarrow Progetto$. (Grazie al ca...! Se so che il progetto è Marte, so che il progetto è Marte. Inutile).
- **DF Non Banale (Quelle che ci interessano):** $Y \rightarrow Z$ è non banale **solo se** nessun attributo che sta a destra (Z) compare anche nella parte sinistra (Y).

VINCOLI E DIPENDENZE FUNZIONALI

È una regola dello Schema (Aspetto Intensionale): Una dipendenza funzionale

($Y \rightarrow Z$) è un vincolo stabilito a priori dal progettista in base al *significato reale* dei dati (la semantica). Non cambia mai, esattamente come la struttura della tabella.

Non si indovina dall'Istanza (Aspetto Estensionale): Non puoi scoprire una DF guardando semplicemente le righe scritte nella tabella in un dato momento.

L'Istanza Legale

Un'istanza di tabella (cioè l'insieme delle righe inserite in un dato momento) si dice **Legale** se e solo se rispetta rigorosamente tutte le DF definite per quello schema.

- *In pratica:* Se hai stabilito che $Impiegato \rightarrow Stipendio$, e qualcuno inserisce due righe dove "Rossi" guadagna prima 20 e poi 50, l'istanza diventa *illegale* e il database va in stato di errore (anomalia).

CHIAVI E ATTRIBUTI

Data una relazione **R(X)** :

- un attributo che partecipa almeno ad una chiave di $R(X)$ è detto **attributo primo**
- un attributo che non partecipa ad alcuna chiave di $R(X)$è detto **attributo non primo**
- Un gruppo di attributi Kè una superchiave dello schema $R(X)$ se e solo se determina funzionalmente l'intera riga (X). $[K \rightarrow X]$

(Se conosco la chiave K, conosco automaticamente il valore di tutte le altre colonne della tabella).

Di conseguenza, se K è la chiave, tutti gli attributi "non primi" della tabella dipendono funzionalmente da essa.

DF Biunivoche

Cosa succede se la freccia va in entrambe le direzioni?

Se hai una regola per cui $Y \to Z$ e contemporaneamente $Z \to Y$, allora se Yè una chiave della tabella **automaticamente anche Z è una chiave**. Sono di fatto interscambiabili (es. Matricola $\leftrightarrow$ Codice Fiscale).

Il Teorema: Vincoli di Chiave e Dipendenze Funzionali (DF)

Dato uno schema relazionale $R(X)$, consideriamo K (un sottoinsieme di attributi) e T(tutti gli altri attributi rimanenti, calcolati come $T = X - K$).

Il teorema afferma che:

$$K \text{ è superchiave di } R(X) \iff K \to T$$

1. Se $K \to T$, allora K è una superchiave ($\Leftarrow$)

Se $K \to T$ allora per ogni istanza legale (tabella) r, se prendiamo due righe qualsiasi t_1 e t_2 che hanno lo stesso valore per gli attributi K, dovranno per forza avere lo stesso valore anche per gli attributi T. Ovvero:

$$t_1[K] = t_2[K] \Rightarrow t_1[T] = t_2[T]$$

- Se due righe sono identiche nella parte K e sono identiche anche nella parte T, allora stiamo guardando **esattamente la stessa identica riga** ($t_1 = t_2$). Abbiamo appena dimostrato che è impossibile avere due tuple distinte con lo stesso valore di K. Questa caratteristica (identificare univocamente ogni riga) è l'esatta definizione di **superchiave**.

2. Se Kè una superchiave, allora $K \to T$ ($\Rightarrow$)

Se K è una superchiave dello schema, allora per la definizione stessa di superchiave non possono esistere nel database due righe distinte con lo stesso valore di K. Quindi, se prendiamo due righe qualsiasi t_1 e t_2 e notiamo che hanno lo stesso valore per la chiave K, ovvero:

$$t_1[K] = t_2[K]$$

Se due righe condividono la stessa superchiave, stiamo per forza guardando **esattamente la stessa identica riga** ($t_1 = t_2$).

Essendo la stessa riga, avrà ovviamente gli stessi valori anche nel resto delle colonne quindi $t_1[T] = t_2[T]$

Abbiamo appena dimostrato che la condizione $t_1[K] = t_2[K]$ implica necessariamente $t_1[T] = t_2[T]$. L'esatta definizione matematica della dipendenza funzionale $K \to T$.

Dipendenza Funzionale COMPLETA

Definizione: Una dipendenza $Y \to Z$ si dice **completa** se Z dipende da tutto l'insieme Y e non dipende da nessun sottoinsieme di Y.

- **Cosa significa in pratica:** Se togliessi anche solo un attributo dalla parte sinistra (Y), la freccia si spezzerebbe e non saresti più in grado di determinare Z. Z ha bisogno di **tutta** la chiave.

Dipendenza Funzionale NON COMPLETA / Parziale (Il problema)

Definizione: Si ha una dipendenza non completa (parziale) quando un attributo dipende dall'intera chiave primaria, ma **dipende anche da un pezzetto di essa preso da solo**.

Questo porta ad avere le anomalie citate in precedenza.

PRIMA FORMA NORMALE (1NF)

Una tabella è in 1NF se **ogni suo attributo è "atomico" (semplice)**.

Non sono ammessi:

1. **Valori multipli (o multi-valore):** Non puoi inserire più di un dato nella stessa cella.
2. **Attributi composti:** Non puoi avere un dato che racchiude logicamente più informazioni raggruppate.

Perché è "scontata"? Nei database relazionali (SQL), non esistono comandi per creare colonne fatte ad "array" o a "liste". Il linguaggio stesso ti costringe a usare tipi di dato semplici (numeri interi, stringhe di testo singole). Quindi, **qualsiasi tabella creata in un database SQL è automaticamente in Prima Forma Normale per natura**.

SECONDA FORMA NORMALE (2NF)

Una tabella è in 2NF se:

1. È già in 1NF.
2. Ogni **attributo non primo è in dipendenza funzionale completa** da **ogni chiave di R(X).**

Cosa significa in pratica? Significa che **tutti** gli attributi "normali" devono aver bisogno dell'intera chiave composta per essere scoperti.

Il test per capire se una tabella è in 2NF è immediato se **la Chiave Primaria di una tabella è formata da UN SOLO ATTRIBUTO (es. Matricola), quella tabella è matematicamente e automaticamente già in Seconda Forma Normale.** Non essendoci "pezzetti" di chiave, è fisicamente impossibile che esista una dipendenza "parziale". Il problema della 2NF si pone *esclusivamente* quando hai tabelle con chiavi composte (es. Articolo + Magazzino).

La Decomposizione in 2NF

Quando un attributo dipende solo da un "pezzetto" della chiave primaria composta, si prende quell'attributo, lo si stacca dalla tabella originale e lo si mette in una **nuova tabella**, portandosi dietro anche il "pezzetto" di chiave da cui dipendeva.

Ma attenzione: non puoi spezzare le tabelle a caso. Esistono due regole ferree (e molto chieste all'orale) che una buona decomposizione deve assolutamente rispettare.

Regola 1: Decomposizione "Senza Perdita" (Lossless)

L'operazione di decomposizione deve essere perfettamente reversibile.

- **Il concetto pratico:** Se io scompongo una tabella di partenza in due nuove tabelle, e poi faccio un NATURAL JOIN tra queste due nuove tabelle, **devo riottenere l'esatta tabella originale**, riga per riga.
- **Il disastro della "Perdita" (Tuple Spurie):** Un errore comune è pensare che "perdita" significhi cancellare dei dati. In realtà, nei database, una decomposizione sbagliata fa il danno opposto: quando provi a ricongiungere le tabelle con un Join, il sistema si confonde e crea delle **Tuple Spurie**, ovvero delle "righe fantasma" o combinazioni di dati false che nella realtà non esistono!
- **La definizione matematica:**

Se scompongo lo schema $R(X)$ nei sottoschemi $R_1(X_1)$ e $R_2(X_2)$, la decomposizione è senza perdita se per ogni istanza legale r vale la formula:

$$\pi_{X_1}(r) \bowtie \pi_{X_2}(r) = r$$

Come garantire che la decomposizione sia Lossless (Domanda d'esame)

Per essere matematicamente certi di non generare tuple spurie, c'è un test facilissimo: bisogna guardare **l'attributo in comune** tra le due nuove tabelle (cioè la colonna "ponte" usata per fare il Join).

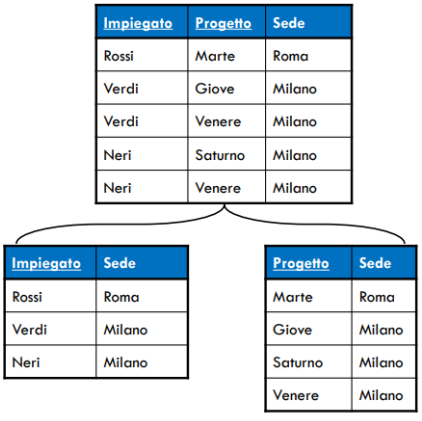
- **La condizione necessaria e sufficiente:** Quell'attributo in comune **DEVE essere una superchiave (o chiave primaria) in almeno una delle due nuove tabelle**.

$$X_1 \cap X_2 \rightarrow X_1 \quad \text{oppure} \quad X_1 \cap X_2 \rightarrow X_2$$

Regola 2: Conservazione delle Dipendenze (Dependency Preservation)

Oltre a non creare righe false, una buona decomposizione deve assicurarsi che nessuna delle regole logiche originarie vada perduta.

Tutte le Dipendenze Funzionali (le frecce logiche) che avevi individuato all'inizio sullo schema gigante, devono continuare ad esistere ed essere verificabili all'interno dei due nuovi sottoschemi più piccoli.



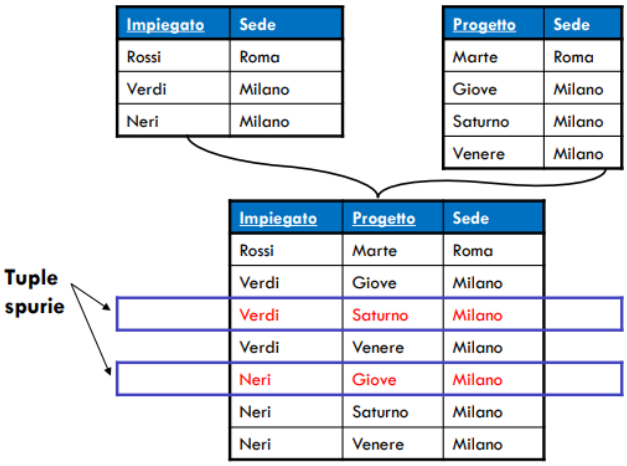
Impiegato	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Venere	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Impiegato	Sede
Rossi	Roma
Verdi	Milano
Neri	Milano

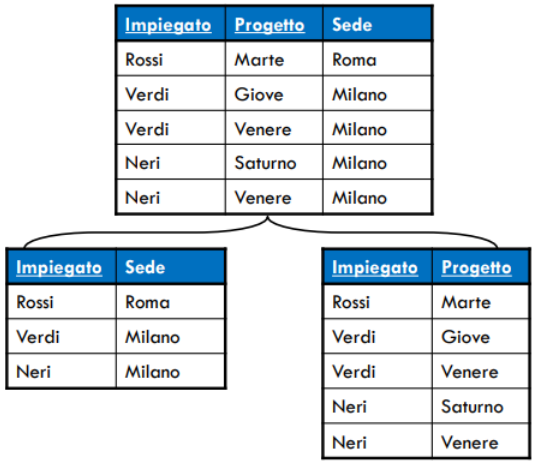
Progetto	Sede
Marte	Roma
Giove	Milano
Saturno	Milano
Venere	Milano

Decomposizione su attributo comune **Sede.**

L'attributo in comune deve essere la chiave primaria in *almeno una* delle due nuove tabelle. Qui la Sede non è la chiave di nessuno!



Quando il database tenta di fare il Join ricucendo i dati usando "Milano", fa un pasticcio. Prende l'impiegato "Verdi" (che sta a Milano) e lo associa matematicamente a *tutti* i progetti che si svolgono a Milano (Giove, Venere e Saturno). Ma nella realtà, Verdi non lavora a Saturno! Il database ha appena inventato un dato falso. Queste righe fantasma generate da un Join errato si chiamano **Tuple Spurie**.



Decomposizione su attributo comune **Impiegato**

L'Impiegato è una chiave in almeno una delle tabelle (A) , ogni impiegato ha una sola e specifica sede logica (infatti $Impiegato \rightarrow Sede$). Quando il database fa il Join, "Rossi" si ricongiunge in modo univoco solo con "Roma", e "Neri" si ricongiunge solo ed esclusivamente con "Milano". Il risultato sarà l'esatta fotocopia della tabella di partenza, senza righe inventate. Questa è una **Decomposizione Senza Perdita**.

Conservazione delle Dipendenze Esempio positivo

Magazzini

Articolo	Magazzino	Quantità	Città
scarpe	NA1	2500	Napoli
scarpe	RM1	4500	Roma
pantaloni	NA1	3000	Napoli

DF1: Articolo, Magazzino ⟶ Quantità, Città

DF2: Magazzino ⟶ Città

DECOMPOSIZIONE SU MAGAZZINO

Articoli_Magazzini

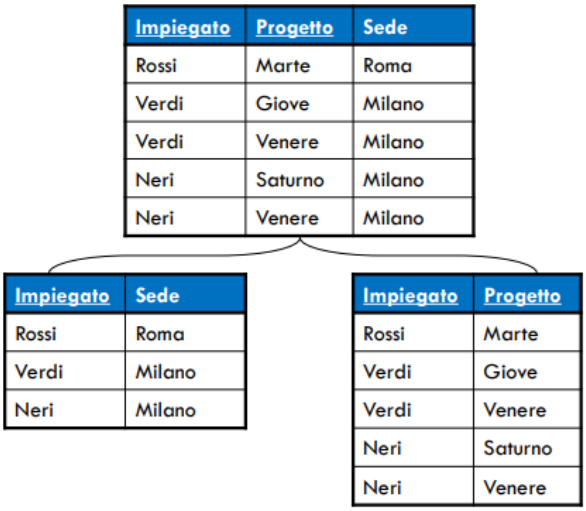
Articolo	Magazzino	Quantità
scarpe	NA1	2500
scarpe	RM1	2500
pantaloni	NA1	3000

Indirizzo_Magazzini

Magazzino	Città
NA1	Napoli
RM1	Roma

Le dipendenze sono salve? **Sì.** La prima regola vive interamente nella prima tabella. La seconda regola vive interamente nella seconda tabella. Il database è sicuro.

ESEMPIO CHE NON MANTIENE LE DF (Decomposizione su attributo comune Impiegato)



Impiegato	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Venere	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Impiegato	Sede
Rossi	Roma
Verdi	Milano
Neri	Milano

Impiegato	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere

DF1: Impiegato ⟶ Sede

DF2: Progetto ⟶ Sede

- Impiegato -> Sede è salva? **Sì**, è tutta dentro la Tabella A.
- Progetto -> Sede è salva? **NO!** Il Progetto sta nella Tabella B, la Sede sta nella Tabella A. La regola è stata spezzata a metà. Il database non è più in grado di controllarla in automatico!

La conseguenza diretta della mancata conservazione di una dipendenza funzionale è la perdita di consistenza globale dei dati. Quando una regola logica viene spezzata su più tabelle, il database perde la capacità di verificarne la validità in modo automatico durante le operazioni di inserimento o aggiornamento.

Dimostrazione Pratica dell'Anomalia (Esempio: Inserimento della tupla {Neri, Marte})

Attualmente, nel nostro database abbiamo l'impiegato Rossi che lavora al progetto Marte, localizzato nella sede di Roma.

A un certo punto, si decide di assegnare anche l'impiegato Neri al progetto Marte. Il DBMS va a registrare questa nuova informazione nella Tabella B e l'operazione va a buon fine: la nuova riga {Neri, Marte} viene accettata perché, a livello locale, non viola nessuna regola ristretta a quella specifica tabella.

Impiegato	Progetto	Sede
Rossi	**Marte**	**Roma**
Verdi	Giove	Milano
Verdi	Venere	Milano
Neri	Saturno	Milano
Neri	Venere	Milano
Neri	**Marte**	**Milano**

Il disastro emerge solo quando cerchiamo di ricostruire l'interezza dei dati unendo le due tabelle. Dalla Tabella A, infatti, risulta che Neri opera fisicamente nella sede di Milano. Di conseguenza, incrociando i dati, il progetto Marte si ritrova improvvisamente ed erroneamente assegnato sia a Roma (tramite Rossi) sia a Milano (tramite Neri).

TERZA FORMA NORMALE (3NF)

Uno schema di relazione R(X) è in 3NF se:

- è in **2NF** e
- ogni attributo non primo di R(X) non dipende transitivamente da ogni chiave di R

In altre parole, ogni attributo non primo dipende dalla chiave (1NF), da tutta la chiave (2NF) e solo dalla chiave (3NF)

Cos'è una Dipendenza Transitiva?

Una dipendenza transitiva si verifica quando un attributo normale (non primo) dipende da un altro attributo normale, che a sua volta dipende dalla chiave primaria.

Definizione Alternativa di 3NF

Uno schema di relazione è in 3NF se e solo se, per **ogni singola dipendenza funzionale non banale** ($Y \rightarrow Z$) presente nello schema, risulta vera **almeno una** delle seguenti due condizioni:

1. Y**(la parte sinistra) è una Superchiave** dello schema.
2. Z **(la parte destra) è un Attributo Primo** (ovvero appartiene a una chiave candidata).

Se la parte sinistra *non* è una chiave (è un attributo normale), allora sta provando a creare una dipendenza transitiva! L'unico caso in cui il database lo "perdona" ed evita di violare la 3NF, è se la parte destra (Z) è un attributo "nobile" (un attributo primo). Se invece l'attributo normale Y determina un altro attributo normale Z

Anomalia

Lo schema TEL(**Prefisso, Numero**, Località, Abbonato, Indirizzo) ha vincoli

Prefisso, Numero ⟶ Località, Abbonato, Indirizzo

Località ⟶ Prefisso

Sottoponiamo questa regola al test della 3NF:

- La Località è una superchiave? No.
- Il Prefisso è un attributo primo? **Sì**, perché fa parte della chiave primaria.

Poiché la seconda condizione è rispettata, **lo schema risulta matematicamente in 3NF**.

Prefisso	Numero	Località	Abbonato	Indirizzo
051	457856	Bologna	Rossi	Via Roma 8
059	452332	Modena	Verdi	Via Bari 16
051	987856	Bologna	Bianchi	Via Napoli 77
051	552346	Castenaso	Neri	Piazza Borsa 12
059	387654	Vignola	Mori	Via Piave 65

Tuttavia, guardando i dati, il problema è lampante: il prefisso di una città (es. "051" per Bologna) verrà ripetuto inutilmente per ogni singolo abbonato di quella città. Questo crea un'enorme **anomalia di inserimento**: se l'azienda telefonica crea un nuovo prefisso per una nuova area geografica, non può registrarlo nel database finché non trova almeno un cliente a cui assegnare un numero (poiché il Numero è parte della chiave primaria e non accetta valori vuoti).

L'origine del problema è che un attributo non primo (la Località) si è permesso di determinare un attributo primo (il Prefisso).

Per risolvere questo paradosso, è stata introdotta la **Forma Normale di Boyce-Codd (BCNF)**.

FORMA NORMALE DI BOYCE E CODD

Uno schema di relazione $R(X)$ è in BCNF se:

- È in 1NF.
- Per **ogni** dipendenza funzionale non banale $Y \rightarrow Z$ definita su R, **la parte sinistra (Y) deve essere una superchiave.**

Riprendiamo l'esempio TEL (Prefisso, Numero, Località, Abbonato, Indirizzo) con il problema $Località \rightarrow Prefisso$. Poiché Località non è una chiave, la BCNF non accetta questa tabella. Dobbiamo decomporla tagliando l'attributo in comune.

Una soluzione consiste nel decomporre lo schema in

NUM_TEL(**Numero, Località**, Abbonato, Indirizzo)

PREF_TEL(**Località**, Prefisso)

Numero	Località	Abbonato	Indirizzo
457856	Bologna	Rossi	Via Roma 8
452332	Modena	Verdi	Via Bari 16
987856	Bologna	Bianchi	Via Napoli 77
552346	Castenaso	Neri	Piazza Borsa 12
387654	Vignola	Mori	Via Piave 65

Prefisso	Località
051	Bologna
059	Modena
051	Castenaso
059	Vignola

- **BCNF implica 3NF:** Se uno schema è in BCNF, è matematicamente garantito che sia anche in 3NF. La BCNF è semplicemente una 3NF "più severa".
- **3NF NON implica BCNF:** Se uno schema è in 3NF, non è detto che sia in BCNF (pensa all'esempio del Telefono appena visto).

Normalizzazione vs Prestazioni (La Denormalizzazione)

Questa è un'ottima domanda di ragionamento per chiudere un esame orale: *"In un'azienda reale, si normalizza sempre tutto fino alla BCNF?"*

La risposta sorprendente è: **No, a volte si sceglie deliberatamente di non farlo.**

Il Problema dei JOIN (Perché non normalizzare?)

La normalizzazione cura le anomalie spezzando una tabella grande in tante tabelle piccole. Tuttavia, quando un'applicazione (es. un sito web) ha bisogno di leggere tutte quelle informazioni insieme, il database deve eseguire dei **JOIN**. Il JOIN è l'operazione più "pesante" e lenta per un computer. Se hai un database con milioni di righe, fare JOIN continui rallenterà drasticamente il sistema.

Le Alternative per aumentare le Performance:

Alternativa 1: La Denormalizzazione Volontaria Il progettista decide di tenere una tabella "sporca" (con ridondanze e non normalizzata) per velocizzare le operazioni di lettura (Accesso più veloce).

- *Il prezzo da pagare:* Lo spazio sprecato su disco aumenta, ma soprattutto serve un enorme sforzo da parte dei programmatori. Dovranno scrivere codice molto complesso per gestire "a mano" le anomalie di aggiornamento e assicurarsi che i dati doppi siano sempre sincronizzati.

Alternativa 2: Le Viste Materializzate (Il compromesso del DBA) Il database viene progettato in modo pulito e normalizzato in BCNF, ma poi si crea una **Vista Materializzata**. Si tratta di una tabella fisica "copia" che contiene i dati già uniti tramite JOIN.

- *Vantaggi:* Lettura velocissima (come se fosse denormalizzato) ma progettazione di base pulita. Lo sforzo di programmazione per tenere aggiornati i dati doppi non ricade sul programmatore, ma viene gestito automaticamente dal motore del DBMS sotto il cofano.

APPLICAZIONI DI BASI DI DATI

L'Architettura "Three-Tier" (I 3 Livelli)

Qualsiasi software moderno (dall'app sul cellulare al sito della banca) non è un unico blocco di codice gigante, ma è diviso in tre "strati" logici ben separati. Devi saperli elencare e descrivere:

1. **Livello Presentazione (Presentation Layer):** È l'interfaccia utente (UI). È quello che l'utente vede e tocca. Bottoni, finestre, campi di testo. Si occupa solo di mostrare i dati e raccogliere gli input.
2. **Livello Applicazione (Application Layer):** È il "cervello" vero e proprio. Qui risiede la **Business Logic** (le regole aziendali). È il codice scritto in Java, Python o C# che prende l'input dell'utente, fa i calcoli, controlla le regole e decide cosa fare.
3. **Livello Dati (Data Layer):** È il regno del **DBMS** (es. MySQL, Oracle). Il suo unico scopo è salvare i dati su disco, garantirne la sicurezza, recuperarli velocemente quando l'Application Layer glieli chiede tramite comandi **SQL**, e gestire la concorrenza (più utenti contemporaneamente).

Come comunicano? Il codice applicativo (Livello 2) deve "connettersi" fisicamente al DBMS (Livello 3). Apre una connessione, gli spedisce la stringa "SELECT * FROM...", aspetta che il DBMS risponda con i dati grezzi, li "traduce" dal formato del database al formato del linguaggio di programmazione (es. da tupla a oggetto Java), e infine li passa al Livello 1 per farli vedere all'utente.

2. Le Stored Procedure: Spostare il "Cervello" nei Dati

Normalmente, come abbiamo detto, i calcoli e i cicli for/if si fanno nel Livello Applicazione (strato 2). Tuttavia, c'è un'eccezione potentissima: le **Stored Procedure**.

- **Cosa sono?** Sono dei veri e propri "mini-programmi" (con variabili, cicli IF e WHILE) che vengono **salvati ed eseguiti direttamente dentro il DBMS** (Livello 3).
- **Il linguaggio:** Siccome l'SQL normale sa solo fare interrogazioni (non ha i cicli IF), sono nati linguaggi ibridi come il **PL/SQL** (Procedural Language/SQL) o il T-SQL. Fondono la logica della programmazione con le query sui dati.

Perché usarle? (Domanda d'esame)

- *Vantaggio (Centralizzazione):* Se una regola aziendale è complessa e uguale per tutti (es. il calcolo delle tasse su uno scontrino), invece di scriverla in Java per il sito web e poi riscriverla in Swift per l'app iOS, la scrivi **una volta sola** nel DBMS come Stored Procedure. Tutte le app la richiameranno. Se la tassa cambia, modifichi il DBMS e tutte le app si aggiornano magicamente.

- *Svantaggio (Collo di bottiglia):* Il povero DBMS, che dovrebbe solo pensare a salvare i dati in fretta, ora deve anche fare calcoli computazionali pesanti. Se ci sono migliaia di utenti, il server database potrebbe rallentare o bloccarsi (calo di performance).

I TRIGGER

Un trigger è una specifica per la quale una data azione o funzione deve attivarsi automaticamente ogni qual volta un'operazione stabilita viene eseguita su un oggetto specificato della base di dati.

Tutti i trigger si basano sul paradigma universale **E-C-A (Evento - Condizione - Azione)**:

- **Evento:** Cosa fa scattare il trigger? lo standard prevede che la codifica dell'evento sia una primitiva per la manipolazione dei dati in SQL (Es. "Qualcuno ha fatto una INSERT nella tabella Impiegati").
- **Condizione (Opzionale):** è un predicato booleano espresso in SQL che controlla se un evento accade, restituendo 'vero' o 'falso'. (Es. WHEN stipendio > 10000). Se non c'è, vale sempre TRUE.
- **Azione:** Cosa deve fare il database in risposta? (Es. "Registra l'operazione in una tabella di LOG", oppure "Blocca l'inserimento").

TIPOLOGIE

- **Trigger DML:** Sono il 99% dei trigger che userai. Scattano quando i dati vengono manipolati tramite INSERT, UPDATE o DELETE.
- **Trigger DDL / di Sistema:** Scattano per eventi amministrativi.
    - **Eventi scatenanti DDL:** Creazione, modifica o distruzione di oggetti nel database (CREATE, ALTER, DROP).
    - **Eventi scatenanti di Sistema:** Accensione/spegnimento del server (STARTUP, SHUTDOWN), accessi degli utenti (LOGON, LOGOFF) o errori critici (SERVER ERROR).

GRANULARITÀ

È semplicemente "quanto spesso" si attiva il trigger. Ci sono due modi:

1. Livello di riga (row-level)

- Il trigger si attiva **una volta per ogni riga** che viene modificata
- Esempio: se cancelli 10 studenti con una sola query, il trigger parte 10 volte (una per ogni studente cancellato)

2. Livello di istruzione (statement-level)

- Il trigger si attiva **una sola volta**, indipendentemente da quante righe vengono modificate
- Esempio: Immagina una regola aziendale che vieta di fare licenziamenti (operazioni di DELETE sulla tabella Dipendenti) durante il fine settimana. Un responsabile lancia una query di venerdì notte per cancellare **10 dipendenti**.
- *Perché lo usiamo qui?* Il trigger a livello di istruzione scatta *una sola volta* appena riceve il comando DELETE. Controlla il calendario del server: se è sabato, blocca l'intera operazione e restituisce un errore. Sarebbe completamente inutile e pesantissimo per il server usare un trigger di riga e fargli controllare il calendario 10 volte di fila per ogni dipendente cancellato. Basta bloccare l'istruzione generale alla radice!

MODALITA' DI ESECUZIONE

Tempistica rispetto all'operazione (Before / After)

- **BEFORE (Prima dell'evento):** Il trigger si attiva *immediatamente prima* che l'operazione SQL venga eseguita e salvata nel database.
    - *Quando usarlo:* È ideale per modificare i dati "al volo" (es. convertire automaticamente un nome in maiuscolo prima del salvataggio) o per bloccare un'operazione se i dati inseriti non sono validi.
- **AFTER (Dopo l'evento):** Il trigger si attiva *immediatamente dopo* che l'operazione SQL è stata completata con successo.
    - *Quando usarlo:* È perfetto per operazioni a cascata o di registrazione (es. scalare la quantità di un prodotto in magazzino dopo una vendita, oppure scrivere un log/registro di sicurezza).

2. Tempistica rispetto alla transazione

Di norma, l'esecuzione del trigger è **Immediata**: viene eseguita subito (in modalità *Before* o *After*) all'interno della stessa transazione che lo ha attivato. Tuttavia, sono previste anche altre modalità di gestione:

- **Esecuzione Differita (Deferred):** Il trigger non viene eseguito subito. La sua esecuzione viene "accodata" e gestita solo al *termine* dell'intera transazione.
- **Esecuzione Distaccata (Detached):** Il trigger viene gestito ed eseguito in una *transazione completamente separata* e indipendente da quella che lo ha attivato.

TRANSITION TABLE

Come fa un trigger a sapere esattamente quali dati stai modificando, inserendo o cancellando? Il database crea di nascosto delle tabelle temporanee, chiamate **Transition Table** (Tabelle di Transizione), che fungono da "sala d'attesa" per i dati prima o dopo l'operazione.

Questo è un argomento molto tecnico, e i professori lo chiedono per capire se hai chiara la differenza tra un trigger che agisce sull'intera tabella e uno che agisce riga per riga.

1. Cosa c'è dentro le Tabelle di Transizione?

A seconda dell'istruzione SQL che lancia il trigger (l'evento), il database prepara scenari diversi:

- **In caso di INSERT (Inserimento):** Stai creando dati dal nulla. Esiste solo una **NEW Transition Table** che contiene le righe nuove appena arrivate.
- **In caso di DELETE (Cancellazione):** Stai distruggendo dati esistenti. Esiste solo una **OLD Transition Table** che contiene le righe destinate ad essere eliminate.
- **In caso di UPDATE (Modifica):** Stai sostituendo dei valori. Qui il database crea **entrambe** le tabelle (OLD e NEW). Hanno la stessa grandezza (cardinalità): la OLD contiene i vecchi valori da rimpiazzare, la NEW contiene i nuovi valori aggiornati.

Il vantaggio del BEFORE Trigger: Se il trigger è di tipo BEFORE (scatta prima) e ha a disposizione una *new transition table*, il codice del trigger può modificare i dati della tabella NEW *prima* che l'operazione venga effettivamente salvata sul database.

Per utilizzare i dati presenti nelle transition table all'interno del codice del trigger, il database mette a disposizione delle **variabili speciali** (transition variable). Il modo in cui queste variabili "leggono" i dati dipende dalla *granularità* del trigger:

- Per un trigger di livello **istruzione**, la variabile ha come valore una **transition table**
- Per un trigger di livello **tupla**, la variabile non contiene tutta la tabella, ma **una singola riga alla volta**. Il trigger viene eseguito in un ciclo: a ogni esecuzione, la variabile "punta" a una riga diversa della transition table, provocando l'esecuzione dell'azione del trigger per quella specifica riga.

Una transition variable può essere una **old** transition variable o una **new** transition variable, a seconda della tabella su cui varia.

Quando un'operazione di UPDATE genera sia una tabella OLD che una NEW, esiste una **corrispondenza esatta uno-a-uno** tra le righe: ogni riga presente nella NEW deriva dalla modifica di una e una sola specifica riga presente nella OLD.

Chi può sfruttare questa corrispondenza? Questa relazione uno-a-uno è visibile **soltanto al trigger di livello tupla**. Poiché il trigger a livello di tupla viene eseguito una volta per ogni singola riga, nel momento in cui scatta può accedere contemporaneamente a:

1. La variabile **OLD** (per vedere com'era la riga *prima* della modifica).
2. La variabile **NEW** (per vedere com'è la riga *dopo* la modifica).

Ordine di Esecuzione dei Trigger DML

Quando più trigger sono associati alla stessa tabella, il database li esegue seguendo un ordine rigoroso e prevedibile:

1. **BEFORE a livello di ISTRUZIONE** (Statement-level): Si attiva una sola volta prima dell'operazione.
2. **BEFORE a livello di RIGA** (Row-level): Si attiva per ogni singola riga coinvolta, prima della modifica.
3. **ESECUZIONE DELL'EVENTO**: L'operazione SQL reale (INSERT, UPDATE o DELETE) viene eseguita.
4. **AFTER a livello di RIGA** (Row-level): Si attiva per ogni singola riga coinvolta, dopo la modifica.
5. **AFTER a livello di ISTRUZIONE** (Statement-level): Si attiva una sola volta alla fine di tutto.

Note importanti:

- **Annidamento (Cascata):** Un trigger può attivare un altro trigger se la sua azione modifica una tabella diversa che ha a sua volta un trigger associato.
- **Cambiare l'ordine:** Non è possibile cambiare questo ordine naturale. Se serve un ordine diverso, l'unica soluzione è unire (accorpare) tutte le logiche in un unico trigger.

Come si crea un Trigger DML: I Componenti

Un trigger DML si crea con il comando CREATE [OR REPLACE] TRIGGER. Per definirlo correttamente, bisogna specificare 5 elementi fondamentali:

1. **Nome:** Deve essere unico all'interno dello schema.
2. **Target:** La tabella (o vista) su cui il trigger agisce.
3. **Evento attivante:** L'operazione che lo fa scattare (INSERT, UPDATE, DELETE). Un trigger può ascoltare più eventi contemporaneamente (es. INSERT OR UPDATE).

4. **Modalità di attivazione:** BEFORE (prima dell'evento) o AFTER (dopo l'evento).
5. **Granularità:**
    - FOR EACH ROW: Il trigger si esegue una volta per *ogni riga* modificata.
    - FOR EACH STATEMENT: Il trigger si esegue una *sola volta* per l'intero comando SQL, indipendentemente dal numero di righe toccate.

La Clausola WHEN e il Corpo del Trigger

- **La clausola WHEN (Condizione):**
    - Si usa **SOLO** nei trigger a livello di riga (FOR EACH ROW).
    - Funziona come un filtro: contiene una condizione booleana (vero/falso). Il trigger esegue l'azione *solo* sulle righe che soddisfano questa condizione.
- **Il Corpo del Trigger (<Corpo Trigger>):**
    - Può iniziare con la sezione DECLARE per definire variabili temporanee locali (visibili solo dentro il trigger).
    - Se il trigger contiene più istruzioni SQL, deve essere racchiuso tra le parole chiave BEGIN (inizio) e END (fine).

Esempio Pratico: Formattazione dei dati

Obiettivo: Assicurarsi che nome e cognome vengano salvati sempre in maiuscolo nella tabella Clienti.

```
CREATE TRIGGER cliente_bef_ins
BEFORE INSERT ON Clienti
FOR EACH ROW
BEGIN
    :new.nome := UPPER(:new.nome);
    :new.cognome := UPPER(:new.cognome);
END;
```

Spiegazione: Poiché è un trigger BEFORE INSERT a livello di riga, intercetta i dati *prima* che vengano salvati. Utilizza la variabile di transizione :new per leggere i valori in arrivo, li trasforma in maiuscolo con la funzione UPPER() e li riscrive, modificando così l'operazione al volo.

5. Trigger DDL (Data Definition Language)

A differenza dei trigger DML (che reagiscono ai *dati*), i trigger DDL reagiscono alle modifiche della *struttura* del database o agli eventi di sistema.

- **Sintassi:** Simile ai DML, ma l'evento attivante può essere:

    - Eventi di sistema: Avvio/Chiusura del database, Login/Logout di un utente.
    - Eventi DDL: CREATE, ALTER, DROP di oggetti (tabelle, viste, ecc.).
- **Limitazioni importanti:**
    - Non è possibile creare un trigger BEFORE sull'avvio del database (il sistema non è ancora pronto per eseguirlo).
    - Non è possibile creare un trigger AFTER sulla chiusura del database (il sistema si sta già spegnendo).

APPROCCI PER LA PROGRAMMAZIONE DI UNA BASE DI DATI

Nessun utente usa direttamente SQL per prelevare soldi dal bancomat. C'è sempre un software (il Livello Applicazione) in mezzo. Il programmatore deve trovare un modo per far comunicare il suo linguaggio procedurale (che ragiona a cicli e variabili) con il linguaggio SQL (che ragiona a insiemi logici e tabelle).

Esistono tre approcci fondamentali per programmare questa comunicazione:

1. SQL Embedded (SQL Incorporato)

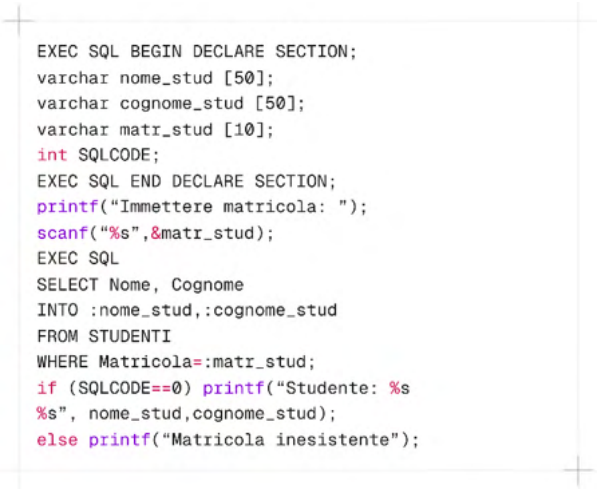
È l'approccio più "classico": le istruzioni SQL vengono **scritte direttamente dentro il codice** del programma ospite (es. C, C++, COBOL) e riconoscibili tramite un prefisso speciale, solitamente EXEC SQL.

Come funziona:

1. Il programmatore scrive il codice mescolando istruzioni del linguaggio ospite e istruzioni SQL.
2. Un **precompilatore** (o preprocessore) analizza il codice sorgente, individua le istruzioni SQL e le sostituisce con chiamate al DBMS.
3. Il programma viene poi compilato normalmente.

Elementi chiave:

- **Sezione DECLARE:** Si usa EXEC SQL BEGIN DECLARE SECTION per dichiarare le variabili condivise tra programma e DB.
- **Clausola INTO:** Permette di salvare il risultato di una SELECT (che restituisce una sola riga) direttamente nelle variabili del programma.
- **Variabili di stato (SQLCODE / SQLSTATE):** Variabili speciali che il DBMS aggiorna dopo ogni operazione per comunicare al programma se l'operazione è andata a buon fine (es. SQLCODE = 0 significa successo).
- **Cursori:** Poiché una SELECT può restituire *tante* righe ma il programma le elabora *una alla volta*, si usano i **cursori**: strutture che funzionano come "puntatori" per scorrere il risultato riga per riga.



```
EXEC SQL BEGIN DECLARE SECTION;
varchar nome_stud [50];
varchar cognome_stud [50];
varchar matr_stud [10];
int SQLCODE;
EXEC SQL END DECLARE SECTION;
printf("Immettere matricola: ");
scanf("%s",&matr_stud);
EXEC SQL
SELECT Nome, Cognome
INTO :nome_stud,:cognome_stud
FROM STUDENTI
WHERE Matricola=:matr_stud;
if (SQLCODE==0) printf("Studente: %s
%s", nome_stud,cognome_stud);
else printf("Matricola inesistente");
```

Limiti dell'SQL Embedded:

- È un approccio **statico**: le query sono scritte nel codice e non possono cambiare senza ricompilare il programma.
- È **dipendente dal DBMS**: se cambi database, devi riscrivere tutto il codice perché ogni DBMS ha le sue librerie specifiche.

2. SQL/CLI (Call Level Interface)

Per superare i limiti dell'SQL Embedded, è nato l'approccio **SQL/CLI**, che è alla base di tecnologie moderne come ODBC e JDBC.

Come funziona:

- Non si scrivono più istruzioni SQL direttamente nel codice.
- Il programmatore usa una **libreria di funzioni** (API) fornita dal DBMS.
- Le istruzioni SQL vengono passate al database come **stringhe di testo** (parametri) all'interno delle chiamate a funzione.

Vantaggi:

- È un approccio **dinamico**: le query possono essere costruite "al volo" durante l'esecuzione del programma.
- **Non serve un precompilatore**: il programma si compila normalmente.
- **Maggiore portabilità**: cambiando DBMS, spesso basta cambiare la libreria, non tutto il codice.

Svantaggi:

- Gli errori di sintassi SQL vengono scoperti **solo a runtime** (durante l'esecuzione), non in fase di compilazione.

La Sequenza Standard delle Chiamate SQL/CLI

Per accedere correttamente alle informazioni, un'applicazione che usa SQL/CLI deve seguire rigorosamente questa sequenza di chiamate a funzione:

1. **Connessione al DB:** L'applicazione stabilisce il collegamento con il database (fornendo credenziali, indirizzo del server, ecc.).
2. **Esecuzione degli Statement SQL:** L'applicazione invia i comandi SQL (come stringhe) tramite le funzioni della libreria, elabora i risultati e gestisce eventuali errori.
3. **Chiusura della Connessione:** Al termine delle operazioni, l'applicazione rilascia le risorse e chiude il collegamento con il database in modo pulito.

3. Linguaggi di Programmazione per Basi di Dati

Il terzo approccio consiste nell'usare linguaggi nati specificamente per lavorare con i database, dove SQL è **esteso** con costrutti tipici dei linguaggi di programmazione (variabili, cicli, condizioni, funzioni, ecc.).

Esempi famosi:

- **PL/SQL** (usato da Oracle)
- **T-SQL** (usato da Microsoft SQL Server)

Vantaggi:

- È l'approccio **più diffuso** per applicazioni con una forte interazione col database.
- **Non c'è conflitto di impedenza**: non serve convertire i tipi di dato tra linguaggio e DB, perché sono lo stesso ambiente.

Il Problema dell'Impedance Mismatch (Conflitto di Impedenza)

Nei primi due approcci (SQL Embedded e SQL/CLI) si presenta un problema fondamentale: il modello dei database (basato su **insiemi di tuple**) è molto diverso dal modello dei linguaggi di programmazione (basato su **variabili e strutture dati**).

Questo "conflitto" richiede un continuo lavoro di **mapping**:

- Convertire i tipi di dato (es. una data SQL in una stringa o oggetto del programma).
- Trasformare il risultato di una query (una tabella) in una struttura dati gestibile dal programma (es. un array, una lista, o tramite i cursori).

Il terzo approccio (PL/SQL, T-SQL) elimina questo problema perché il linguaggio è già "nativo" del database.

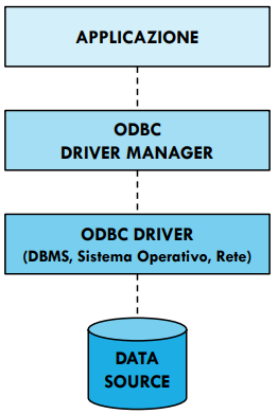
ODBC - Open Database Connectivity

ODBC è uno standard che permette ad applicazioni diverse (scritte in linguaggi diversi, su sistemi operativi diversi) di accedere a database remoti utilizzando comandi SQL.

Si basa sul modello **client-server**:

- **Client:** L'applicazione che richiede l'accesso ai dati
- **Server:** Il DBMS che eroga il servizio di accesso ai dati
- **Linguaggio:** SQL viene usato per formulare le richieste

Il Driver ODBC



Il **driver** è il componente fondamentale che permette la comunicazione tra l'applicazione e il database.

Cos'è:

- Una libreria di funzioni che viene caricata dinamicamente dall'applicazione quando deve accedere al database.

A cosa serve:

- **Maschera le differenze:** Nasconde all'applicazione tutte le differenze legate al DBMS specifico, al sistema operativo del server e ai protocolli di rete utilizzati.
- **Astrazione:** Fornisce un'interfaccia di programmazione comune, così l'applicazione non deve conoscere i dettagli proprietari di ogni singolo database.
- **Traduzione:** Converte le chiamate ODBC standard nel "dialetto" specifico del DBMS locale.

Chi lo fornisce: I produttori di DBMS devono creare driver diversi a seconda:

- Del protocollo di comunicazione utilizzato
- Del sistema operativo su cui gira il server

Vantaggio per lo sviluppatore: Usando le API ODBC, l'applicazione può accedere a database diversi (Oracle, SQL Server, MySQL, ecc.) senza dover imparare interfacce proprietarie per ciascuno.

I 4 Componenti dell'Architettura ODBC

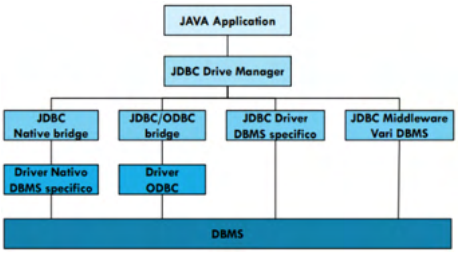
L'accesso a un database remoto tramite ODBC richiede la cooperazione di quattro componenti:

1. **Applicazione**
    - Richiama le funzioni SQL per eseguire interrogazioni e acquisire i risultati.
    - Grazie ai driver, per l'applicazione sono **trasparenti**:
    - Il protocollo di comunicazione di rete
    - Il tipo di DBMS (Oracle, SQL Server, ecc.)
    - Il sistema operativo del server
2. **Driver Manager**
    - È il "centralino" che gestisce i driver.

- Si occupa di **caricare il driver corretto** su richiesta dell'applicazione, in base al database a cui ci si vuole connettere.

3. **Driver ODBC**
   - Implementa le funzioni ODBC specifiche per un determinato DBMS.
   - **Traduce** le query SQL standard nel dialetto del DBMS locale.
   - **Restituisce i risultati** all'applicazione usando meccanismi di buffering (memoria temporanea).
4. **Sorgente Dati (Data Source)**
   - È il **database remoto** vero e proprio.
   - Il sistema che esegue fisicamente le operazioni richieste e restituisce i dati.

JDBC - Java Database Connectivity



JDBC è un insieme di **API orientate agli oggetti**, basato sullo standard CLI, definito dalla Sun Microsystems. Consiste in un set di classi e interfacce scritte in **Java** (contenute nei package java.sql e javax.sql) e rappresenta l'equivalente Java di ODBC, fornendo una collezione di oggetti e metodi per l'interazione con una base di dati.

Servizi offerti da JDBC

JDBC è lo standard *de facto* per lo sviluppo di applicazioni Java orientate ai database ed è parte integrante del JDK (Java Development Kit). Offre ai programmatori i seguenti servizi fondamentali:

- **Connessione al database:** Stabilire il collegamento con la sorgente dati.
- **Invio di statement SQL:** Eseguire interrogazioni e comandi di aggiornamento.
- **Processing dei risultati:** Elaborare i dati restituiti dalle query.
- **Gestione delle transazioni:** Controllare le operazioni atomiche (commit, rollback).

Architettura JDBC

L'architettura di JDBC è organizzata in **livelli (layer)**, come visibile nello schema:

1. Java Application (Livello Applicativo)

È il livello più alto, dove risiede l'applicazione Java che richiede l'accesso ai dati. L'applicazione interagisce esclusivamente con le API JDBC, senza conoscere i dettagli del database sottostante.

2. JDBC Driver Manager

Opera come intermediario tra l'applicazione e i driver. Le sue funzioni principali sono:

- Tenere traccia di tutti i driver JDBC disponibili.
- Gestire la connessione tra l'applicazione e il driver appropriato per il database target.

3. JDBC Driver (Livello di Comunicazione)

È lo strato più basso dell'architettura JDBC, a diretto contatto con il database. I driver si occupano di:

- Creare la connessione fisica con la sorgente dati.
- Inviare le istruzioni SQL (interrogazioni o aggiornamenti).

- Processare e restituire i risultati all'applicazione.

4. DBMS (Database Management System)

Il database relazionale vero e proprio, che esegue fisicamente le operazioni richieste.

I 4 Tipi di Driver JDBC

I driver JDBC si classificano in base a come gestiscono la comunicazione tra l'applicazione Java e il database. Possono essere raggruppati in due modelli architetturali:

Modello	Tipi di Driver	Come funziona	Portabilità
Two-Tier (Client → DB)	**JDBC Native Bridge** e **JDBC Driver Puro**	L'applicazione comunica direttamente con il DB. Il bridge converte le chiamate JDBC in API native del DBMS; il driver puro usa un protocollo di rete proprietario del DB (es. TCP/IP).	Il driver puro è totalmente portabile (100% Java). Il bridge richiede librerie native sul client, quindi è meno portabile.
Three-Tier (Client → Middleware → DB)	**JDBC/ODBC Bridge** e **JDBC Middleware**	Le chiamate JDBC vengono convertite in un protocollo intermedio. Il bridge usa ODBC; il middleware usa un protocollo di rete standard indipendente dal DBMS.	Richiede configurazione lato client (ODBC) o un server middleware. Meno diretto, ma utile in ambienti enterprise complessi.

In sintesi: I driver "puri" (scritti interamente in Java) sono i più moderni e portabili, poiché non richiedono installazione di software aggiuntivo sul client e comunicano direttamente con il database via rete.

2. Classi e Interfacce Fondamentali (java.sql)

JDBC espone un set di interfacce e classi core che ogni sviluppatore utilizza per interagire con il database:

- **Driver**: Interfaccia di base. I produttori di driver JDBC devono implementare un'interfaccia Driver (mediante un'opportuna classe) affinché un'applicazione possa interfacciarsi con un tipo particolare di DBMS. Rappresenta il punto di ingresso per stabilire una connessione.
- **DriverManager**: Classe centrale che tiene traccia di tutti i driver registrati. Si occupa di caricare il driver giusto e di restituire un oggetto Connection.

- **Connection**: Rappresenta una **sessione attiva** con il database. Da qui si creano gli statement e si gestiscono le transazioni.
- **Statement**: Usato per inviare query SQL **statiche** (senza parametri).
- **PreparedStatement**: Usato per query con **parametri** (?). Precompila la query, migliorando sicurezza (previene SQL injection) e prestazioni.
- **CallableStatement**: Specializzato per eseguire **stored procedure** e funzioni memorizzate nel DB.
- **ResultSet**: Contiene i dati restituiti da una query SELECT. Permette di scorrere le righe risultato una alla volta.

3. La Connessione e il JDBC URL

Per connettersi, l'applicazione deve fornire al DriverManager una stringa di connessione univoca, chiamata **JDBC URL**.

Sintassi: jdbc:<sottoprotocollo>:<identificatore_database>

- jdbc: Protocollo fisso che indica l'uso di Java Database Connectivity.
- <sottoprotocollo>: Identifica il driver da usare (es. odbc, oracle, mysql).
- <identificatore_database>: Specifica dove si trova il DB e come accedervi.

Esempi Pratici:

- **Connessione tramite bridge ODBC:** jdbc:odbc:dbsegreteria
  (Qui dbsegreteria è il nome di una sorgente dati ODBC configurata nel sistema)
- **Connessione diretta con driver Java puro (Oracle Thin):**
  jdbc:oracle:thin:@db.unina.it:1521:dbsegreteria
  (Server: db.unina.it | Porta: 1521 | Nome DB/SID: dbsegreteria)

4. Introduzione a PL/SQL

PL/SQL (*Procedural Language / Structured Query Language*) è il linguaggio di programmazione proprietario di Oracle Database.

A cosa serve:

- Colma il divario tra SQL (linguaggio dichiarativo, orientato agli insiemi) e i linguaggi procedurali tradizionali.
- Permette di creare **Stored Procedure**: programmi memorizzati ed eseguiti direttamente sul server DB, garantendo maggiore velocità, sicurezza ed efficienza.
- Ideale per logiche di business complesse che richiedono una forte interazione con i dati.

Cosa aggiunge a SQL:

- Definizione di variabili e tipi di dato personalizzati.
- Strutture di controllo: sequenziali (BEGIN...END), condizionali (IF...ELSE), iterative (FOR, WHILE, LOOP).
- Possibilità di definire procedure, funzioni e gestire errori in modo strutturato.

5. Struttura di un Blocco PL/SQL

Un programma PL/SQL è organizzato in **blocchi**. La sintassi base è la seguente:

```
[HEADER]
DECLARE
   -- Sezione Dichiarativa (opzionale)
   -- Variabili, costanti, tipi, cursori
BEGIN
   -- Sezione Esecutiva (OBBLIGATORIA)
   -- Istruzioni SQL e logica procedurale
EXCEPTION
   -- Sezione Gestione Eccezioni (opzionale)
   -- Codice eseguito solo se si verifica un errore
END;
/
```

Regole Fondamentali:

1. **Solo BEGIN...END è obbligatorio.** Le sezioni DECLARE ed EXCEPTION possono essere omesse se non servono.
2. **Sezione Dichiarativa:** Definisce tutto ciò che il blocco utilizzerà. Le variabili possono essere inizializzate qui.
3. **Sezione Esecutiva:** Contiene la logica operativa e le query SQL. È il cuore del blocco.
4. **Sezione Exception:** Gestisce gli errori a runtime. Il codice al suo interno viene ignorato finché non si verifica un'eccezione.

ESEMPIO DI CATTURA DELL'ERRORE NELLA SEZIONE EXCEPT

```
DECLARE
   Error_code NUMBER(5);
   Error_msg  VARCHAR2(200);
   User       VARCHAR2(50);
   Error_info VARCHAR2(100);
BEGIN
   -- ... istruzioni che potrebbero generare errori ...
EXCEPTION
   WHEN OTHERS THEN
      Error_code := SQLCODE;
      Error_msg  := SQLERRM;
      User       := USER;
      Error_info := 'Errore rilevato il ' || TO_CHAR(SYSDATE) || ' dall''utente ' ||
User;
      INSERT INTO log_table (code, message, info)
      VALUES (Error_code, Error_msg, Error_info);
END;
```

- **SQLCODE**: Restituisce il codice numerico dell'errore.
- **SQLERRM**: Restituisce il messaggio testuale dell'errore.
- **WHEN OTHERS**: Intercetta *qualsiasi* tipo di errore non gestito specificamente

Tipi di Blocchi PL/SQL

I blocchi possono essere sequenziali o annidati.

A. Blocchi Anonimi e con Nome

Sono costruiti dinamicamente ed eseguiti una sola volta.

- **Anonimo:** Non ha un nome identificativo.
- **Con Nome:** Ha un'etichetta (label) <<nome>> che lo identifica.

```
-- Blocco Anonimo
DECLARE
   Nome VARCHAR2(50) := 'Pippo';
BEGIN
   INSERT INTO Prova(nome) VALUES (Nome);
END;

-- Blocco con Nome
<<prova>>
DECLARE
   Nome VARCHAR2(50) := 'Pippo';
BEGIN
   INSERT INTO Prova(nome) VALUES (Nome);
END;
```

B. Sottoprogrammi (Procedure e Funzioni)

Sono blocchi con nome, memorizzati nel database, richiamabili più volte. Possono avere parametri di input/output.

```
CREATE PROCEDURE ProvaIns AS
Nome VARCHAR2(50):='Pippo';
BEGIN
INSERT INTO Prova(nome)
VALUES (Nome);
END;
```

C. Trigger

Sono blocchi con nome memorizzati nel DB, eseguiti automaticamente al verificarsi di un evento specifico (es. un INSERT su una tabella).

```
CREATE OR REPLACE TRIGGER
Provatr
BEFORE INSERT OF nome
ON Prova
FOR EACH ROW
DECLARE
 cont NUMBER(5);
 info VARCHAR2(100):=
 'TableSpace Esaurito';
BEGIN
 SELECT COUNT(nome)
 INTO cont
 FROM Prova;
EXCEPTION
 WHEN cont>50
 INSERT INTO Log(msg)
 VALUES (info);
END;
```

Commenti

- **Single-line:** -- Questo è un commento su una riga
- **Multi-line:** /* Questo è un commento su più righe */

Variabili, Tipi e Strutture Dati

Le variabili sono dichiarate all'interno della sezione dichiarativa di un blocco PL/SQL

Attraverso la sintassi: DECLARE Codice NUMBER(10); si definisce una variabile "Codice" di tipo numerico

Famiglie di Tipi PL/SQL

Famiglia	Tipi Principali
Numerici	NUMBER, INTEGER, DECIMAL, REAL, BINARY_INTEGER, NATURAL, POSITIVE
Carattere	VARCHAR2, CHAR, LONG
Data	DATE
Booleani	BOOLEAN (valori: TRUE, FALSE, NULL)

%TYPE e %ROWTYPE

Sono attributi utilissimi per allineare le variabili PL/SQL alla struttura del database:

- **%TYPE**: Eredita il tipo di una colonna specifica.

```
Titolo bookinfo.booktitle%TYPE;
```

- **%ROWTYPE**: Eredita l'intera struttura di una riga di una tabella (utile per i cursori)

```
ImpiegatoRec Impiegato%ROWTYPE;
```

Tipi Definiti dall'Utente (SUBTYPE)

Un tipo definito dall'utente o subtype è un tipo PL/SQL basato su tipi esistenti. Permettono di creare alias di tipi esistenti per rendere il codice più leggibile.

```
DECLARE
   SUBTYPE Contatore IS NUMBER(4);
   cont Contatore;
```

RECORD (simile alle struct in C)

Permette di raggruppare variabili diverse in un'unica unità logica.

```
DECLARE
   TYPE ImpiegatoRecord IS RECORD (
      CF      NUMBER(20),
      Nome    VARCHAR2(50),
      Cognome VARCHAR2(50)
   );
   Impiegato ImpiegatoRecord;
BEGIN
   Impiegato.Nome := 'Pippo';
END;
```

Tabelle PL/SQL (Array Associativi)

Simili agli array, permettono di memorizzare collezioni di dati in memoria.

```
DECLARE
   TYPE Char_Table IS TABLE OF VARCHAR2(10)
   INDEX BY BINARY_INTEGER;
   tabella Char_Table;
BEGIN
   tabella(1) := 'Primo';
   tabella(2) := 'Secondo';
END;
```

Attributi utili delle tabelle:

- COUNT: Numero di elementi.
- FIRST / LAST: Indice del primo/ultimo elemento.
- NEXT(n) / PRIOR(n): Indice dell'elemento successivo/precedente.
- EXISTS(n): TRUE se l'elemento all'indice *n* esiste.
- DELETE: Cancella gli elementi.

CONVERSIONE TRA TIPI

PL/SQL mette a disposizione dell'utente funzioni per la conversione tra variabili appartenenti a tipi diversi

Funzioni:

- TO_CHAR
- TO_DATE
- TO_NUMBER

5. Assegnazione e Operatori

- **Assegnazione:** Si usa :=

```
Tax := price * tax_rate;
```

- **Assegnazione da query (INTO):**

```
SELECT sal INTO current_salary FROM emp WHERE ename = 'SMITH';
```

- **Operatori di confronto:** <, >, =, !=, BETWEEN, IS NULL, LIKE.

6. Strutture di Controllo

Selezione (IF, THEN, ELSIF, ELSE .... END IF)

```
IF acc_bal >= DEBIT_AMT THEN
   UPDATE accounts SET bal = bal - DEBIT_AMT WHERE account_id = ACCT;
ELSE
   INSERT INTO temp VALUES (ACCT, acc_bal, 'Fondi insufficienti.');
END IF;
```

Loop (Cicli Iterativi)

PL/SQL offre tre tipi di loop:

1. **Loop Semplice** (con uscita condizionale):

```
LOOP
   INSERT INTO Prova(id) VALUES(contatore);
   contatore := contatore + 1;
   EXIT WHEN contatore > 10;
END LOOP;
```

2. **Loop FOR** (contatore automatico):

```
FOR contatore IN 1..10 LOOP
   INSERT INTO Prova(id) VALUES(contatore);
END LOOP;
```

3. **Loop WHILE** (ciclo finché la condizione è vera):

```
WHILE contatore <= 10 LOOP
   INSERT INTO Prova(id) VALUES(contatore);
   contatore := contatore + 1;
END LOOP;
```

GOTO: Permette un salto incondizionato a un'etichetta (da usare con cautela).

```
IF contatore > 10 THEN
   GOTO label_endloop;
END IF;
...
<<label_endloop>>
END;
```

7. SQL in PL/SQL e Cursori

Statement SQL supportati

- **DML:** SELECT, INSERT, UPDATE, DELETE
- **DDL:** CREATE TABLE, CREATE INDEX, ecc. (tramite SQL dinamico)

Il Problema dell'Impedance Mismatch

SQL lavora su **insiemi di righe**, mentre i linguaggi procedurali lavorano su **record singoli**. La soluzione sono i **Cursori**.

Cursori Espliciti

Servono per elaborare riga per riga il risultato di una query che restituisce più tuple.

Ciclo di vita di un cursore:

1. **DECLARE:** Definisce il cursore e la query associata.
2. **OPEN:** Inizializza il cursore ed esegue la query.
3. **FETCH:** Recupera la riga corrente e sposta il puntatore alla successiva.
4. **CLOSE:** Chiude il cursore liberando le risorse.

Attributi del cursore:

- c1%NOTFOUND: TRUE se l'ultimo FETCH non ha trovato righe.
- c1%FOUND: Opposto di NOTFOUND.
- c1%ROWCOUNT: Numero di righe già recuperate.
- c1%ISOPEN: TRUE se il cursore è aperto.

Esempio completo:

```
DECLARE
   CURSOR c1 IS SELECT n1, n2, n3 FROM data_table WHERE exper_num = 1;
   num1 data_table.n1%TYPE;
   num2 data_table.n2%TYPE;
   num3 data_table.n3%TYPE;
   Result temp.col1%TYPE;
BEGIN
   OPEN c1;
   LOOP
      FETCH c1 INTO num1, num2, num3;
      EXIT WHEN c1%NOTFOUND;
      Result := num2 / (num1 + num3);
      INSERT INTO temp VALUES (result, NULL, NULL);
   END LOOP;
   CLOSE c1;
   COMMIT;
END;
```

Cursor FOR LOOP (versione semplificata):

Gestisce automaticamente OPEN, FETCH e CLOSE.

```
FOR c1_rec IN c1 LOOP
   result := c1_rec.n2 / (c1_rec.n1 + c1_rec.n3);
   INSERT INTO temp VALUES (result, NULL, NULL);
END LOOP;
```

Cursore Implicito (SQL%)

Oracle apre automaticamente un cursore per ogni statement SQL eseguito. Si riferisce ad esso con SQL%.

- SQL%ROWCOUNT: Numero di righe modificate dall'ultima operazione DML.
- SQL%NOTFOUND: TRUE se l'operazione non ha modificato nessuna riga.

8. Gestione delle Transazioni

Una transazione è un'unità logica di lavoro. PL/SQL supporta:

- **COMMIT**: Rende permanenti tutte le modifiche della transazione corrente.
- **ROLLBACK**: Annulla tutte le modifiche dall'inizio della transazione.
- **SAVEPOINT nome**: Crea un punto di ripristino parziale all'interno della transazione.

9. Procedure, Funzioni e Package

Creazione di una Procedura

```
CREATE OR REPLACE PROCEDURE MyInsert (
   CF_in       IN Impiegato.CF%TYPE,
   Nome_in     IN Impiegato.Nome%TYPE,
   Cognome_in  IN Impiegato.Cognome%TYPE
) AS
BEGIN
   INSERT INTO Impiegato(CF, Nome, Cognome)
   VALUES (CF_in, Nome_in, Cognome_in);
   COMMIT;
END MyInsert;
```

Creazione di una Funzione

Simile a una procedura, ma **restituisce un valore**.

```
CREATE OR REPLACE FUNCTION MyCheck RETURN BOOLEAN IS
   cont NUMBER(5);
BEGIN
   SELECT COUNT(*) INTO cont FROM Impiegato;
   IF cont > 50 THEN
      RETURN TRUE;
   ELSE
      RETURN FALSE;
   END IF;
END MyCheck;
```

Package

Un **Package** è un contenitore che raggruppa procedure, funzioni, variabili e tipi logicamente correlati. Si compone di due parti:

- **Specification (Header):** Dichiara gli elementi pubblici.
- **Body:** Contiene l'implementazione effettiva.

10. Trigger in PL/SQL

Un trigger è una routine PL/SQL eseguita automaticamente al verificarsi di un evento (es. INSERT, UPDATE, DELETE su una tabella).

```
CREATE [OR REPLACE] TRIGGER trigger_name
[BEFORE | AFTER] evento_dml
ON nome_tabella
[FOR EACH ROW]
[WHEN (condizione)]
BEGIN
   -- corpo del trigger
END;
```

Categoria	Valori	Significato
Statement	INSERT, UPDATE, DELETE	Tipo di operazione DML che attiva il trigger
Timing	BEFORE, AFTER	Quando si attiva (prima o dopo l'operazione)
Level	ROW, STATEMENT	Se si attiva per ogni riga modificata o una volta sola per istruzione

SISTEMI INFORMATIVI DIREZIONALI

1. Cos'è un Sistema Informativo

Un **sistema informativo** è l'insieme di tutti i flussi informativi (elaborati manualmente o automaticamente) necessari a un'organizzazione per raggiungere i propri obiettivi.

Funzione principale:

- Affianca tutti i processi aziendali
- Agevola l'esecuzione delle attività
- Migliora i rendimenti generali dell'organizzazione

2. I Due Livelli Aziendali

Livello Direzionale (Strategico)

Si occupa di:

- Definire gli **obiettivi** da raggiungere
- Stabilire le **strategie** per perseguirli

fornirà al livello **operativo** le **istruzioni** sulle strategie da seguire per raggiungere gli **obiettivi** aziendali

Livello Operativo (Esecutivo)

Si occupa di:

- Attività per produrre i **servizi e/o prodotti** aziendali

fornirà a sua volta al livello **direzionale** le informazioni sui risultati raggiunti.

4. La Piramide di Anthony

La piramide di Anthony suddivide le attività direzionali in tre livelli gerarchici:

Livello	Funzione	Orizzonte Temporale
Pianificazione Strategica	Determina gli obiettivi generali dell'azienda	Lungo termine
Controllo Direzionale (di Gestione)	Definisce traguardi economici a medio termine e ne verifica il raggiungimento	Medio termine
Controllo Operativo	Assicura che le attività dei processi aziendali procedano come pianificato	Breve termine

Ogni livello della piramide necessita di informazioni specifiche:

Pianificazione Strategica:

- Scenari macroeconomici
- Analisi di mercato
- Trend settoriali

Controllo Direzionale:

- Verifica del raggiungimento degli obiettivi a medio termine
- Indicatori di performance gestionale
- Report economici e finanziari

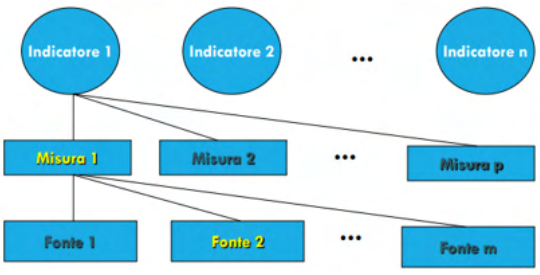
Controllo Operativo:

- Informazioni continue dai processi aziendali
- Verifica del rispetto dei piani esecutivi
- Dati operativi dettagliati

Caratteristica dei sistemi direzionali: Possono essere alimentati:

- Automaticamente da altri sistemi informativi
- Manualmente tramite inserimento dati

5. Il Paradigma "Indicatori – Misure – Fonti"

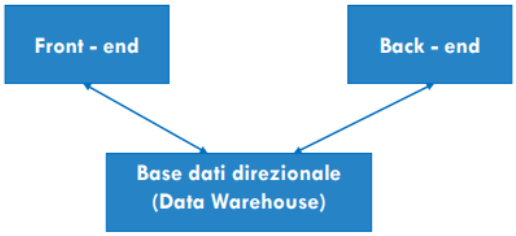


I sistemi direzionali lavorano con informazioni **fortemente aggregate** per fornire ai dirigenti **informazioni sintetiche** chiamate **indicatori prestazionali** (o KPI - Key Performance Indicators).

Esempio pratico:

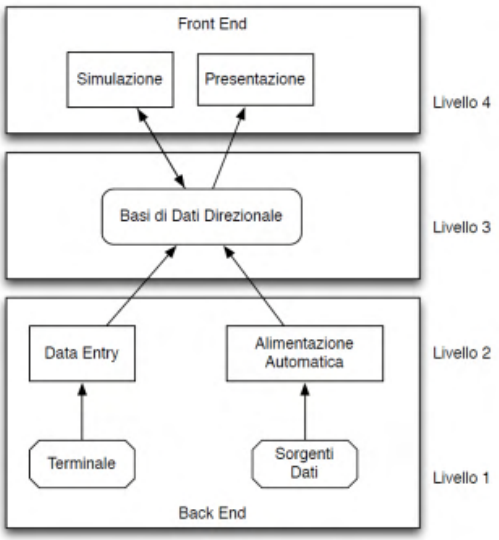
- **Indicatore:** "Fatturato totale trimestre"
- **Misure:** "Vendite per regione", "Vendite per prodotto"
- **Fonti:** "Transazioni di vendita giornaliere", "Ordini clienti"

6.Architettura Generale dei SID



Un Sistema Informativo Direzionale (SID) si compone di tre macro-componenti fondamentali:

- **Front-end:** Si occupa della **presentazione** delle informazioni all'utente finale (report, grafici, simulazioni).
- **Back-end:** Si occupa di **alimentare** il Data Warehouse, estraendo e trasformando i dati dai sistemi operativi.
- **Data Warehouse (DWH):** È il cuore del sistema, il repository centrale dei dati direzionali.



LIVELLO 1 - Sorgenti Dati (Back-end)

È la base del sistema e rappresenta le **fonti dei dati grezzi** che possono essere **prelevate automaticamente** secondo strategie che dipendono dall'applicazione.

Sono presenti:

- **Basi di dati transazionali**: i database operativi aziendali.
- **Altre sorgenti:** file piatti (flat files), documenti, fogli di calcolo.
- **Dati manuali:** informazioni di carattere generale non presenti nei sistemi operativi, inserite manualmente dagli utenti finali tramite terminale.

LIVELLO 2 - Interfaccia di Alimentazione (Back-end)

Fa da ponte tra le sorgenti dati (Livello 1) e il Data Warehouse (Livello 3). È composto da due sottosistemi:

- **Data Entry:** Permette l'acquisizione manuale dei dati non ricavabili automaticamente.
- **Alimentazione Automatica:** Estrae i dati operazionali e li **trasforma in macro-informazioni** (aggregate e temporizzate) per il Data Warehouse.

LIVELLO 3 - Base di Dati Direzionale (Data Warehouse)

È il cuore del sistema. Contiene i dati direzionali organizzati secondo uno **schema multidimensionale** (non più relazionale classico). Questo livello sarà approfondito nel dettaglio in questo documento.

LIVELLO 4 - Front-end (Presentazione e Analisi)

Realizza l'interfaccia utente (front-end) tramite strumenti software di **OLAP** (*On Line Analytical Processing*).

Requisiti FASMI: Le funzionalità OLAP devono rispettare l'acronimo **FASMI**:

- **F**ast (Veloci): recupero rapido dei dati nonostante la mole di elaborazioni.
- **A**nalysis (Analisi): capacità di analisi multidimensionale.
- **S**hared (Condivisi): accesso contemporaneo da parte di più utenti.
- **M**ultidimensional (Multidimensionali): gestione di più dimensioni di analisi.
- **I**nformation (Informazioni): accesso a informazioni di alto livello.

7.Il Data Warehouse (DWH)

Il Data Warehouse è la **soluzione relazionale** per la rappresentazione della base di dati direzionale (Livello 3).

Caratteristica	Significato
Orientata ai soggetti	Organizzata per aree tematiche (es. vendite, clienti, prodotti) e non per processi operativi
Integrata	I dati provenienti da fonti diverse sono uniformati e coerenti tra loro
Tempo-variante	I dati sono storici: ogni informazione è associata a un periodo temporale
Non volatile	I dati, una volta caricati, non vengono modificati o cancellati, ma solo consultati

Inoltre, il DWH è **ricavato dagli ambienti operativi (OLTP)** ed è **fisicamente separato** da essi.

Obiettivi del Data Warehouse

Rendere l'informazione aziendale:

- **Accessibile:** facile da consultare per i decisori
- **Consistente:** coerente e uniforme
- **Affidabile:** basata su dati verificati
- **Sicura:** protetta da accessi non autorizzati
- **Usabile:** pronta per supportare le decisioni strategiche

Orientata al Soggetto (Subject-Oriented)

Il DWH è una collezione di dati organizzata attorno ai **soggetti dell'elaborazione** (es. Clienti, Prodotti, Vendite), e non attorno ai processi operativi che generano i dati.

- **DWH:** risponde alla domanda *"Di cosa vogliamo analizzare i dati?"*
- **DB Operativo:** risponde alla domanda *"Quale operazione stiamo svolgendo?"*

Questa è la differenza principale rispetto ai database classici, che hanno un orientamento **processivo/funzionale** (es. gestione ordini, gestione fatture, gestione spedizioni).

Integrata

I dati del DWH provengono da **fonti diverse** (diverse applicazioni, diversi database operativi) e quindi possono avere **formati, convenzioni e strutture inconsistenti**. Prima di essere caricati nel DWH, i dati devono essere **integrati e uniformati**.

L'integrazione si manifesta in diversi aspetti:

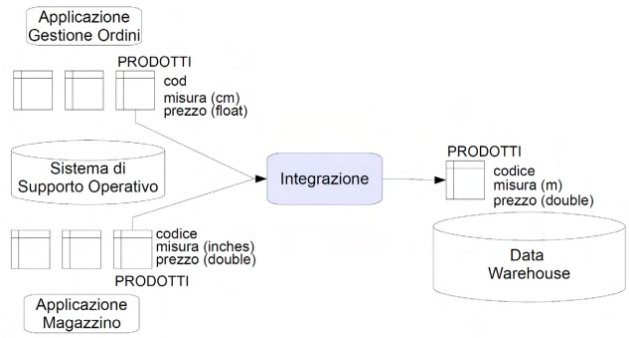
- **Misure consistenti:** le variabili devono essere espresse nella stessa unità di misura.
- **Attributi fisici:** i tipi di dato devono essere uniformati.
- **Strutture di codifica:** i codici devono seguire la stessa convenzione.
- **Convenzioni sui nomi:** gli attributi devono essere chiamati allo stesso modo.

ESEMPIO

Applicazione	Campo Codice	Campo Misura	Campo Prezzo
Gestione Ordini	cod	misura (cm)	prezzo (float)
Magazzino	codice	misura (inches)	prezzo (double)

Prima di caricare questi dati nel Data Warehouse, il processo di **integrazione** deve:

1. **Uniformare i nomi:** cod e codice ⟶ codice
2. **Uniformare le unità di misura:** cm e inches ⟶ m (metri)
3. **Uniformare i tipi di dato:** float e double ⟶ double



Tempo-Variante

Tutti i dati contenuti in un DWH sono associati a un **preciso arco temporale**. Questa è una differenza cruciale rispetto ai database operativi.

Aspetto	Database Operativo	Data Warehouse
Orizzonte temporale	Breve (giorni, settimane)	Lungo (5, 10 anni o più)
Dati correnti	Solo lo stato attuale	Storico completo
Riferimento temporale	Implicito	Esplicito (chiave temporale)

Ogni struttura del DWH contiene, in modo **implicito o esplicito**, un riferimento a un valore temporale. Questo riferimento è gestito tramite una **tabella dei tempi** (dimensione tempo), che permette agli utenti direzionali di analizzare l'evoluzione dei fenomeni nel tempo (es. "vendite del 2023 rispetto al 2022").

Non Volatile

A differenza di un database tradizionale, dove i dati possono essere **inseriti, modificati, cancellati e consultati**, i dati di un DWH possono essere solo:

- **Caricati** (una volta, in fase di alimentazione)
- **Consultati** (per analisi)

Perché? Perché i dati del DWH rappresentano **istantanee storiche della realtà aziendale** (snapshot). Una volta registrata una vendita nel 2020, quel dato non viene più modificato: rappresenta la realtà di quel momento. Questo garantisce:

- **Storicità** dei dati
- **Riproducibilità** delle analisi
- **Affidabilità** delle decisioni basate sui dati

DWH e Ambienti Operativi: La Questione della Ridondanza

Poiché i dati del DWH provengono dagli ambienti operativi, potrebbe sembrare che ci sia una **notevole ridondanza** (duplicazione) dei dati. In realtà, **questa ridondanza è minima** per tre motivi:

1. **Filtraggio:** I dati vengono filtrati e selezionati prima di passare dagli ambienti operativi al DWH. Non tutto viene copiato.
2. **Differente arco temporale:** I due sistemi coprono periodi temporali diversi. Il DWH contiene dati più vecchi e storici, mentre il DB operativo contiene solo i dati correnti.

3. **Dati riassuntivi:** Il DWH contiene dati **aggregati e riassuntivi** (es. "vendite totali per mese") che non sono esplicitamente rappresentati nei database operativi (che registrano solo le singole transazioni).

In sintesi: Il DWH non è una semplice copia del database operativo, ma una **rappresentazione diversa, integrata e storica** delle informazioni aziendali, ottimizzata per l'analisi e non per l'operatività quotidiana.

Business Intelligence (BI)

La **Business Intelligence (BI)** è il processo di trasformazione di dati e informazioni in **conoscenza** utile per l'organizzazione.

Scopo delle tecnologie BI:

- Supportare l'organizzazione nello sfruttare al meglio il proprio patrimonio informativo (interno ed esterno) nei **processi decisionali** (*decision making*).
- Fornire viste **storiche, correnti e predittive** sui processi di business.

Sistema DW/BI

Un Data Warehouse ha senso solo come parte di un sistema più ampio: il **sistema DW/BI**, che racchiude:

- Il **sottosistema Data Warehouse (DW)**: i dati stessi (Enterprise Data Warehouse).
- Il **sottosistema Business Intelligence (BI)**: le applicazioni che interrogano e analizzano i dati.

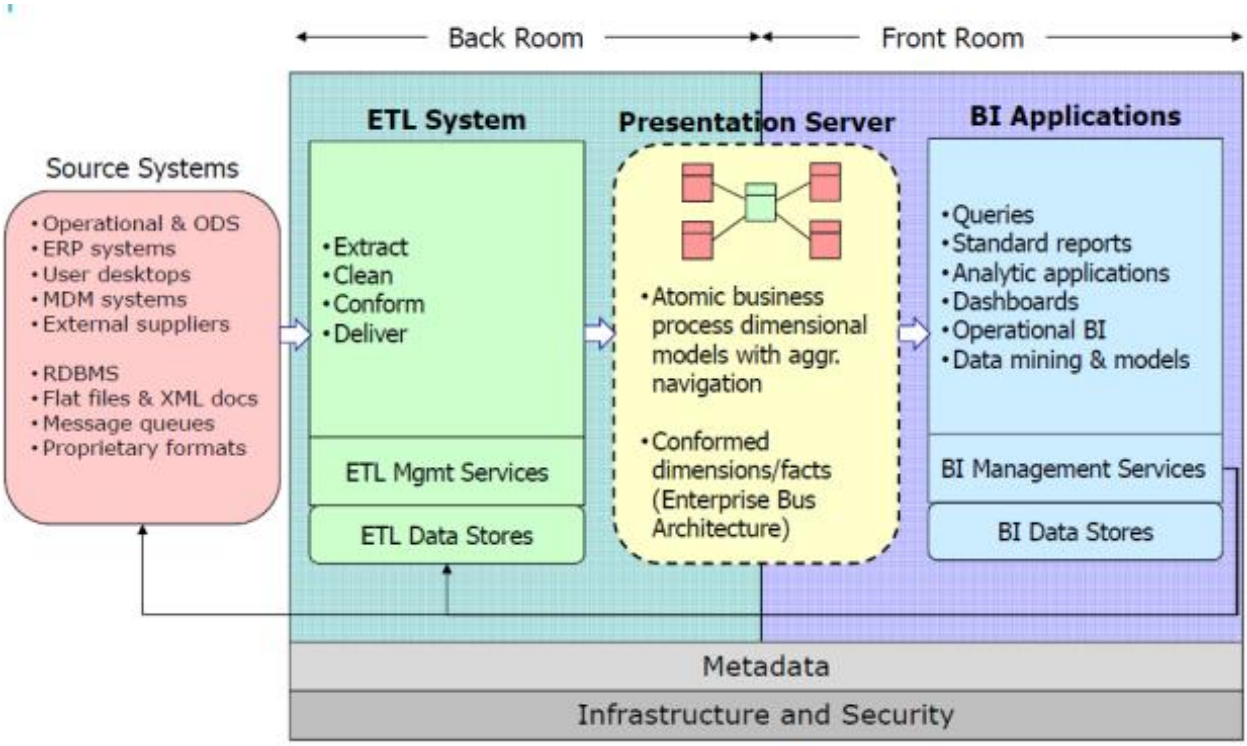
Il Data Warehouse costituisce le fondamenta per la BI.

Funzioni principali del sistema DW/BI

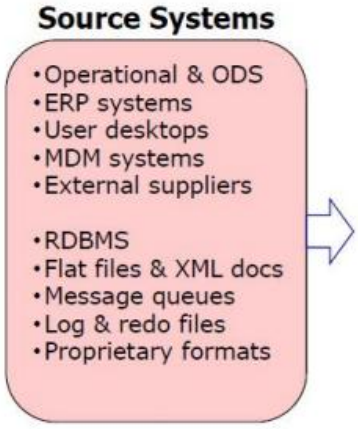
1. **Estrarre** i dati dai sistemi sorgente, pulirli, allinearli, conformarli e trasportarli verso il DW.
2. **Rendere disponibili** questi dati agli utenti finali in modo efficace.

Architettura di un Sistema DW/BI

L'architettura si divide in **tre macro-aree** principali, più i sistemi sorgente esterni:



I Sistemi Sorgente (Source Systems)



I sistemi sorgente forniscono i dati grezzi al sistema DW/BI e comprendono:

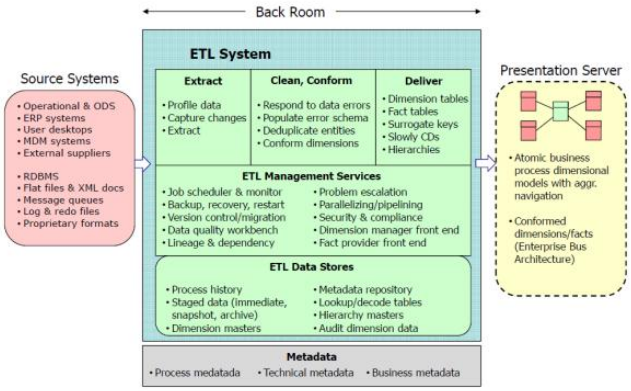
Sorgenti interne (sistemi operazionali):

- Sistemi transazionali **OLTP** (gestione ordini, inventario, contabilità, ecc.)
- **Nota:** di solito non mantengono dati storici e possono essere sistemi "legacy" (obsoleti).

Sorgenti esterne:

- Dati pubblici o forniti da società specializzate di analisi
- Spesso relativi al segmento di business di interesse

Back Room (Area di Staging ed ETL)



La **Back Room**, anche detta area di preparazione dei dati, è la parte **privata** del DW dove avviene l'elaborazione **ETL** (*Extract-Transform-Load*).

l'interesse primario è far muovere i dati dal punto A (sorgenti) al punto B (Presentation Server), dopo un'appropriata trasformazione e nel momento appropriato.

Le 4 operazioni principali dell'ETL

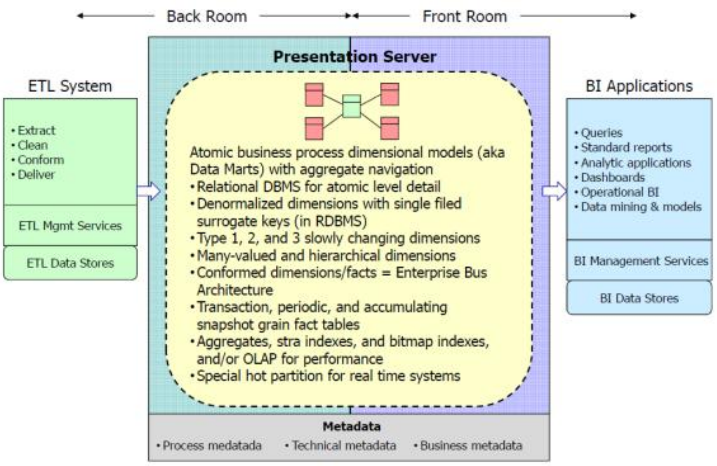
- **estrazione dei dati** dalle sorgenti informative
- applicazione di un insieme di **trasformazioni di pulitura e conformazione**
- **caricamento nel DW** – ovvero, nei server di presentazione
- **gestione dei processi ETL** e dell'ambiente della back room

L'Area di Staging

I dati integrati, corretti, filtrati e validati vengono materializzati nell'**area di staging**, che contiene i **dati riconciliati**:

- Costituiscono un **modello di dati comune e di riferimento** per l'azienda.
- Sono **pronti** per alimentare il DWH.

Il Presentation Server



Il **Presentation Server** è la piattaforma target in cui sono **effettivamente memorizzati i dati del Data Warehouse**, pronti per essere interrogati dagli utenti finali e dalle applicazioni BI.

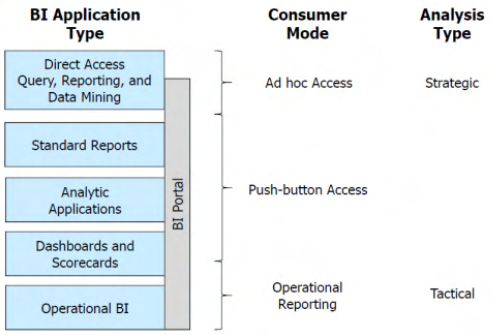
1. Caratteristiche Principali (A cosa serve)

- **Focalizzato sull'utente:** I dati sono organizzati in base a *cosa serve agli analisti*, non in base a come funzionano i sistemi operativi.
- **Database unico:** C'è un'unica base di dati per tutte le analisi.
- **Doppio livello di dettaglio:** Permette di consultare sia i **dati atomici** (il dettaglio massimo, es. la singola vendita) sia i **dati aggregati** (i riassunti, es. il totale vendite mensile).

2. Come sono organizzati i dati

- **Modello Dimensionale:** I dati sono divisi in **Fatti** (i numeri da misurare, es. importo vendita) e **Dimensioni** (il contesto, es. data, prodotto, cliente).
- **Velocità:** Tutto è progettato per essere letto e consultato il più velocemente possibile.

La Front Room (Applicazioni BI)



La **Front Room** è la **faccia pubblica** del sistema DW/BI, composta dalle applicazioni di Business Intelligence, **le applicazioni BI accedono direttamente ai dati del DW**

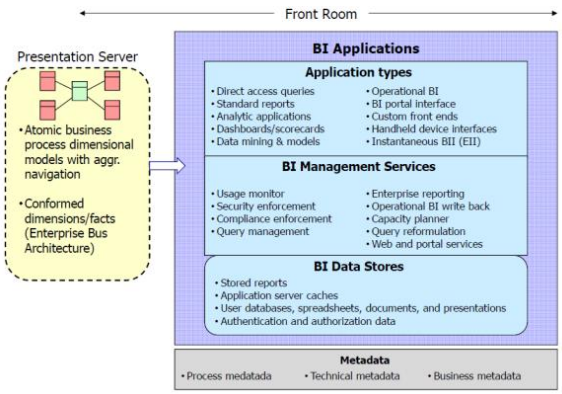
Obiettivi delle applicazioni BI

Poiché i dati nel DW sono molto complessi, le applicazioni BI devono:

- **Nascondere la complessità** dei dati.
- **Aiutare gli utenti** a trovare quello che cercano.
- **Erogare i risultati** nelle forme e nei formati richiesti.

Non tutti gli utenti hanno le stesse necessità di analisi, per questo esistono **diversi tipi di applicazioni BI**, che vanno da s emplici report standard parametrizzati a sistemi complessi di analisi.

Il Ciclo di Analisi di Business (Front Room)



Il Ciclo di Analisi di Business è il processo logico e continuo che le aziende seguono per prendere decisioni strategiche basate sui dati. Si tratta di un percorso circolare che parte dal monitoraggio e arriva all'azione, per poi ricominciare.

Il ciclo si articola in **5 fasi fondamentali**:

1. Attività di Monitoraggio

È la fase di controllo continuo dell'andamento aziendale.

- **Come avviene:** Attraverso l'uso di **report standard, dashboard e scorecard**.
- **Cosa si fa:** Si confrontano i risultati attuali con quelli previsti (budget) o con i dati storici (periodi precedenti).

2. Identificazione delle Eccezioni

Durante il monitoraggio, si notano degli scostamenti (variazioni) rispetto alle prestazioni normali.

- **Significato:** Queste eccezioni non sono solo "errori", ma possono indicare un **problema** da risolvere (es. calo delle vendite) oppure un'**opportunità** da cogliere (es. picco di richieste in una zona specifica).

3. Determinazione delle Cause (Root Cause Analysis)

Una volta individuata l'eccezione, bisogna capire *perché* si è verificata.

- **Come si fa:** Si analizzano i dati in profondità per trovare la causa radice.
- **Strumenti utilizzati:** Analisi statistiche, **Data Mining**, o accesso a ulteriori informazioni di dettaglio.

4. Valutazione delle Alternative Decisionali

Capita la causa, si valutano le possibili soluzioni.

- **Modellizzazione:** Si costruiscono modelli che descrivono le relazioni causa-effetto dei fenomeni.
- **Simulazione:** I dati del Data Warehouse vengono usati per **validare i modelli** e prevedere l'impatto delle diverse decisioni.
- **Strumenti:** Analisi statistiche, simulazioni e analisi **What-if** (*"Cosa succederebbe se..."*).

5. Intraprendere Azioni e Tracciare i Risultati

È la fase operativa in cui si passa dalle parole ai fatti.

- **Azione:** Le decisioni raccomandate vengono applicate ai **sistemi operazionali** aziendali.
- **Tracciamento:** Si avvia immediatamente la misurazione e il monitoraggio dei risultati di queste azioni.
- **Il ciclo riparte:** I nuovi risultati tornano nella Fase 1 (Monitoraggio), chiudendo il cerchio e avviando un nuovo ciclo di miglioramento continuo.

Warehousing Integration

Oltre ai dati veri e propri, il DWH contiene anche i **metadati** (dati sui dati), indispensabili per la gestione del sistema stesso.

I Metadati del DWH

I metadati sono "dati sui dati" e permettono di gestire e comprendere il contenuto del Data Warehouse. Sono organizzati in un **catalogo relazionale** (una vera e propria base di dati di metadati).

Si dividono in **3 categorie principali**:

Categoria	Cosa descrive	Esempio
Loading (Caricamento)	Sorgenti dati e trasformazioni da applicare	"Il campo prezzo viene estratto dall'ERP e convertito in euro"
Query Manager	Profili delle query degli utenti	"Il reparto marketing esegue spesso query sulle vendite per regione"
Data Management	Organizzazione fisica dei dati	"La tabella Vendite ha un indice sulla colonna Data"

Data Mart

Un **Data Mart** è un sottoinsieme del DWH, focalizzato su una **specifica area di business** o dipartimento (es. Vendite, Produzione, Marketing).

Caratteristiche principali

- Contiene dati **fortemente aggregati**
- È **più piccolo** e quindi più facile da gestire e navigare
- Può essere **autonomo** o **collegato** al DWH centrale
- Non contiene dati operativi, ma solo informazioni direzionali.

I 5 Flussi del Data Warehouse

Il movimento dei dati nel DWH è gestito attraverso 5 processi distinti:

📥 INFLOW (Flusso in ingresso)

Processi di **estrazione, filtraggio e caricamento** dei dati dai sistemi OLTP al DWH.

- I dati vengono estratti dalle sorgenti
- Vengono **riorganizzati** (normalizzazione, aggiunta/rimozione campi)
- Si verifica la **consistenza** con i dati già presenti
- Passano attraverso una **memoria temporanea** (staging area) per i test di consistenza e alle operazioni di filtraggio

Esempio pratico:

Un'azienda ha 3 sistemi diversi: uno per le vendite in Italia (euro), uno per le vendite in USA (dollari), uno per il magazzino. L'INFLOW estrae i dati da tutti e 3, converte tutto in euro, uniforma i codici prodotto e li carica nel DWH in un formato unico e coerente.

📈 UPFLOW (Flusso verso l'alto)

Processi associati **all'aggiunta** di dati al DWH attraverso operazioni di **aggregazione**.

Cosa succede durante l'UPFLOW:

I dati operazionali vengono organizzati in **"varie forme"** a diversi livelli di dettaglio:

Livello	Descrizione	Esempio
Dettagliati	Dati atomici, massimo livello di dettaglio	Singola transazione di vendita del 15/03/2026

Parzialmente aggregati	Dati raggruppati per alcune dimensioni	Vendite totali per mese e regione
Fortemente aggregati	Dati molto sintetici	Fatturato annuale totale

A cosa serve: Questi diversi livelli di aggregazione sono utili per agevolare gli utenti finali nella creazione di:

- Diagrammi
- Animazioni
- Grafici
- Dashboard

📉 DOWNFLOW (Flusso verso il Basso)

Definizione: Processi associati all'**archiviazione sicura** dei dati nel DWH.

Cosa succede durante il DOWNFLOW:

- **Backup periodici** dei dati del DWH
- **Replicazione** dei dati su sistemi ridondanti
- **Logging** delle operazioni effettuate
- **Archiviazione** storica dei dati

Scopo fondamentale: Garantire la **ricostruzione dello stato del DWH** in caso di:

- Guasti hardware
- Perdita di dati
- Errori di sistema
- Disaster recovery

📤 OUTFLOW (Flusso in Uscita)

Definizione: Insieme dei processi demandati a **rendere disponibili i dati all'utente finale.**

Cosa succede durante l'OUTFLOW:

- Gestione delle **richieste di query** dagli utenti
- Ottimizzazione delle **prestazioni di accesso**
- Consegna dei dati nei **formati richiesti** (tabelle, grafici, report)

- Gestione della **concorrenza** (più utenti che accedono contemporaneamente)

Obiettivo principale: Soddisfare le richieste degli utenti in maniera **efficiente** e rapida.

🔄 METAFLOW (Flusso dei Metadati)

Definizione: Processi associati alla **gestione dei metadati** (dati sui dati).

Cosa succede durante il METAFLOW:

- **Aggiornamento** del catalogo dei metadati
- **Manutenzione** delle informazioni sulla struttura del DWH
- **Tracciamento** della provenienza dei dati (data lineage)
- **Gestione** delle regole di trasformazione
- **Monitoraggio** della qualità dei dati

Query & Reporting (Q&R)

Il **Query & Reporting** è il processo fondamentale per estrarre informazioni dal Data Warehouse. In parole semplici, è il meccanismo che permette a un utente di "fare una domanda" al database e ricevere una risposta strutturata.

Il processo si articola in **5 fasi sequenziali**:

Le 5 Fasi del Processo Q&R

1. **Query Definition (Definizione della Query)**
    - *Cosa si fa:* L'utente formula la domanda o il requisito informativo.
    - *Esempio:* "Voglio sapere quanti aspirapolvere sono stati venduti..."
2. **Data Access and Retrieval (Accesso e Recupero Dati)**
    - *Cosa si fa:* Il sistema interroga il Data Warehouse e preleva i dati grezzi necessari.
    - *Risultato:* Si ottiene un **Answer Set** (l'insieme dei dati grezzi recuperati).
3. **Calculation Manipulation (Manipolazione e Calcolo)**
    - *Cosa si fa:* I dati grezzi vengono elaborati. Si effettuano calcoli, somme, medie o si trasformano i dati per inserirli nel contesto corretto richiesto dall'utente.
4. **Report Preparation (Preparazione del Report)**
    - *Cosa si fa:* I dati elaborati vengono organizzati e formattati in una struttura leggibile e ordinata (es. una tabella, un grafico, un PDF).

5. **Report Delivery (Consegna del Report)**
    - *Cosa si fa:* Il report finale viene inviato o reso disponibile all'utente che ha fatto la richiesta.

Analisi Multidimensionale: Concetti Base

L'analisi dei dati nel Data Warehouse avviene rappresentandoli in forma **multidimensionale**, come se fossero organizzati in un "cubo" di dati.

I 3 Concetti Fondamentali

Concetto	Definizione	Esempio
Fatto	Il concetto centrale su cui si basa l'analisi	Vendita
Misura	Una proprietà numerica (atomica) del fatto	Incasso, quantità venduta
Dimensione	Un punto di vista attraverso cui analizzare il fatto	Articolo, Luogo, Tempo

Le Gerarchie delle Dimensioni

Ogni dimensione è organizzata in una **gerarchia** che rappresenta i possibili livelli di aggregazione dei dati:

- **Dimensione Tempo:** Giorno → Mese → Trimestre → Anno
- **Dimensione Luogo:** Negozio → Città → Provincia → Regione
- **Dimensione Articolo:** Prodotto → Categoria → Famiglia

Approcci di Implementazione OLAP

Esistono due approcci principali per implementare l'analisi multidimensionale:

A. MOLAP (Multidimensional OLAP)

I dati sono archiviati in una **struttura a matrice sparsa** (database multidimensionale) dove sono già registrate tutte le sintesi statistiche possibili.

- **Come funziona:** Il visualizzatore chiede i dati direttamente al database multidimensionale.
- ✅ **Vantaggio principale: Tempi di risposta velocissimi** (i dati sono già pre-calcolati).
- ❌ **Svantaggio:** Il database multidimensionale deve essere **allineato** ad ogni aggiornamento dei dati di base.

B. ROLAP (Relational OLAP)

I dati sono archiviati in **tabelle relazionali** standard. Le sintesi vengono calcolate al volo tramite query SQL.

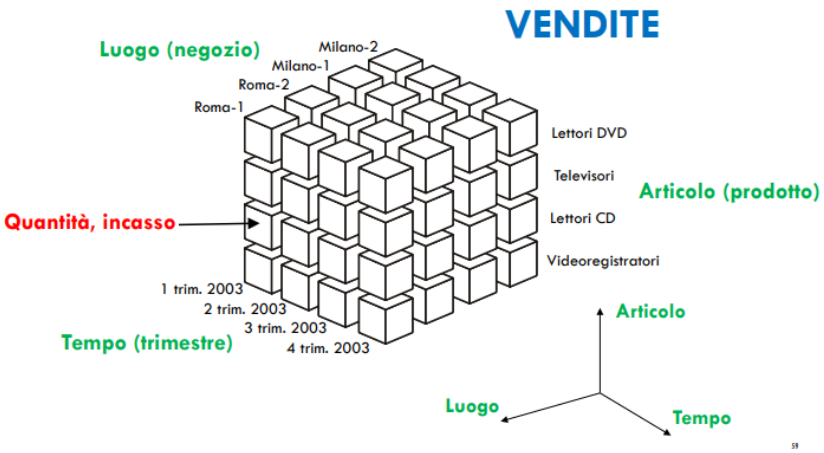
- **Come funziona:** I dati sono acceduti tramite query, poi vengono effettuate le sintesi necessarie per la visualizzazione.
- ✅ **Vantaggio principale:** I dati acceduti sono **sempre gli ultimi disponibili** (aggiornati in tempo reale).
- ❌ **Svantaggio:** Una volta usciti dal visualizzatore, i dati di sintesi **si perdono** e per riaccedervi è necessario rieseguire le estrazioni (più lento).

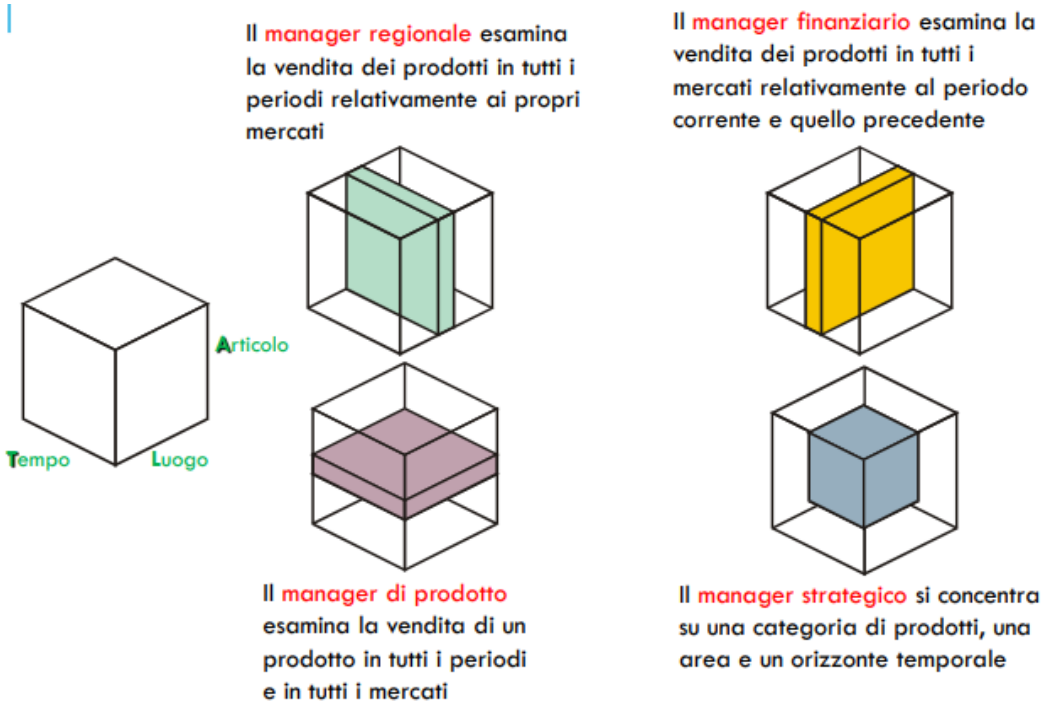
Le 4 Operazioni sui Dati Multidimensionali

Per navigare e analizzare il "cubo" dei dati, si utilizzano 4 operazioni principali:

A. Slice & Dice (Taglia e Dadi)

Cos'è: Seleziona una porzione del cubo filtrando su una o più dimensioni. Permette di "affettare" i dati per focalizzarsi su un sottoinsieme specifico.





Risultato di Slice & Dice (Esempio: Lettori DVD per negozio e trimestre):

LETTORI DVD	1 trim. 03	2 trim. 03	3 trim. 03	4 trim. 03
Roma-1	38	91	66	198
Roma-2	155	219	248	265
Milano-1	121	273	266	326
Milano-2	222	122	155	200

B. Roll-Up (Aggregazione)

Cos'è: Sale nella gerarchia delle dimensioni, **aggregando i dati** (passando dal dettaglio al riassunto).

Esempio 1 - Roll-Up per Luogo (da negozio a città): Sommando i dati dei singoli negozi (Roma-1 + Roma-2 = Roma), si ottiene:

LETTORI DVD	1 trim. 03	2 trim. 03	3 trim. 03	4 trim. 03
Roma	193	310	314	463
Milano	343	395	421	526

Esempio 2 - Roll-Up per Prodotto (da dettaglio a categoria): Aggregando tutti i negozi per categoria di prodotto:

VENDITE TRIM.	1 trim. 03	2 trim. 03	3 trim. 03	4 trim. 03
Lettori DVD	536	705	735	989
Televisori	567	716	606	717
Lettori CD	187	155	186	226
Videoregistratori	175	191	202	319

C. Drill-Down (Disaggregazione)

Cos'è: Scende nella gerarchia delle dimensioni, passando dai dati aggregati al **dettaglio**. È l'operazione inversa del Roll-Up.

Esempio: Partendo dai dati trimestrali aggregati per categoria, si scende al dettaglio mensile:

VENDITE MENSILI	Gen 03	Feb 03	Mar 03	Apr 03	Mag 03	Giu 03	...
Lettori DVD	165	178	193	205	244	256	...
Televisori	154	201	212	245	255	216	...
Lettori CD	54	88	45	24	65	66	...
Videoregistratori	56	64	55	52	64	75	...

D. Pivot (Rotazione) Cos'è: Ruota il cubo dei dati, cambiando l'orientamento delle dimensioni. Permette di visualizzare gli stessi dati da una prospettiva diversa (es. scambiando righe e colonne).

1. Cos'è il Data Mining

Il **Data Mining** è un approccio all'analisi multidimensionale che serve a **estrarre informazioni nascoste** dai dati per supportare le decisioni aziendali.

Caratteristiche:

- Si applica ai dati del Data Warehouse
- Può essere utilizzato anche su dati "destrutturati" (es. collezioni di transazioni, testi, ecc.)
- Cerca relazioni e pattern che non sono evidenti a prima vista

2. KDD vs Data Mining: La Differenza

Spesso si confondono, ma c'è una differenza importante:

Concetto	Definizione
KDD (Knowledge Discovery in Database)	È l'**intero processo** di scoperta della conoscenza (dai dati grezzi alla conoscenza utile)
Data Mining	È **una fase** del processo KDD (la fase 5), dove si applicano algoritmi per trovare pattern

Analogia: Il KDD è come cucinare un piatto completo. Il Data Mining è come tagliare le verdure: è un passaggio importante, ma non è tutto il processo!

3. Definizione di KDD

Il **KDD** è un processo **non banale** per identificare nei dati:

- ✅ **Pattern validi** (sono reali, non casuali)
- ✅ **Precedentemente sconosciuti** (scoperte nuove)
- ✅ **Potenzialmente utili** (possono aiutare nelle decisioni)
- ✅ **Comprensibili** (si possono spiegare e interpretare)

Termini chiave:

- **Dato**: Un insieme di fatti (es. transazioni di vendita)
- **Pattern**: Un modello o insieme di regole che si applica a un sottoinsieme di dati (es. "I clienti che comprano pannolini spesso comprano anche birra")

4. Le 7 Fasi del Processo KDD

Il KDD è un processo **interattivo e iterativo** (si può tornare indietro se necessario). Ecco le 7 fasi:

FASE 1: Identificazione del Problema

- Si definisce **cosa si vuole scoprire**
- Si tiene conto della conoscenza già acquisita
- Si stabiliscono gli obiettivi del progetto

Esempio: "Vogliamo capire quali prodotti vengono acquistati insieme per migliorare il posizionamento negli scaffali."

FASE 2: Selezione dei Dati

- Si seleziona l'insieme dei dati su cui lavorare

Esempio: "Prendiamo tutti gli scontrini degli ultimi 2 anni dal database delle vendite."

FASE 3: Pre-processing (Pulizia dei Dati)

Si "puliscono" e normalizzano i dati:

- Eliminazione dei **dati rumorosi** (errori, duplicati)
- Gestione dei **valori estremi** (outliers)
- Compilazione o rimozione dei **campi vuoti**

- Correzione delle **inconsistenze**

Esempio:

- *Rimuovo scontrini con prezzi negativi (errore)*
- *Sostituisco i valori mancanti con la media*
- *Elimino acquisti anomali (es. 1000 pezzi dello stesso prodotto)*

FASE 4: Trasformazione (Feature Selection)

- Si individuano le **caratteristiche salienti** per rappresentare il fenomeno
- Si **riduce il numero di variabili** (si tiene solo ciò che serve) ottenendo un formato adatto al data mining

Esempio:

- *Invece di usare "data di nascita", uso "età"*
- *Invece di "codice prodotto", uso "categoria prodotto"*

FASE 5: Data Mining (Il Cuore del Processo)

È la fase in cui si **applicano algoritmi** per cercare i pattern di interesse.

Tecniche principali:

- **Classificazione**: Assegnare elementi a categorie (es. cliente fedele/occasionale)
- **Clustering**: Raggruppare elementi simili (es. segmentazione clienti)
- **Associazione**: Trovare relazioni tra elementi (es. market basket analysis)
- **Regressione**: Prevedere valori numerici (es. previsioni di vendita)

Esempio: Applico un algoritmo di associazione e scopro che:

"Il 70% dei clienti che compra pannolini compra anche birra nello stesso scontrino"

FASE 6: Interpretazione e Valutazione

- Si **interpretano** i pattern scoperti
- Si **valuta** se sono utili e validi
- Se i risultati non sono soddisfacenti, **si torna alle fasi precedenti** (processo iterativo)

Esempio:

- *Il pattern "pannolini + birra" è valido? Sì, è confermato dai dati*
- *È utile? Sì, posso posizionarli vicini*

- *È comprensibile? Sì, ha senso (padri che fanno spesa veloce)*

FASE 7: Consolidamento della Conoscenza

Si trasforma la scoperta in **azione concreta**:

- **Realizzazione/integrazione** di un sistema applicativo
- **Redazione di documentazione**
- **Presentazione** alle parti interessate (manager, stakeholder)
- **Implementazione** delle decisioni basate sulla scoperta

Esempio:

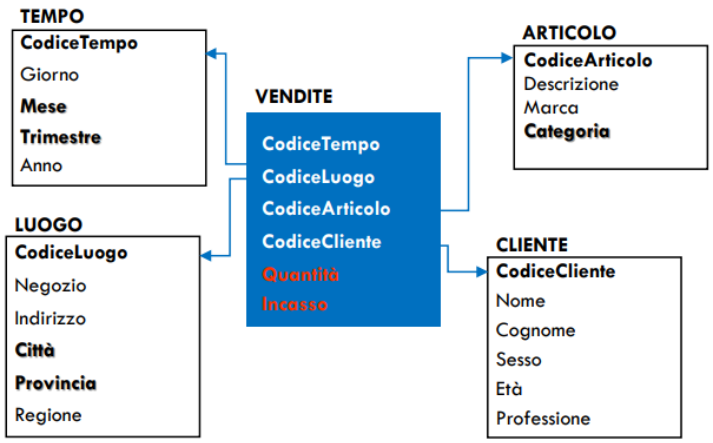
- *Modifico il layout del negozio: metto pannolini e birra vicini*
- *Aggiorno il sistema di raccomandazione online*
- *Creo un report per il management*
- *Formo il personale sulle nuove strategie*

MODELLI DI DWH SCHEMI RELAZIONALI A STELLA

SCHEMA A STELLA (Star Schema)

Immagina una **stella** con:

- **Al centro**: Una tabella principale chiamata **tabella FATTI** (in blu: VENDITE)
- **Intorno**: Le tabelle **DIMENSIONI** (TEMPO, ARTICOLO, LUOGO, CLIENTE)



Come funziona:

Tabella FATTI (VENDITE) - Il centro della stella:

- Contiene le **misure numeriche** da analizzare:
  - Quantità (quanti prodotti venduti)
  - Incasso (quanto abbiamo guadagnato)
- Contiene le **chiavi esterne** che collegano alle dimensioni:
  - CodiceTempo, CodiceLuogo, CodiceArticolo, CodiceCliente
- è in 3NF

Tabelle DIMENSIONI - I puntali della stella:

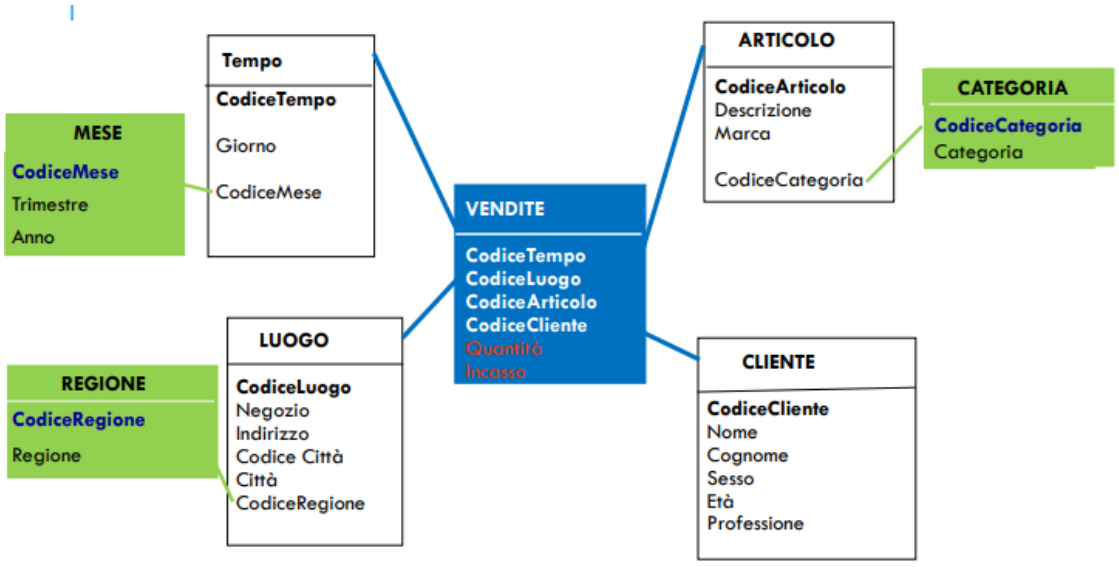
- Descrivono **il contesto** dei fatti:
    - **TEMPO**: Quando? (Giorno, Mese, Trimestre, Anno)
    - **LUOGO**: Dove? (Negozio, Città, Provincia, Regione)
    - **ARTICOLO**: Cosa? (Descrizione, Marca, Categoria)
    - **CLIENTE**: Chi? (Nome, Cognome, Età, Professione)
- Tipicamente denormalizzata; se normalizzata o parzialmente normalizzata si ha uno schema a fiocco di neve

SCHEMA A FIOCCO DI NEVE (Snowflake Schema)

È una **versione normalizzata** dello schema a stella. Le dimensioni si "ramificano" come un fiocco di neve!

Differenze principali:

Nello schema a fiocco di neve, le tabelle dimensioni sono **suddivise** in sotto-tabelle:



Vantaggi e Svantaggi:

Aspetto	Stella	Fiocco di Neve
Spazio	Occupa più spazio (dati ripetuti)	Occupa meno spazio (normalizzato)
Prestazioni	Più veloce (meno join)	Più lento (più join tra tabelle)
Semplicità	Facile da capire	Più complesso
Manutenzione	Più difficile aggiornare	Più facile aggiornare

GRANULARITÀ E ADDITIVITÀ DEI FATTI

La granularità indica il livello di aggregazione a cui un fatto è associato

- più i dati sono dettagliati, più bassa ne è la granularità
- meno dettagli si hanno nei dati e più alta è la granularità

L'additività indica la capacità di una misura di essere aggregata.

Un fatto è normalmente additivo cioè ha senso sommarlo rispetto ad ogni possibile combinazione delle dimensioni da cui dipende.

Tipo	Significato	Esempio
Additivo	Si può sommare su tutte le dimensioni	Incasso: ha senso calcolare la somma degli incassi per un certo intervallo di tempo, insieme di prodotti e insieme di negozi
Semi-additivo	Si può sommare solo su alcune dimensioni	Giacenza magazzino: posso sommare per prodotto, ma NON per tempo (non ha senso sommare le giacenze di giorni diversi)
Non additivo	Non si può sommare	Prezzo unitario: non ha senso sommare i prezzi

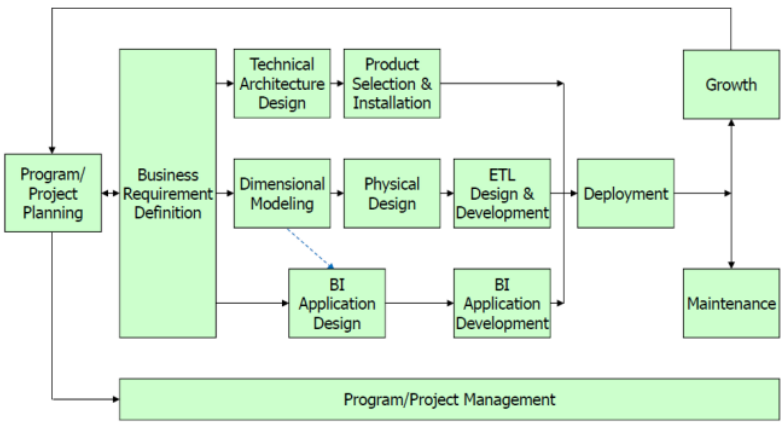
CICLO DI VITA DIMENSIONALE (Kimball Lifecycle)

È una **metodologia** per progettare e realizzare un Data Warehouse partendo dai **bisogni degli utenti**, non dalla tecnologia!

Filosofia:

Si lavora **"all'indietro"**:
Utenti ⟶ Report ⟶ Applicazioni ⟶ Database ⟶ Software ⟶ Infrastruttura

Le Fasi Principali:



Caratteristiche Fondamentali:

- **Business-driven**: Si parte dai bisogni degli utenti, non dalla tecnologia
- **Iterativo**: Il progetto si sviluppa in piccoli progetti successivi
- **Incrementale**: Si aggiunge funzionalità gradualmente
- **Flessibile**: Si può tornare indietro e modificare

Fase	Cosa si fa	Esempio concreto
1. Planning	Si pianifica il progetto	"Vogliamo analizzare le vendite per regione"
2. Requisiti	Si parla con i manager	"Il direttore vendite vuole report mensili"
3. Dimensional Modeling	Si progetta lo schema a stella	Creo tabelle: VENDITE (fatti), TEMPO, LUOGO, PRODOTTO
4. Physical Design	Si crea il database fisico	Creo le tabelle in Oracle/SQL Server
5. ETL	Si caricano i dati	Estraggo dati dai gestionali, li pulisco, li carico nel DWH
6. BI Applications	Creo i report	Dashboard con grafici delle vendite
7. Deployment	Rilascio agli utenti	Formo il personale, metto in produzione
8. Growth	Aggiungo funzionalità	"Ora vogliamo anche l'analisi per cliente" ⟶ si ricomincia!

Le Fasi della Progettazione del DWH

La progettazione di un Data Warehouse segue un percorso strutturato in **5 fasi principali**:

Fase	Attività	Output
1. Identificazione	Individuare fatti, misure e dimensioni	Lista di elementi chiave
2. Ristrutturazione concettuale	Rappresentare i fatti come entità, individuare nuove dimensioni, raffinare i livelli	Schema concettuale del DWH
3. Grafo dimensionale	Derivare le relazioni tra dimensioni e fatti	Grafo delle dipendenze
4. Progettazione logica	Derivare lo schema multidimensionale	Schema a stella/fiocco di neve
5. Progettazione fisica	Determinare lo schema relazionale finale	Tabelle fisiche con chiavi e indici

Data Lake

Cos'è un Data Lake?

Un **Data Lake** è un repository centralizzato e ampiamente scalabile che contiene **grandi quantità di dati grezzi**, archiviati "as is" (così come sono), senza bisogno di strutturarli prima.

Nel Data Lake lo schema non è imposto a priori sui dati, ma viene definito dinamicamente al momento dell'interrogazione in base alle esigenze dell'utente.

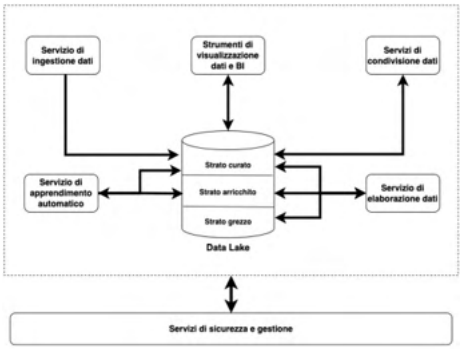
Differenza fondamentale con il Data Warehouse

Aspetto	Data Warehouse (DWH)	Data Lake
Dati	Strutturati e puliti	Grezzi, non strutturati
Approccio	**Schema-on-Write** (strutturo prima di archiviare)	**Schema-on-Read** (strutturo solo quando leggo)
Processo	**ETL** (Extract → Transform → Load)	**ELT** (Extract → Load → Transform)
Flessibilità	Rigida (schema fisso)	Massima (schema definito dall'utente)
Utenti	Analisti business	Data scientist, analisti avanzati
Scopo	Reporting e analisi strutturate	Esplorazione, machine learning, analisi avanzate

Come funziona il paradigma Schema-on-Read

1. I dati vengono caricati **senza uno schema predefinito**
2. Lo schema viene applicato **solo al momento della lettura**
3. Ogni utente può definire lo schema in base alle proprie esigenze

Le caratteristiche che definiscono un'architettura di storage Data Lake:



- Un catalogo di metadati che stabilisce la qualità dei dati
- Policies di data governance e tools di gestione
- Gestione ed accesso condiviso, quindi multi-utente
- Dati provenienti di fonti di vario tipo
- Organizzazione logica e fisica ad hoc
- Scalabilità in termini di archiviazione ed elaborazione dati

Big Data

Perché i Big Data?

Viviamo in un mondo iper-connesso:

- **3+ miliardi** di dispositivi connessi
- App mobili che tracciano ogni attività quotidiana
- Sensori che generano dati 24/7 (IoT, satelliti, GPS)
- **Previsione 2025:** 175 zettabyte di dati generati (1 ZB = 1 miliardo di TB!)

Definizione di Big Data

I **Big Data** sono collezioni di dati:

- **Strutturati** (database relazionali)
- **Semi-strutturati** (JSON, XML, log)
- **Non strutturati** (email, video, social media, sensori)

Per cui i **metodi tradizionali di gestione e analisi non sono più adeguati**.

Le 5 V dei Big Data

Le 5 dimensioni che caratterizzano i Big Data:

V	Nome	Significato	Esempio
1	**Volume**	Dimensione enorme del dataset	Terabyte, petabyte, exabyte
2	**Velocità**	Alta velocità di generazione e necessità di analisi rapida	Streaming dati in tempo reale
3	**Varietà**	Diversi tipi di dati da fonti eterogenee	Testo, immagini, video, sensori, log
4	**Veridicità**	Qualità e affidabilità dei dati	Dati rumorosi, incompleti, inconsistenti
5	**Valore**	Incremento di business derivante dall'analisi	Insight utili per decisioni strategiche

Sorgenti dei Big Data

Le sorgenti più note sono le seguenti:

- **Enterprise Data**: Insieme dei dati condivisi all'interno di un'organizzazione (es. email);
- **Public Data**: Dati aperti al pubblico senza alcuna misura di protezione;
- **Sensor Data**: Insieme dei dati catturati dai sensori (es. satelliti, GPS);
- **Social Media**: Informazioni condivise on-line sotto forma di immagini, testo, audio e video;
- **Transactions**: Insieme di tutte le informazioni derivanti da transazioni.

L'idea alla base dei **Big Data** va ritrovata dunque nella necessità di **analizzare**, contemporaneamente, collezioni **molto grandi** di dati **correlati** per ricavare **informazioni aggiuntive** rispetto all'analisi separata di piccoli insiemi di dati.

Database NoSQL

Cos'è NoSQL?

NoSQL (*Not Only SQL*) è una famiglia di tecnologie per la gestione dei dati che **non segue il modello relazionale tradizionale**.

Perché NoSQL?

I database relazionali hanno limiti con i Big Data: Difficoltà con dati non strutturati, Scalabilità verticale costosa, Schema rigido, JOIN complessi su grandi volumi

Differenza: Relazionale vs NoSQL

Aspetto	Database Relazionale (RDBMS)	Database NoSQL
Modello	Tabelle con righe e colonne	Vari modelli (documenti, grafi, key-value)
Schema	Rigido e predefinito	Flessibile e dinamico
Scalabilità	Verticale (server più potente)	**Orizzontale** (più server)
Proprietà ACID	Tutte garantite	Solo alcune (BASE)
JOIN	Supportati nativamente	Limitati o assenti
Dati	Strutturati	Strutturati, semi-strutturati, non strutturati
Prestazioni	Buone su dati strutturati	Ottime su grandi volumi

Proprietà dei sistemi NoSQL

- **Scalabilità orizzontale** per operazioni semplici
- **Replicazione e distribuzione** su più server
- **Interfaccia semplificata** (API REST, protocolli leggeri)
- **Modello di concorrenza più debole** (non tutte le proprietà ACID)
- **Uso efficiente** di memoria e indici distribuiti
- **Aggiunta dinamica** di nuovi attributi

Classificazione dei Database NoSQL

A. Key-Value Store

Modello: Coppie chiave-valore (come una grande hash table)

Caratteristiche:

- Struttura dati semplicissima
- Ogni chiave è associata a un valore (stringa, lista, oggetto)
- Accesso velocissimo tramite chiave primaria

Casi d'uso: Cache applicativa, Sessioni web, Carrelli della spesa

Esempi: Amazon DynamoDB, Redis, Voldemort (LinkedIn)

B. Column-Oriented Database

Modello: Dati organizzati per colonne invece che per righe

Caratteristiche:

- Una **row key** identifica un oggetto
- Ogni riga mappa a una tupla di colonne
- Struttura multi-livello per ricerche avanzate
- Ottimizzato per analisi su grandi volumi

Casi d'uso: Analytics su grandi dataset, Time series, Data warehousing

Esempi: Apache Cassandra, HBase, Google Bigtable

ROW KEY	Colonna1	Colonna2	Colonna3
user:1	"Mario"	"Roma"	25
user:2	"Luigi"	"Milano"	30
user:3	"Anna"	"Napoli"	28

C. Document-Oriented Database

Modello: Dati gestiti come documenti (JSON, BSON, XML)

Caratteristiche:

- Strutture complesse (liste, mappe, insiemi)
- Riferimenti tra documenti (simile a database a oggetti)
- Indicizzazione su singoli attributi o indici composti
- Formato di scambio: JSON/BSON

Casi d'uso:

- CMS (Content Management System)
- Cataloghi prodotti
- Profili utente

Esempi: MongoDB, CouchDB, Amazon DocumentDB

D. Graph-Oriented Database

Modello: Nodi (oggetti) e archi (relazioni tra oggetti)

I nodi sono oggetti e gli archi definiscono particolari relazioni tra gli oggetti stessi.

Nodi e relazioni hanno proprietà per la memorizzazione dei dati.

La forza di questo tipo di database è di gestire dati fortemente **interconnessi**, proprietà che permette un operazione di attraversamento del grafo (**graph trasversal**), rispetto ad una normale query su database chiave-valore, permette il passaggio da un nodo all'altro attraverso le relazioni tra i nodi stessi.

Casi d'uso: Social network (amicizie, follower), Recommendation engine, Knowledge graph, Network analysis

Esempi: Neo4j, Titan, Amazon Neptune

E. Multi-Model Database (MMDBMS)

Modello: Supporta **più modelli di dati** su un unico back-end

Caratteristiche:

- Combina: grafo, relazionale, key-value, documenti
- Base matematica: **teoria delle categorie** (categoria e funtore)
- Trasformazione dichiarativa tra modelli

- Flessibilità massima

Casi d'uso: Applicazioni complesse con requisiti diversi, Migrazione da modelli legacy, Sistemi che evolvono nel tempo

Esempi: ArangoDB, OrientDB, Cosmos DB (Azure)

TECNOLOGIA E ARCHITETTURA DEI DBMS

1. Cos'è un DBMS?

Un **DBMS (Database Management System)** è un sistema complesso che deve svolgere diverse funzioni fondamentali per gestire efficacemente i dati.

Funzioni principali di un DBMS:

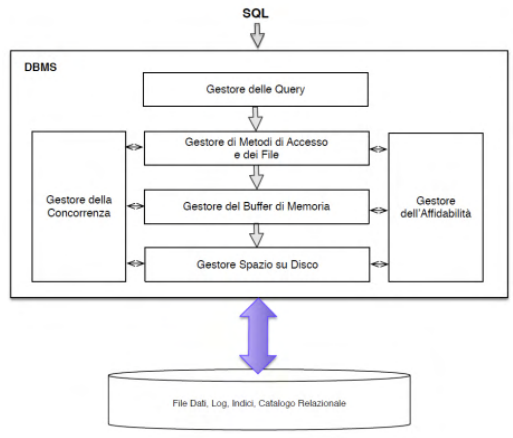
Funzione	Descrizione
Memorizzazione dati	Organizzare i file e gli indici su disco
Gestione buffer	Gestire la memoria centrale per i dati frequentemente usati
Ottimizzazione query	Trovare il piano di esecuzione più efficiente
Gestione transazioni	Garantire le proprietà ACID (Atomicità, Consistenza, Isolamento, Durabilità)
Controllo concorrenza	Gestire accessi simultanei di più utenti senza conflitti
Controllo affidabilità	Garantire il recupero dei dati in caso di guasti
Gestione integrità	Assicurare il rispetto dei vincoli di integrità
Controllo accessi	Garantire sicurezza e autorizzazioni

Aspetti avanzati:

- **DDBMS (Distributed DBMS)**: Gestione delle transazioni in basi di dati distribuite su più nodi
- **Replicazione**: Copie multiple dei dati per migliorare affidabilità ed efficienza

2. Architettura di un DBMS

L'architettura di un DBMS è modulare e composta da diversi gestori specializzati.



3. I Moduli dell'Architettura

A. Gestore delle Query

Analizza e ottimizza le query, trasforma l'istruzione SQL nell'azione fisica più veloce possibile. Lavora in 3 fasi:

1. **Parsing (Analisi Sintattica):** Controlla che non ci siano errori grammaticali nell'SQL e che le tabelle richieste esistano davvero.
2. **Ottimizzazione:** Valuta tutte le "strade" possibili per recuperare i dati e sceglie la più veloce (il **Piano di Esecuzione**).
3. **Generazione del Piano:** Traduce l'SQL (linguaggio ad alto livello) in una sequenza di **operazioni fisiche di basso livello**.

Quali sono queste Operazioni di Basso Livello?

- **Scansioni Sequenziali:** Leggere la tabella intera, riga per riga (metodo lento).
- **Accessi Diretti:** Usare gli *Indici* per "saltare" esattamente al dato cercato (metodo veloce).
- **Operazioni Relazionali:** Eseguire Join, applicare filtri e ritagliare colonne.
- **Ordinamenti:** Riordinare i risultati prima di mostrarli.

Il Passaggio Logico-Fisico: Approvato il piano, il Gestore delle Query passa il testimone ai moduli inferiori. Da questo momento, il database non vede più "Tabelle e Colonne", ma inizia a maneggiare l'hardware puro: **file, pagine di memoria e blocchi di record**.

B. Gestore dei Metodi di Accesso e dei File

Gestisce l'accesso fisico ai dati. Collega le operazioni logiche alla memoria fisica. Necessita di dialogare costantemente con la memoria centrale per sapere dove sono posizionati i dati richiesti prima di andarli a cercare.

C. Gestore del Buffer di Memoria (La RAM)

Gestisce la porzione di memoria veloce (RAM) destinata a ospitare temporaneamente i dati.

- **Compito:** Decide strategicamente *come* e *quando* trasferire le pagine di record dalla memoria di massa (disco lento) alla memoria centrale (RAM veloce), e viceversa.

D. Gestore dello Spazio su Disco

È il livello più basso a contatto con l'hardware.

- **Compito:** Esegue materialmente le funzioni di **lettura, scrittura, allocazione e rilascio** delle singole pagine fisiche sull'hard disk.

E. Gestore della Concorrenza (L'Arbitro)

Filtra e organizza le richieste simultanee di più utenti, garantendo l'esecuzione corretta delle transazioni (proprietà ACID).

- **Meccanismo:** Utilizza i **Lock** (protocolli di blocco) per "congelare" temporaneamente le risorse ed evitare che due utenti modifichino lo stesso dato contemporaneamente creando conflitti.

F. Gestore dell'Affidabilità (Il Salvavita)

Interviene per ripristinare il database in caso di guasti hardware o crash del sistema.

- **Meccanismo:** Scrive un **Log** (un registro continuo e cronologico) di tutto ciò che accade. Se c'è un guasto, legge il log e ripristina il database all'ultimo stato sicuro.

G. Gestore dell'Integrità

Il controllore delle regole. Si assicura che, a valle di qualsiasi operazione di modifica, inserimento o cancellazione, tutti i **vincoli di integrità** (es. chiavi primarie, esterne, check) rimangano soddisfatti.

H. Gestore degli Accessi (La Sicurezza)

Il buttafuori del database.

- **Compito:** Garantisce che solo gli utenti e le applicazioni esplicitamente **autorizzati** possano entrare. Verifica inoltre che le azioni tentate siano compatibili con i **privilegi** assegnati a quell'utente.

La Memorizzazione Fisica dei Dati

Per gestire enormi moli di dati in modo permanente (cioè senza perderli quando si spegne il computer), il DBMS deve per forza salvarli su una memoria di massa (es. l'Hard Disk o l'SSD). Tuttavia, per poter eseguire calcoli o ricerche, i dati devono essere temporaneamente spostati nella memoria centrale (la RAM), che è infinitamente più veloce.

Il continuo "palleggio" di dati tra il disco lento e la RAM veloce è il cuore della gestione fisica di un database.

1. File, Record e Pagine

Come vengono organizzati i dati sull'hard disk?

- **Il File (Livello Logico):** Fisicamente, una tabella del database (come quella degli *Impiegati*) viene salvata come un **File di Record**. Immaginalo come un lungo elenco in cui ogni singola riga della tabella logica (es. la tupla di "Antonio De Aurelis") diventa un **Record Logico**.
- **La Pagina (Livello Fisico):** Ecco il concetto chiave. L'hard disk **non legge mai un singolo record alla volta**. Sarebbe uno spreco di tempo enorme. L'hard disk legge e scrive i dati a "blocchi" di dimensioni fisse, chiamati **Pagine** (o *Record Fisici*). Una pagina è tipicamente grande 4 KB o 8 KB.
- **Il Costo di I/O:** Ogni singola volta che il disco deve recuperare o scrivere una Pagina, il sistema paga un "Costo di I/O" (Input/Output). Questo costo è **l'operazione in assoluto più pesante e lenta** di tutto il database. L'obiettivo primario di un DBMS è ridurre al minimo il numero di pagine lette dal disco.

2. Il Fattore di Blocco (Fattore di Blocking)

Il Fattore di Blocco è un concetto matematico facilissimo ma essenziale per l'esame. Stabilisce **quanti Record Logici (righe) riescono a entrare in una singola Pagina (Record Fisico)**.

ID	*Nome*	*Cognome*	*Stipendio*
003	Antonio	De Aurelis	5000
001	Marco	Marchi	2000
005	Antonio	Di Padova	3000
004	Francesco	Di Sales	1000

- **Come funziona:** Quando un utente cerca la tupla "Antonio De Aurelis", il DBMS individua in quale Pagina fisica si trova, e carica *l'intera Pagina* in RAM. Se nella pagina, oltre ad Antonio, ci entrano anche le tuple di "Marco" e "Francesco", il database avrà caricato in RAM tre impiegati al costo di una sola lettura dal disco!
- **La Definizione:** Il fattore di blocco è il numero intero di record logici contenuti all'interno di un record fisico (pagina).
- **Il Calcolo:** Si calcola dividendo la dimensione della Pagina per la dimensione del singolo Record (arrotondando per difetto).

- *Esempio:* Se una Pagina è grande 4000 byte e ogni impiegato occupa 1000 byte, il Fattore di Blocco è **4**. Significa che con 1 solo accesso al disco (1 I/O) porto in memoria 4 impiegati.

$$Fattore\ di\ Blocco = \frac{\text{Dimensione Pagina}}{\text{Dimensione Record}}$$

Perché è importante? Maggiore è il fattore di blocco, più il database è veloce ed efficiente, perché riuscirà a leggere moltissime righe facendo pochissimi "viaggi" verso il disco lento.

Metodi di Organizzazione dei File

Quando crei una tabella, il DBMS deve decidere come scrivere i suoi record fisicamente sul disco. Ha tre opzioni:

A. File Heap (Il "Mucchio")

È il metodo più grezzo. I record vengono "buttati" nel file in modo completamente casuale, o meglio, in ordine cronologico di arrivo.

- **Inserimento:** Molto veloce. Il nuovo record viene semplicemente messo nell'ultima pagina disponibile in fondo al file.
- **Ricerca:** Disastrosa. L'unico modo per trovare qualcosa è la *Ricerca Lineare* (scansionare l'intero file dall'inizio alla fine).
- **Cancellazione:** Se elimini un record, il DBMS non sposta gli altri per chiudere il buco, ma marca solo lo spazio come "liberato". Col tempo il file si deteriora (diventa frammentato) e necessita di riorganizzazione.

B. File Ordinati (File Sequenziali)

I record sono salvati fisicamente ordinati in base a un campo (es. l'ID).

- **Ricerca:** Molto veloce. Essendo ordinato, il DBMS può usare la *Ricerca Binaria* (dimezzando l'area di ricerca ad ogni passaggio).
- **Inserimento/Cancellazione:** Molto costosi. Per mantenere l'ordine fisico sull'hard disk, se inserisci un nuovo impiegato con ID "002" tra "001" e "003", il DBMS dovrà fare spazio spostando fisicamente le pagine di tutti i record successivi!

C. File Hash (Accesso Calcolato)

È un metodo matematico. Non c'è ordine, ma una logica rigorosa.

- La posizione esatta in cui verrà salvato un record viene calcolata tramite una **Funzione di Hash**. Prendi l'ID, lo passi alla funzione, e lei ti restituisce un numero: l'indirizzo esatto (il *bucket*) in cui scrivere.

- **Vantaggio:** Velocità estrema. Se cerchi l'ID 004, non devi scansionare il file. Passi 004 alla funzione di Hash, ottieni l'indirizzo e vai dritto a colpo sicuro.
- **Il Problema (Le Collisioni):** Cosa succede se la formula matematica, per pura sfortuna, dà lo stesso indirizzo a due record diversi? Questa è una **Collisione**. Il DBMS la risolve creando "un'area di overflow": lascia il primo record nel bucket, e collega il secondo record sfortunato creando una lista concatenata in uno spazio di riserva.

Gli Indici

Visti i difetti dei tre metodi precedenti, i database usano un'arma segreta: gli **Indici**.

- **Cos'è un Indice?** Immagina l'indice analitico in fondo a un grosso libro di storia. Invece di sfogliare 1000 pagine (File Heap) per trovare la parola "Napoleone", vai all'indice, cerchi la "N", e trovi scritto: "Napoleone: Pagina 450". Vai dritto a pagina 450.
- **Nel Database:** Un indice è un piccolo file separato (molto leggero, spesso tenuto direttamente nella veloce RAM) composto da righe chiamate **Data Entry**. Ogni riga contiene solo due cose: la *chiave di ricerca* (es. l'ID "004") e il *RID* (Record Identifier, l'indirizzo fisico in cui si trova tutto il resto del record nel file grosso sul disco).

Il Data Entry

L'indice è composto da una lista di righe. Ognuna di queste righe prende il nome di **Data Entry**. A seconda di come viene progettato il database, un Data Entry può essere strutturato in tre modi diversi:

1. **Il record intero:** Il Data Entry contiene *sia* la chiave di ricerca *sia* tutti gli altri dati. In questo caso, l'indice non è un file separato, ma è la struttura stessa con cui è organizzato il file principale.
2. **Coppia (Chiave, RID):** È il metodo più classico. Il Data Entry contiene solo la chiave di ricerca (es. ID "004") e il **RID** (*Record Identifier*). Il RID è l'indirizzo fisico "nudo e crudo" che dice al sistema in quale blocco del disco si trova il resto del record.
3. **Coppia (Chiave, Lista di RID):** Ottimo per risparmiare spazio se ci sono valori duplicati. La chiave compare una volta sola, seguita da una lista di indirizzi fisici (es. *"Tutti gli impiegati che prendono 2000€ si trovano agli indirizzi X, Y e Z"*).

2. Classificazione degli Indici (Le Tipologie)

In base a come vengono costruiti rispetto al file dei dati, gli indici si dividono in tre famiglie:

- **Indice Primario:** Si crea quando il file dei dati principale è già *fisicamente ordinato* in modo sequenziale, e l'indice viene costruito proprio sulla chiave primaria di quell'ordinamento.
- **Indice Secondario:** Viene costruito su un campo che *non* è la chiave primaria e su cui il file principale *non* è ordinato fisicamente (es. creare un indice sul "Cognome" in una tabella ordinata per "ID").
- **Indice di Clustering:** Viene costruito su un campo non univoco (che ammette duplicati). Serve a raggruppare (creare un *cluster*) tutti i record che hanno lo stesso valore per quel campo.

Densità

Quanti indici posso mettere su una singola tabella?

- Puoi avere **un solo** Indice Primario *oppure* **un solo** Indice di Clustering (perché il file fisico su disco può essere ordinato in un solo modo per volta!).
- Puoi avere **molteplici** Indici Secondari (perché sono file separati e scollegati dall'ordine fisico).

Infine, come si comportano gli indici rispetto al numero di record?

- **Indice Denso:** Esistono n Data Entry nell'indice per *ogni singolo valore* della chiave presente nel database. Se hai 10.000 impiegati, l'indice avrà 10.000 righe.
- **Indice Sparso:** L'indice contiene un Data Entry solo per *alcuni* valori (di solito, solo per il primo record di ogni Pagina fisica del disco). È molto più piccolo e veloce da leggere, ma funziona solo se il file principale è rigidamente ordinato.

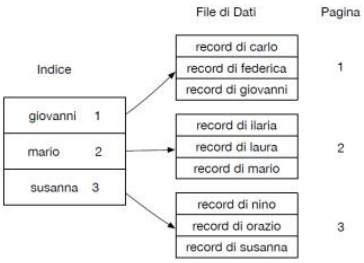
Il File Sequenziale Indicizzato

Il file sequenziale indicizzato è la forma più classica di archiviazione: i dati fisici (i record) sono tenuti ordinati in base a una chiave (es. in ordine alfabetico per cognome), e per velocizzare la ricerca viene affiancato un **Indice Primario**.

- **L'Architettura a 3 aree:** Un file sequenziale indicizzato classico è diviso in tre blocchi fisici:
    1. **L'Indice:** Mantenuto in memoria veloce, dice dove cercare.
    2. **L'Area Primaria:** Il "file grosso" dove sono memorizzati i dati veri e propri.

3. **L'Area di Overflow:** Una zona di emergenza. Se devi inserire un nuovo record, ma la pagina in cui dovrebbe stare nell'Area Primaria è piena, il record viene temporaneamente parcheggiato nell'Area di Overflow per non bloccare il sistema.

I limiti dell'Indice Sparso Semplice



- Abbiamo un Indice molto piccolo che tiene traccia solo del primo nome di ogni Pagina (es. *giovanni* per la Pagina 1, *mario* per la Pagina 2).
- Essendo molto piccolo, l'indice viene caricato tutto nella **Memoria Principale (RAM)**. E siccome è ordinato, il DBMS trova subito la riga giusta usando la velocissima *Ricerca Binaria*.
- **Il problema delle modifiche (Aggiornamenti Dinamici):** Questo sistema è perfetto se il database è "statico" (in sola lettura). Ma se ci sono continue operazioni di inserimento e cancellazione (es. un e-commerce), mantenere l'ordine fisico nel file dei dati e contemporaneamente tenere aggiornato l'indice diventa computazionalmente **complesso e lentissimo**. Bisogna spostare record, aggiornare i puntatori delle pagine e gestire l'overflow. Serve una struttura più dinamica.

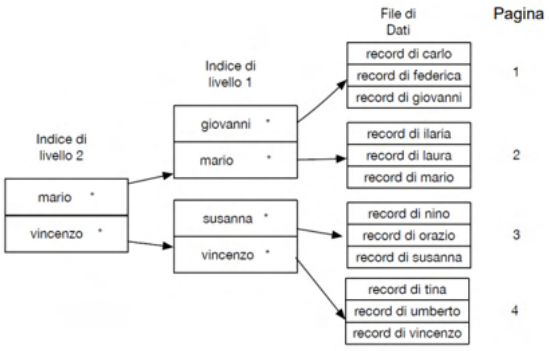
L'Indice Multilivello

Cosa succede se il database diventa così gigantesco che l'Indice Principale stesso diventa troppo grande per entrare in RAM o per essere scansionato velocemente?

La soluzione è **creare un Indice dell'Indice**.

- **Il concetto:** Se un file Dati occupa K pagine sul disco, possiamo creare un Indice (Livello 1) molto più piccolo. Se anche il Livello 1 diventa troppo grande, creiamo un Indice di Livello 2 che punta al Livello 1, e così via. È un sistema "a matrioska".
- **Come funziona la navigazione**

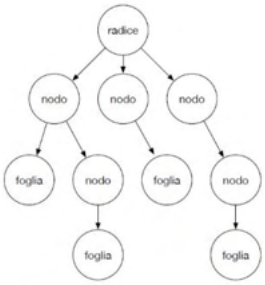
Supponiamo di cercare il nome **'federica'**.



1. Il sistema interroga il livello più alto (Indice di Livello 2). Trova due macro-puntatori: '*mario*' e '*vincenzo*'.
2. Il sistema sa che 'federica' viene alfabeticamente *prima* di 'mario', quindi segue il puntatore di '*mario*'.
3. Il puntatore porta al blocco dell'Indice di Livello 1 che contiene '*giovanni*' e '*mario*'.

4. 'federica' viene prima di 'giovanni', quindi segue il puntatore di '*giovanni*'.
5. Il puntatore finale porta dritto alla **Pagina 1** del File di Dati, dove effettivamente si trova il record di 'federica'.

Gli Indici ad Albero



La struttura logica appena descritta nell'Indice Multilivello è la perfetta applicazione pratica di una famosissima struttura dati informatica: **L'Albero** (nello specifico, gli alberi B e B+).

- **Anatomia dell'Albero:**
    - **Radice (Root):** È il punto di partenza (l'Indice di livello più alto). L'unico nodo a non avere un "padre" sopra di sé.
    - **Nodi Interni (Archi):** Sono i livelli intermedi dell'indice. Ricevono il percorso dal padre e indirizzano la ricerca verso i figli corretti in base a delle condizioni logiche (es. maggiore/minore di).
    - **Nodi Foglia (Leaves):** Sono il punto di arrivo. Non hanno "figli" sotto di sé. Nel contesto dei database, **le foglie rappresentano i puntatori finali alle Pagine Fisiche** del disco dove risiedono i dati.

1. Il B-Tree (Albero Bilanciato)

Abbiamo visto che l'Indice Multilivello ad albero è fantastico per trovare i dati velocemente. Tuttavia, c'è un rischio: se inserisci tanti nomi che iniziano con la "A" e pochissimi con la "Z", l'albero rischia di crescere "storto", diventando lunghissimo da una parte e corto dall'altra.

Per evitare questo, i database usano un **Albero Bilanciato (B-Tree)**.

- **La Regola d'Oro del Bilanciamento:** Tutte le "foglie" (i nodi finali in basso) devono trovarsi *esattamente allo stesso livello* di profondità.
- **Cosa garantisce?** Che per cercare qualsiasi record (che sia "Andrea" o "Zorro"), il database farà sempre *lo stesso numero esatto di passaggi*. Le prestazioni sono costanti e prevedibili.

La Definizione Formale: Anatomia di un B-Tree di ordine p

Per definire un B-Tree, si parte dal concetto di **Ordine (indicato con p)**. L'ordine definisce la capacità massima di ramificazione della struttura, ovvero **il numero massimo di puntatori ai figli** (sottoalberi) che un singolo nodo può ospitare. Di conseguenza, un nodo conterrà al massimo $p - 1$ chiavi di ricerca.

Struttura Interna del Nodo

Ogni nodo del B-Tree è costituito da una sequenza alternata di puntatori e chiavi, che possiamo formalizzare come:

$$(P_1, K_1, PK_1, P_2, K_2, PK_2, \dots, P_q)$$

(dove $q \leq p$ rappresenta il numero effettivo di puntatori correntemente nel nodo).

Analizziamo i tre elementi fondamentali della sequenza:

- **Le Chiavi di Ricerca (K_i):** Sono i valori su cui è costruito l'indice (i valori che stai cercando es. "giovanni", "mario"). Fondamentale è il vincolo di ordinamento: all'interno del nodo, le chiavi sono rigorosamente ordinate in senso crescente ($K_1 < K_2 < \cdots < K_{q-1}$).
- **I Puntatori ai Dati (PK_i):** Questa è una caratteristica specifica del B-Tree *classico* (non presente nel B+ Tree). Ogni chiave K_i è affiancata da un puntatore PK_i che punta direttamente all'indirizzo del record fisico sul disco. Se, durante la discesa, il DBMS trova la chiave cercata in un nodo intermedio, utilizza subito PK_i per recuperare il dato, interrompendo l'attraversamento dell'albero.
- **I Puntatori all'Albero (P_i):** Definiscono la topologia dell'albero e implementano la logica di partizionamento dello spazio di ricerca.
    - Un puntatore generico P_i indirizza verso un sottoalbero che contiene esclusivamente chiavi **minori o uguali** alla rispettiva chiave K_i.
    - L'ultimo puntatore del nodo (P_q) indirizza verso un sottoalbero che contiene chiavi strettamente **maggiori** dell'ultima chiave presente nel nodo (K_{q-1}).
- Il puntatore *prima* di "mario" ti porta a tutti i nomi minori di "mario".
- Il puntatore *dopo* "mario" ti porta a tutti i nomi maggiori di "mario".

I Vincoli Strutturali (Regole di Bilanciamento)

Per garantire che l'albero offra prestazioni di ricerca logaritmiche ottimali, la definizione impone vincoli ferrei sulla gestione dello spazio e dell'altezza:

- **Vincolo di Riempimento Minimo:** Per evitare che l'albero degeneri in una struttura lineare e per non sprecare memoria, i nodi non possono essere quasi vuoti. Ogni nodo interno (ad eccezione della radice) deve contenere un **minimo di** $p/2$

puntatori. La radice, se l'albero non è vuoto o composto da un solo nodo, deve avere almeno $_2$ figli.

- **Bilanciamento Perfetto:** L'albero è perfettamente bilanciato dal basso. Tutte le operazioni di inserimento o cancellazione (che provocano *split* o *merge* dei nodi) devono garantire che **tutti i nodi foglia risiedano esattamente allo stesso livello di profondità**. Questo assicura un costo di I/O costante e prevedibile nel caso peggiore per qualsiasi ricerca.

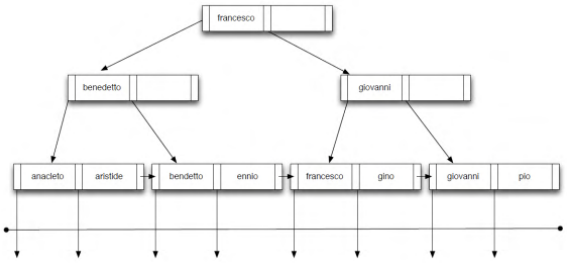
Come si adatta l'albero? (Split e Merge)

L'albero deve rimanere bilanciato anche se inserisci o cancelli 10.000 righe.

- **In caso di Inserimento (Lo Split):** I nodi hanno una capienza massima (es. massimo 3 nomi). Se un nodo è già pieno e tu provi a inserirne un quarto, il nodo "scoppia" e si divide in due (**Split**). Il valore centrale sale al piano di sopra (nel nodo padre) per fare da divisorio. Se anche il padre è pieno, scoppia anche lui. L'albero cresce verso l'alto.
- **In caso di Cancellazione (Il Merge):** I nodi hanno anche una capienza minima (devono essere pieni almeno a metà, $_{p/2}$). Se cancelli troppi dati e un nodo si svuota troppo, il DBMS prende i pochi superstiti e li unisce al nodo vicino (**Merge**). Se i nodi si svuotano troppo, l'albero si "rimpicciolisce".

Il B+ Tree

Il B+ Tree è una variante ottimizzata dell'albero bilanciato, progettata per massimizzare l'efficienza delle ricerche su disco e ridurre l'impatto dei vincoli strutturali.



1. Le Due Differenze Strutturali (Fondamentali)

Rispetto al B-Tree classico, il B+ Tree introduce due modifiche architetturali cruciali:

- **Separazione Logica-Fisica (I dati solo nelle foglie):** Nel B-Tree classico, i puntatori ai dati reali potevano trovarsi anche nei nodi intermedi. Nel B+ Tree, i nodi interni fungono *esclusivamente da "cartelli stradali"* direzionali (contengono solo le chiavi per instradare la ricerca). **I puntatori ai dati reali sul disco sono memorizzati unicamente nei nodi foglia.**
- **La Lista Concatenata:** Tutti i nodi foglia (essendo posizionati allo stesso livello di profondità) sono collegati tra loro orizzontalmente tramite puntatori, formando una **lista concatenata sequenziale**.

I 3 Vantaggi Principali del B+ Tree

1. **Range Query Istantanee:** Nelle ricerche per intervallo (es. stipendi da 1500€ a 2500€) l'albero viene disceso *una sola volta*. Dopodiché, il sistema sfrutta la lista concatenata per scorrere orizzontalmente le foglie, senza dover risalire e ridiscendere i rami.
2. **File Dati "Libero" (File Heap):** L'indice B+ Tree è un indice denso in cui le foglie sono già perfettamente ordinate. Questo permette al database di salvare i record reali sull'hard disk in modo completamente disordinato e veloce (File Heap), senza doversi preoccupare dell'ordine fisico.
3. **Tempi di Ricerca Costanti:** Poiché i puntatori ai dati si trovano *esclusivamente* nelle foglie, ogni singola ricerca richiede esattamente lo stesso numero di passaggi (pari all'altezza dell'albero). Le prestazioni sono fisse e prevedibili.

Il Limite Fisiologico e la Soluzione (B* Tree)

- **Il Limite:** Le operazioni di inserimento e cancellazione restano "pesanti", perché il database deve continuamente dividere (Split) o fondere (Merge) i nodi per mantenere l'albero bilanciato.
- *La Variante B*Tree:* Per risolvere il problema, l'industria ha creato il B* Tree. Questa variante alza la soglia minima di riempimento dei nodi: invece di essere pieni almeno a metà ($1/2$), **devono essere pieni almeno per due terzi ($2/3$)**. Questo stratagemma ritarda il momento in cui i nodi collassano o esplodono, riducendo le operazioni di manutenzione e ottimizzando lo spazio su disco.

Indici Speciali per Data Warehouse (DWH)

I Data Warehouse (DWH) sono enormi archivi storici usati dalle aziende per fare analisi sui dati (es. "Quante scarpe rosse abbiamo venduto in Lombardia nel 2023?"). Essendo database pensati per l'analisi e non per le transazioni quotidiane, usano indici particolari:

- **Indici di Bitmap:** Vengono usati per colonne che hanno una **bassa cardinalità**, ovvero pochissimi valori possibili (es. il sesso "M/F", lo stato civile, o i giorni della settimana).
    - *Come funziona:* Invece di salvare l'indirizzo di ogni riga, il database crea una stringa lunghissima di bit (0 e 1). Se la riga contiene quel valore, il bit è 1, altrimenti 0. Le query analitiche che incrociano questi dati diventano fulminee, perché il processore riesce a sommare e confrontare stringhe di bit a una velocità disarmante.
- **Indici di Join:** L'operazione di JOIN (unire due tabelle) è la più faticosa in assoluto per un database. Visto che nei DWH le stesse query analitiche vengono ripetute spesso, l'Indice di Join **pre-calcola l'operazione**. Esegue l'unione in anticipo e ne

salva i collegamenti nell'indice. Quando l'utente lancia la query, il database ha il risultato già pronto.

Buffer Manager (Il Gestore della Memoria)

Buffer: È un'area di **memoria centrale (RAM)** condivisa tra tutte le transazioni. È organizzata in **pagine** della stessa dimensione dei blocchi su disco (es. 1KB - 100KB).

Il **Buffer Manager** è il gestore dei buffer che decide quali pagine fisiche del disco possono andare nel buffer (e quindi in RAM) , quanto ci possono restare, e quando devono essere rispedite sul disco.

Per fare il suo lavoro, per ogni pagina caricata in RAM, il gestore usa due "etichette" (variabili di stato):

1. **Count (Contatore):** Indica quante transazioni stanno usando quella pagina *in questo esatto momento*. Se vale 0, significa che la pagina è libera e nessuno la sta usando.
2. **Dirty (Il bit di "sporco"):** Ti indica se la pagina è stata **modificata** in RAM ma non ancora salvata su disco (1 = sì, 0 = no).

Il Ciclo di Vita di una Richiesta

Quando un'applicazione chiede un dato, il Buffer Manager segue questo rigoroso algoritmo:

1. Controlla se la Pagina è già nel Buffer (RAM). [**BUFFER HIT**]
    - *Se sì:* Aumenta il count di 1 (+1 utente). Se l'utente la modifica, accende il bit dirty a 1.
    - *Se no:* Deve andarla a prendere dal disco. [**BUFFER MISS**]
2. Per farla salire dal disco, serve un posto libero nel Buffer. Se il buffer è pieno, usa una politica di rimpiazzo per decidere chi cacciare:
    - **FIFO (First In, First Out):** Caccia la pagina che è entrata per prima (la più vecchia).
    - **LRU (Least Recently Used):** Caccia la pagina che non viene letta da più tempo (la più intelligente).
3. Scelta la "vittima" da cacciare, controlla il suo bit dirty. Se è a 1, la salva fisicamente sul disco. Se è a 0, la sovrascrive direttamente (tanto non era stata modificata).
4. Infine, carica la nuova pagina richiesta, imposta il suo count a 1 e il dirty a 0.

Le Politiche di Scrittura (Steal e Force)

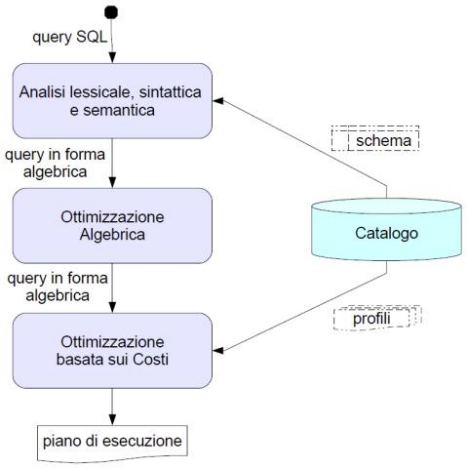
Questa è una classica domanda teorica legata alle transazioni e alla gestione dei guasti. Il Buffer Manager deve seguire delle "regole di ingaggio" per decidere quando scrivere i dati dalla RAM al disco:

- **Regole di Rimpiazzo (Steal vs No-Steal):**
  - **Steal (Rubare):** Il database può "rubare" una pagina a una transazione *non ancora conclusa (non committata)* e salvarla sul disco per fare spazio ad altro. È la politica più efficiente e usata, ma richiede file di Log complessi nel caso la transazione fallisca in seguito.
  - **No Steal:** Una pagina modificata da una transazione in corso è intoccabile. Resta bloccata in RAM finché la transazione non finisce (rischioso se la RAM si riempie, le altre operazioni vengono messe in attesa).
- **Regole di Chiusura (Force vs No-Force):**
  - **Force (Forzare):** Non appena l'utente preme "Salva" (fa il *Commit*), il database è obbligato a scrivere *immediatamente* tutte le modifiche sull'hard disk. È sicurissimo in caso di sbalzi di corrente, ma lentissimo se ci sono decine di salvataggi al secondo.
  - **No Force:** Quando l'utente fa il *Commit*, il database gli dice "Ok, salvato!", ma in realtà lascia i dati tranquillamente nella RAM (col bit dirty a 1). Li scriverà sul disco con calma più tardi. È velocissimo, ma in caso di crash si affida ciecamente al Gestore dell'Affidabilità e ai file di Log per non perdere i dati confermati.

Il Gestore delle Query (Query Processor)

Il **Gestore delle Query** è il componente del Database Management System (DBMS) responsabile dell'**ottimizzazione delle interrogazioni**.

Il suo obiettivo principale è analizzare un'istruzione SQL in ingresso e scegliere la "migliore" strategia esecutiva tra le diverse alternative possibili. Lo scopo finale di questo processo è **minimizzare i tempi di risposta** del sistema.



Il Processo di Ottimizzazione: Le 3 Fasi

Il processo di ottimizzazione di una query si divide in tre fasi distinte, ognuna con un compito specifico:

1. Analisi (Lessicale, Sintattica e Semantica)

In questa fase preliminare, il sistema verifica la correttezza della query. Per farlo, si basa sulle informazioni relative allo schema del database contenute all'interno del **catalogo**. Al termine di questa validazione, la query SQL viene tradotta nella sua forma algebrica corrispondente.

2. Ottimizzazione Algebrica

Questa fase manipola la query appena tradotta. L'obiettivo è trasformarla in una forma **totalmente equivalente** a quella di partenza, ma strutturata in modo da poter essere eseguita con maggiore efficienza dal sistema.

3. Ottimizzazione Basata sui Costi

Sfruttando le informazioni sulle relazioni della base di dati prelevate anch'esse dal catalogo, il sistema valuta il "costo" delle diverse operazioni. Al termine di questa fase, viene generato il **piano di esecuzione finale**, ovvero la sequenza precisa di operazioni di basso livello (scansioni, ordinamenti, ecc.) necessarie per risolvere l'interrogazione.

Approfondimento: Ottimizzazione Algebrica

L'ottimizzazione algebrica si fonda sui principi dell'**algebra relazionale**, e in particolare sul concetto di equivalenza.

Definizione: Espressioni dell'algebra equivalenti Due espressioni algebriche relative a un'interrogazione si definiscono equivalenti se **producono l'identico risultato**, indipendentemente dallo stato (istanza attuale) in cui si trova la base di dati in quel momento.

Le Euristiche Fondamentali

Le strategie principali (euristiche) utilizzate per migliorare le espressioni algebriche sono due:

1. **Push selections down:** Anticipare le operazioni di selezione.
2. **Push projections down:** Anticipare le operazioni di proiezione.

1. Push selections down (Anticipare le selezioni)

Perché funzionano? (Esempio Pratico) Consideriamo due tabelle:

IMPIEGATI(Matricola, Nome, Cognome, Dip)

DIPARTIMENTI(Cod, Nome, Indirizzo, Città) e la seguente query:

```sql
SELECT *
FROM DIPARTIMENTI D JOIN IMPIEGATI I ON D.Cod=I.Dip
WHERE I.Cognome='Moscato';
```

Applicando la regola del ***Pushing selection down***, la condizione di selezione (**Cognome='Moscato'**) viene eseguita **prima** dell'operazione di JOIN. Questo rende l'esecuzione molto più efficiente: filtrando prima i dati, si riduce drasticamente il numero di tuple (righe) che dovranno essere combinate durante il JOIN vero e proprio.

2. Push projections down (Anticipare le proiezioni)

Mentre il primo approccio agisce sulle righe, questo secondo approccio ha l'obiettivo di ridurre la dimensione in byte di ogni singola riga, eliminando le colonne superflue il prima possibile per non sovraccaricare la memoria durante i calcoli.

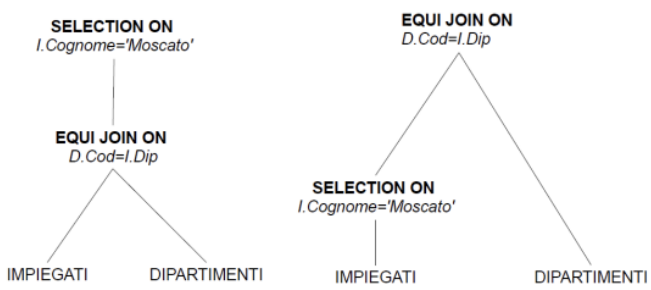
Esempio Pratico Utilizzando le stesse tabelle, consideriamo la seguente query:

```sql
SELECT I.Nome, I.Cognome, D.Nome AS NomeDipartimento
FROM DIPARTIMENTI D JOIN IMPIEGATI I ON D.Cod = I.Dip;
```

Applicando il Pushing projections down: L'ottimizzatore capisce che molte colonne non servono né per il risultato finale né per calcolare il JOIN, quindi anticipa la proiezione "tagliando" le tabelle **prima** di unirle:

- Sulla tabella IMPIEGATI, proietta (mantiene) solo Nome, Cognome (richiesti nell'output) e Dip (necessario per il JOIN). Scarta subito Matricola.
- Sulla tabella DIPARTIMENTI, proietta solo Nome (richiesto nell'output) e Cod (necessario per il JOIN). Scarta subito Indirizzo e Città.
- Alla fine, esegue il JOIN.

RAPPRESENTAZIONE CON ALBERI DI QUERY



Un albero di query è la rappresentazione grafica del piano di esecuzione di un'interrogazione e **si legge sempre dal basso verso l'alto.**

Albero a Sinistra (Piano NON ottimizzato / Ingenuo)

Albero a Destra (Piano OTTIMIZZATO - Pushing selection down)

La Procedura di Ottimizzazione (Step-by-Step)

Per applicare correttamente l'ottimizzazione algebrica tramite gli alberi di query, il sistema segue questi passaggi:

1. **Scomposizione:** Decomporre le selezioni complesse congiuntive (collegate da AND) in una serie di selezioni atomiche (singole).

   ```
   SELECT * FROM IMPIEGATI
   WHERE Città = 'Milano' AND Ruolo = 'Manager' AND AnnoAssunzione = 2024;
   ```

   L'ottimizzatore del DBMS "smonta" quella riga e la trasforma in tre **selezioni atomiche** (cioè non più riducibili):

   - Filtro A: Città = 'Milano'
   - Filtro B: Ruolo = 'Manager'
   - Filtro C: AnnoAssunzione = 2024
2. **Anticipazione Selezioni:** Spingere le selezioni il più in basso possibile nell'albero (anticiparle).
3. **Priorità di Selettività:** Se ci sono più selezioni in sequenza, eseguire per prime quelle più "selettive" (che scartano più dati).
4. **Formazione dei Join:** Combinare i prodotti cartesiani seguiti da selezioni per trasformarli direttamente in operazioni di JOIN.
5. **Anticipazione Proiezioni:** Spingere le proiezioni il più in basso possibile, anche introducendone di nuove se aiuta a scartare colonne inutili in anticipo.

Approfondimento: Ottimizzazione Basata sui Costi

Una volta individuato l'albero algebrico più conveniente, il DBMS passa all'ottimizzazione basata sui costi. Per farlo, utilizza i **Profili**, ovvero metadati statistici conservati nel catalogo che descrivono le caratteristiche delle tabelle.

I profili contengono dati quantitativi fondamentali, tra cui:

- **Cardinalità:** Il numero totale di tuple in una relazione.
- **Dimensioni:** Lo spazio occupato dalle tuple e dai singoli attributi.
- **Valori:** Il numero di valori distinti per un attributo, oltre ai suoi valori minimi e massimi.

I Piani di Query

Utilizzando i profili, il DBMS stima quanto saranno grandi i risultati temporanei durante l'esecuzione della query. In base a queste stime, sceglie la migliore combinazione di operazioni di basso livello (come *scansione sequenziale, accesso diretto tramite indice, ordinamento, o algoritmi di join*).

Esistono due approcci per la gestione dei piani di esecuzione:

Modalità	Descrizione
Compile & Store	Il piano di esecuzione generato viene salvato nel catalogo. Se la stessa query viene richiesta in futuro, il sistema richiama direttamente il piano salvato senza dover ricalcolare l'ottimizzazione.
Compile & Go	L'ottimizzazione viene eseguita *on the fly*. Il sistema determina e calcola il piano di esecuzione ottimale ogni singola volta che la query viene lanciata.

Gestione delle Transazioni

In un database moderno, numerosi programmi e utenti richiedono di accedere e modificare i dati simultaneamente. Questa elevata mole di operazioni concorrenti richiede un controllo rigoroso da parte del DBMS per garantire due aspetti vitali:

- **Consistenza:** Mantenere intatta l'integrità dei dati.
- **Affidabilità:** Assicurare la stabilità del sistema complessivo.

Per gestire tutto questo, il DBMS si affida a due moduli specifici:

1. **Gestore della Concorrenza:** Si assicura che gli accessi simultanei agli stessi dati non generino conflitti o letture errate, orchestrando il traffico delle richieste.
2. **Gestore dell'Affidabilità:** Ha il compito fondamentale di preservare il contenuto della base di dati, garantendo che i dati non vadano persi o corrotti nemmeno in caso di *crash* improvviso del sistema.

Le Transazioni nei Database

Definizione

Una **transazione** è un'operazione singola, o una sequenza di operazioni, generata da un utente o da un programma applicativo. Il suo scopo è accedere al database per **leggere** o **aggiornare** i dati in esso contenuti.

Caratteristiche Principali

- **Unità logica di elaborazione:** A livello concettuale è vista come un'unica azione, ma "dietro le quinte" si traduce in una serie di operazioni fisiche elementari (letture e scritture sul disco).
- **Flessibilità:** Può coincidere con un intero programma, con una porzione di esso, o anche semplicemente con un singolo comando SQL (come un INSERT o un UPDATE).
- **Atomicità (Indivisibilità):** È la proprietà più importante. Una transazione costituisce un blocco *indivisibile* di modifiche allo stato del database. Questo significa che non può essere applicata a metà: o viene eseguita per intero, o non viene eseguita affatto.

Gli Esiti: Commit e Abort

Una transazione possiede due soli stati finali possibili:

- **Commit (Successo):** La transazione termina regolarmente nello stato previsto dal programma. Tutte le modifiche vengono salvate in modo definitivo e la base di dati raggiunge un **nuovo stato consistente**.
- **Abort (Fallimento):** Se si verifica un errore, la transazione fallisce e viene interrotta. La base di dati viene quindi riportata esattamente allo stato in cui si trovava *prima* del suo inizio.

Il concetto di Rollback (o Undo): Quando si verifica un *Abort*, il sistema deve obbligatoriamente "disfare" (annullare) eventuali azioni o effetti parziali che la transazione aveva già applicato prima di interrompersi. Se queste modifiche a metà venissero mantenute, il database si troverebbe in uno stato di **inconsistenza** (cioè conterrebbe dati illogici o corrotti).

Le 4 Proprietà ACID delle Transazioni

1. Atomicità (A - *Atomicity*)

Una transazione atomica deve essere eseguita nella sua interezza oppure non eseguita per niente

2. Consistenza (C - *Consistency*)

Il database parte da uno stato "consistente" (valido e corretto). Una transazione, al termine della sua esecuzione, deve lasciare il database in un **nuovo stato anch'esso consistente**. Il DBMS si assicura che, durante le modifiche, non venga mai violato alcun vincolo di integrità definito nel database (ad esempio, chiavi primarie, chiavi esterne o vincoli sui valori).

3. Isolamento (I - *Isolation*)

La regola del "Lavorare a porte chiuse". In un database moderno, migliaia di transazioni avvengono contemporaneamente (in concorrenza). L'isolamento garantisce che ogni transazione venga eseguita in modo **totalmente indipendente**, come se fosse l'unica attiva in quel momento. Gli effetti parziali di una transazione ancora in corso non devono mai essere visibili alle altre transazioni.

- *In pratica:* Se due utenti stanno cercando di prenotare l'ultimo posto disponibile su un volo contemporaneamente, le loro transazioni vengono isolate. Il sistema non permetterà mai al secondo utente di vedere il posto come "disponibile" se il primo utente lo ha già messo nel carrello ma non ha ancora cliccato su "Paga" (ovvero, non ha ancora fatto il *commit*).

4. Persistenza (D - *Durability*)

Una volta che una transazione termina con successo e viene emesso un comando di *Commit*, i suoi effetti diventano **permanenti e definitivi**.

"Blocco" di una Transazione

A livello di codice, una transazione non è altro che un blocco di istruzioni racchiuso tra un punto di inizio e un punto di fine:

- **BoT (Begin of Transaction):** Segna l'avvio formale della transazione (spesso indicato con begin transaction, o implicito).
- **EoT (End of Transaction):** Segna la fine del blocco.

All'interno di questo blocco, a tempo di esecuzione verrà eseguito **uno e un solo comando** per chiudere la transazione. La scelta su quale dei due attivare viene gestita nel codice tramite condizioni logiche (es. costrutti IF/ELSE) o sistemi di cattura delle eccezioni (es. TRY/CATCH):

- **commit**: eseguito se non ci sono errori, conferma il blocco e rende permanenti (salvate su disco) tutte le modifiche.
- **rollback**: innescato in caso di errori o fallimento dei vincoli, dichiara l'abort e annulla qualsiasi modifica fatta fino a quel momento.

Esempio di blocco base (esito ottimistico):

```
begin transaction
  UPDATE PRODOTTI SET Quantità = Quantità - y WHERE Codice = x;
  INSERT INTO CARRELLO(Cliente, Prodotto, Qta, Data) VALUES (z, x, y, '13-Gen-2015');
commit
```

Transazioni e Vincoli di Integrità

Come abbiamo visto, le transazioni devono rispettare la regola della *Consistenza*. Ma **quando** il database controlla che i vincoli non vengano violati? Esistono due approcci:

- **Controllo Differito:** Il DBMS aspetta la fine. Esegue tutte le istruzioni e, solo quando arriva la richiesta di commit, verifica i vincoli. Se c'è una violazione, forza un *abort* e annulla tutto.
- **Controllo Diretto (o Attivo):** È il programmatore a inserire dei controlli *durante* l'esecuzione, guidando la transazione verso il successo o il fallimento senza subire blocchi imprevisti dal sistema.

Esempio di controllo Diretto: In questo caso, prima di vendere, controlliamo attivamente se ci sono abbastanza prodotti in magazzino (Q > 0).

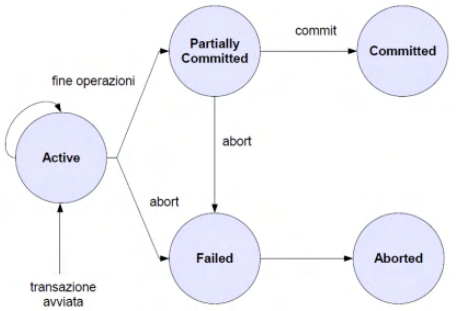
```
begin transaction
  SELECT Quantità INTO Q FROM PRODOTTI WHERE Codice = x;

  IF (Q > 0) THEN  -- Controllo diretto del vincolo
    UPDATE PRODOTTI SET Quantità = Quantità - y WHERE Codice = x;
    INSERT INTO CARRELLO(Cliente, Prodotto, Qta, Data) VALUES (...);
    commit         -- Tutto ok, confermo!
  ELSE
    rollback       -- Quantità insufficiente, annullo io la transazione!
  END IF
```

Fase Operativa

- **Active (Attiva):** È lo stato iniziale. La transazione è stata avviata e le sue istruzioni (letture/scritture) sono attualmente in corso di esecuzione.

Fasi Transitorie

- **Partially Committed (Parzialmente Committata):** La transazione entra in questo stato quando ha eseguito con successo la sua *ultima istruzione logica*, ma non ha ancora scritto i dati in modo permanente sul disco (memoria secondaria). È una fase delicata: la transazione potrebbe ancora fallire se, proprio in questo istante, si verifica un crash del sistema (impedendo il salvataggio) o se il controllo differito dei vincoli rileva un'anomalia logica.



- **Failed (Fallita):** La transazione entra in questo stato "ponte" in due casi:
    1. Se subisce un errore o viene abortita mentre è ancora in stato *Active*.
    2. Se qualcosa va storto mentre è in stato *Partially Committed* (es. crash o violazione dei vincoli).

Fasi Finali (Terminazione)

- **Committed (Confermata):** Se dal parziale commit il salvataggio su disco va a buon fine, la transazione passa a questo stato definitivo. Le modifiche sono ora permanenti (proprietà di *Persistenza*).
- **Aborted (Annullata):** Dallo stato *Failed*, il sistema esegue il *rollback* (disfa le modifiche parziali) e chiude la transazione in questo stato. Il database è tornato esattamente com'era prima dell'avvio.

Accesso Concorrente

Uno degli scopi primari di un database moderno è permettere a più utenti o applicazioni di accedere ai dati condivisi **nello stesso momento**.

Tuttavia, quando avvengono accessi simultanei e *almeno uno di questi comporta una modifica* (scrittura/aggiornamento), si rischia di corrompere i dati, generando quelle che vengono definite **anomalie** (inconsistenze).

Come può difendersi il database? Esistono due approcci:

Approccio	Descrizione	Pro e Contro
Esecuzione Seriale (Soluzione banale)	Le transazioni vengono messe in fila indiana ed eseguite una dopo l'altra.	**Pro:** Zero anomalie. **Contro:** È impensabile nei sistemi reali (OLTP), che gestiscono centinaia di transazioni al secondo. Rallenterebbe drasticamente i tempi di risposta.
Esecuzione Interleaved (Alternata)	Il sistema esegue più transazioni in concorrenza, alternando le operazioni dell'una con quelle dell'altra.	**Pro:** Massimizza il *throughput* (transazioni per secondo). **Contro:** Genera anomalie se non controllata rigorosamente.

L'Obiettivo del Gestore della Concorrenza

Come nei sistemi operativi, l'esecuzione *interleaved* sfrutta i tempi morti: mentre la Transazione A attende che il disco legga un dato (operazione di I/O), la Transazione B usa il processore (CPU) per fare un calcolo.

Il **Gestore della Concorrenza** ha quindi un compito vitale: implementare politiche che permettano questa sovrapposizione rapida, ma *prevenendo in anticipo* il verificarsi di qualsiasi anomalia.

Il Modello Astratto di Transazione

Per studiare e prevenire le anomalie, il DBMS non analizza l'intero codice SQL, ma utilizza un **modello matematico semplificato**.

In questo modello a singolo livello di controllo:

- **Apertura/Chiusura:** Si usano i comandi base **bot** (*begin transaction*), **commit** e **rollback**.
- **Operazioni elementari:** Tutto si riduce a due sole azioni:
  - $read(x, y)$: legge l'oggetto x dal database e lo salva in una variabile temporanea y.
  - $write(x, y)$: scrive il valore della variabile y all'interno dell'oggetto x nel database.
- **Asse temporale:** Ogni singola operazione è etichettata con un istante di tempo ($t_1, t_2, t_3 ..$) per capire l'ordine esatto di esecuzione.

Nota pratica: Nella realtà fisica, il database non estrae il singolo campo della tabella (l'oggetto astratto x), ma carica in memoria l'intera "pagina" (il blocco di memoria) che contiene quel dato. Nel modello teorico, però, questa complessità viene ignorata.

La transazione deve aggiornare il magazzino e poi inserire tutti i dati del cliente, del prodotto e della data nel carrello. Richiede ben 13 istanti temporali:

```
// --- INIZIO TRANSAZIONE ---
t1:  bot
     // "Begin of Transaction". Il database registra l'inizio delle operazioni.

// --- FASE 1: AGGIORNAMENTO DEL MAGAZZINO ---
t2:  read(qtaP_x, A)
     // Il DB va sul disco, cerca la quantità attuale del prodotto 'x',
     // la legge e la copia nella memoria RAM dentro una variabile chiamata 'A'.

t3:  A = A - y
     // Il DB fa un calcolo veloce in RAM: prende il valore di 'A'
     // e gli sottrae 'y' (la quantità che stiamo acquistando).
     // Ora 'A' contiene la nuova quantità aggiornata.

t4:  write(qtaP_x, A)
     // Il DB prende il nuovo valore di 'A' dalla RAM e lo va a sovrascrivere
     // sul disco fisso (aggiornando definitivamente la giacenza in magazzino).

// --- FASE 2: INSERIMENTO NEL CARRELLO ---
t5:  B = z
     // Prepara in RAM una variabile 'B' e le assegna il codice cliente ('z').

t6:  write(clienteC_xz, B)
     // Scrive sul disco il codice del cliente ('B') nel record del carrello.

t7:  C = x
     // Prepara in RAM una variabile 'C' e le assegna il codice prodotto ('x').

t8:  write(prodC_xz, C)
     // Scrive sul disco il codice prodotto ('C') nel record del carrello.

t9:  D = y    // (Nella slide c'è scritto t7 per errore)
     // Prepara in RAM una variabile 'D' e le assegna la quantità acquistata ('y').

t10: write(qtaC_xz, D)
     // Scrive sul disco la quantità ('D') nel record del carrello.

t11: E = '13-Gen-2015 20:30:54'
     // Prepara in RAM una variabile 'E' con la data e l'ora attuali.

t12: write(dataC_xz, E)
     // Scrive sul disco la data ('E') nel record del carrello.

// --- FINE TRANSAZIONE ---
t13: commit
     // "Transazione completata". Il DB conferma che tutte le operazioni
     // precedenti sono andate a buon fine e rende le modifiche permanenti.
```

LOST UPDATE

Il "Lost Update" avviene quando due transazioni leggono lo stesso dato quasi nello stesso istante, fanno un calcolo separato e poi scrivono il risultato sul database una dopo l'altra. La seconda transazione finisce per **sovrascrivere e cancellare (perdere)** il lavoro fatto dalla prima, perché ha basato i suoi calcoli su un dato vecchio.

L'Esempio Pratico

Immaginiamo un e-commerce. In magazzino ci sono **100 unità** di un prodotto ($qtaP_x$).

- **Transazione 1 (T_1):** L'utente z_1 sta comprando 50 unità.
- **Transazione 2 (T_2):** L'utente z_2 sta comprando 30 unità.

Se tutto funzionasse correttamente (in modo seriale), alla fine in magazzino dovrebbero rimanere **20 unità** (100 - 50 - 30). Ma vediamo cosa succede nell'esecuzione "interleaved" (alternata) mostrata nella tabella:

Time	T_1	T_2	$qtaP_x$
t_1	**bot**		100
t_2	$read(qtaP_x, A)$	**bot**	100
t_3	$A = A - 50$	$read(qtaP_x, A)$	100
t_4	$write(qtaP_x, A)$	$A = A - 30$	50
t_5	$write(qtaC_{xz_1}, 50)$	$write(qtaP_x, A)$	70
t_6	**commit**	$write(qtaC_{xz_2}, 30)$	70
t_7		**commit**	70

- **Istante t_1:** La transazione T_1 inizia. In magazzino ci sono 100 pezzi.
- **Istante t_2:** T_1 legge la quantità dal disco e se la salva nella sua RAM locale (A = 100). Nel frattempo, la transazione T_2 inizia.
- **Istante t_3 (IL MOMENTO CRITICO):** T_1 fa il suo calcolo: 100 - 50 = 50.
    - Contemporaneamente, T_2 **legge la quantità dal disco**. Poiché T_1 non ha ancora salvato nulla sul disco, T_2 **legge 100**. Anche T_2 si salva questo valore nella sua RAM locale (A = 100). *Qui nasce l'errore: T_2 sta lavorando su un dato che sta per diventare obsoleto.*
- **Istante t_4:** * T_1 scrive il suo risultato sul disco fisso: aggiorna il magazzino a **50**.
    - T_2 fa il suo calcolo locale: 100 - 30 = 70.

- **Istante t_5 (IL DISASTRO):** T_1 registra tranquillamente nel carrello dell'utente z_1 che ha comprato 50 pezzi.
    - T_2 scrive il suo risultato sul disco fisso: **aggiorna il magazzino a 70**. Questo passaggio **sovrascrive brutalmente** il 50 scritto da T_1 un secondo prima. L'aggiornamento di T_1 è ufficialmente "perso".
- **Istanti t_6 e t_7:** Entrambe le transazioni fanno il *commit* e si chiudono credendo di aver fatto un ottimo lavoro.

LETTURA SPORCA

La lettura sporca (Dirty Read) si verifica quando una transazione, durante la sua esecuzione, legge un valore “sporco” di un oggetto, ovvero modificato da un’altra transazione che poi viene abortita.

L'Esempio Pratico (La Tabella)

Partiamo con **100 unità** di un prodotto in magazzino ($qtaP_x$).

Time	T_1	T_2	$qtaP_x$
t_1	**bot**		100
t_2	$read(qtaP_x, A)$		100
t_3	$A = A - 50$		100
t_4	$write(qtaP_x, A)$		50
t_5	$write(qtaC_{xz_1}, 50)$	**bot**	50
t_6	...	$read(qtaP_x, A)$	50
t_7	...	$A = A - 30$	50
t_8	...	$write(qtaP_x, A)$	20
t_9	**rollback**	$write(qtaC_{xz_2}, 30)$	70
t_{10}		**commit**	70

- **Istanti $t_1 - t_4$:** L'utente 1 avvia la sua transazione (T_1). Vuole comprare 50 unità. Legge 100, fa il calcolo (100 - 50 = 50) e *scrive* 50 nel database. **Attenzione:** a questo punto T_1 non ha ancora finito, non ha fatto il commit. Il valore 50 è solo temporaneo.
- **Istanti $t_5 - t_6$ (LA LETTURA SPORCA):** L'utente 2 avvia la sua transazione (T_2). Vuole fare un acquisto e il sistema va a leggere quante unità ci sono in magazzino. Poiché T_1 ha appena scritto 50, T_2 **legge 50**. *Questo è l'errore: T_2 sta leggendo un dato "sporco", non ancora confermato.*
- **Istanti $t_7 - t_8$:** L'utente 2 decide di comprare 30 unità. Fa il calcolo basandosi sul dato sporco (50 - 30 = 20) e scrive il nuovo totale, **20**, nel database.
- **Istante t_9 (IL COLPO DI SCENA):** L'utente 1 cambia idea e annulla l'ordine (oppure la sua app va in crash). La transazione T_1 esegue un **rollback**. Il database deve annullare tutto ciò che T_1 aveva fatto, "restituendo" le 50 unità al magazzino. Il magazzino passa misteriosamente da 20 a **70**.
- **Istante t_{10}:** La transazione T_2 fa il *commit*, del tutto ignara di quello che è appena successo.

Perché è un problema grave? (Le Conseguenze)

Guardando la fine della tabella, potresti pensare: *"Beh, alla fine in magazzino ci sono 70 unità, ed è corretto perché l'unico acquisto reale è stato quello di 30 da parte dell'utente 2 (100 - 30 = 70)."*

Il problema non è il risultato matematico finale, ma **l'esperienza e le limitazioni subite dall'utente 2 a causa del dato falso.**

Quando T_2 ha letto che c'erano solo **50 unità** disponibili, l'interfaccia dell'e-commerce ha mostrato all'utente 2 che non poteva comprare più di 50 pezzi. Se l'utente 2 avesse voluto comprare **80 unità**, il sistema lo avrebbe bloccato dicendogli *"Disponibilità insufficiente"*. In realtà, poiché l'utente 1 ha poi annullato l'ordine, c'erano ben **100 unità** a disposizione! Il business ha perso una grande vendita perché il database ha permesso a T_2 di leggere un dato "sporco".

LETTURA INCONSISTENTE

La lettura inconsistente (Inconsistent Read) si verifica quando una transazione legge in due diversi istanti della sua esecuzione due valori differenti dello stesso oggetto che viene, intanto, modificato da un'altra transazione.

L'Esempio Pratico (La Tabella)

Questa volta i protagonisti sono un po' diversi. Abbiamo un magazzino con **20 unità** ($_{qtaP_x}$) e una regola aziendale: *dobbiamo avere sempre almeno 50 pezzi in magazzino*.

- **Transazione 1 ($_{T_1}$):** È il software gestionale che si occupa del riordino automatico dai fornitori.
- **Transazione 2 ($_{T_2}$):** È un normale cliente che fa un acquisto.

Time	T_1	T_2	$qtaP_x$
t_1	**bot**		20
t_2	$read(qtaP_x, A)$		20
t_3	...		20
t_5	...	**bot**	20
t_6	...	$read(qtaP_x, A)$	20
t_7	...	$A = A - 20$	20
t_8	...	$write(qtaP_x, A)$	0
t_9	...	$write(qtaC_{xz}, 30)$	0
t_{10}	...	**commit**	0
t_{11}	$read(qtaP_x, A)$		0
t_{12}	...		0
t_{13}	**commit**		0

- **Istanti $_{t_1 - t_2}$:** Il sistema di riordino ($_{T_1}$) inizia il suo controllo. Legge che in magazzino ci sono **20 unità**. Fa il suo calcolo: *"L'obiettivo è 50, ne ho 20, quindi preparo un ordine per il fornitore di 30 unità"*.
- **Istanti $_{t_5 - t_{10}}$ (L'INTRUSIONE):** Mentre $_{T_1}$ sta ancora elaborando l'ordine, arriva il cliente ($_{T_2}$). Il cliente compra tutte le 20 unità rimaste, porta il magazzino a **0** e fa il *commit*. La sua operazione è perfettamente valida e definitiva.
- **Istante $_{t_{11}}$ (LA LETTURA INCONSISTENTE):** Il sistema di riordino ($_{T_1}$), prima di inviare l'email definitiva al fornitore, fa un ultimo controllo di sicurezza e rilegge la quantità in magazzino. **Sorpresa: legge 0!** *(Nota per te: nell'immagine alla riga $_{t_9}$ c'è un piccolo errore del prof. C'è scritto che scrive 30 nel carrello, ma dal calcolo*

precedente $A = A - 20$ è chiaro che il cliente sta comprando 20 unità, come spiegato anche nel testo della tua slide).

La Conseguenza (Il caos logico)

Arrivati al punto t_{11}, il software gestionale (T_1) va in crisi.

All'inizio aveva preparato un ordine per **30 unità** (basandosi sul fatto che ce ne fossero 20 in magazzino). Ora che il magazzino è a **0**, quell'ordine da 30 non basta più per raggiungere l'obiettivo aziendale di 50! L'ordine andrebbe ricalcolato a 50.

Il software si trova bloccato in uno stato di incertezza: i dati su cui ha basato tutta la sua logica sono cambiati a metà strada. L'applicazione potrebbe andare in errore, annullare tutto, o peggio, inviare un ordine sbagliato al fornitore.

Tutte queste anomalie (Lost Update, Dirty Read, Inconsistent Read) avvengono perché manca un elemento fondamentale: **l'Isolamento** (la "I" delle proprietà ACID).

AGGIORNAMENTO FANTASMA

L'aggiornamento fantasma accade quando una transazione legge dei dati correlati *mentre* un'altra transazione li sta modificando a metà, finendo per vedere solo una parte delle modifiche e ignorando l'altra (che diventa "invisibile" o "fantasma"), generando così un'incoerenza nei dati.

L'Esempio Pratico

Abbiamo un magazzino che vende un kit composto da due pezzi: x_1 (es. cuffie) e x_2 (es. microfono).

- **Vincolo:** Le quantità in magazzino devono essere sempre identiche ($qtaP_{x_1} == qtaP_{x_2}$). Partiamo da **100 unità** per entrambi.
- **Transazione 1 (T_1):** Un software di controllo che si avvia periodicamente per verificare che la regola d'oro sia rispettata.
- **Transazione 2 (T_2):** È un cliente che compra 30 kit (quindi preleva 30 cuffie e 30 microfoni).

Time	T_1	T_2	$qtaP_{x_1}$	$qtaP_{x_2}$
t_1	**bot**		100	100
t_2	$read(qtaP_{x_1}, A)$		100	100
t_2	...	**bot**	100	100
t_3	...	$read(qtaP_{x_1}, A)$	100	100
t_4	...	$A = A - 30$	100	100
t_5	...	$write(qtaP_{x_1}, A)$	70	100
t_6	...	$write(qtaC_{x_1 z}, 30)$	70	100
t_7	...	$read(qtaP_{x_2}, B)$	70	100
t_8	...	$B = B - 30$	70	100
t_9	...	$write(qtaP_{x_2}, B)$	70	70
t_{10}	...	$write(qtaC_{x_2 z}, 30)$	70	70
t_{11}	...	**commit**	70	70
t_{12}	$read(qtaP_{x_2}, B)$		70	70
t_{13}	$qtaP_{x_1} \neq qtaP_{x_2}$		70	70
t_{14}	**commit**		70	70

Vediamo la linea del tempo *(nota: c'è un piccolo refuso nella slide, ci sono due istanti t_2, ma la sequenza è chiara)*:

- **Istante $t_1 - t_2$ (La prima lettura):** Il software di controllo (T_1) si sveglia. Legge la disponibilità delle cuffie (x_1) e si appunta nella sua memoria che ce ne sono **100**. Poi va in pausa un istante.
- **Istanti $t_3 - t_6$ (Il cliente compra x_1):** Entra il cliente (T_2). Compra le cuffie: legge 100, sottrae 30, e aggiorna il magazzino a **70**.
- **Istanti $t_7 - t_{11}$ (Il cliente compra x_2):** Il cliente (T_2) fa la stessa cosa con i microfoni. Legge 100, sottrae 30, e aggiorna il magazzino a **70**. Alla riga t_{11}, il cliente fa il *commit*. La sua transazione è perfetta: ha comprato 30 kit e ha lasciato il magazzino in uno stato consistente (70 cuffie e 70 microfoni).
- **Istante t_{12} (La seconda lettura):** Il software di controllo (T_1) si "risveglia" e va a leggere la quantità dei microfoni (x_2). Poiché il cliente ha finito di comprare, T_1 legge **70**.
- **Istante t_{13} (IL FALSO ALLARME):** Il software T_1 fa il suo controllo. Confronta i dati che ha in mano: $qtaP_{x_1}$ (che aveva letto all'inizio ed era 100) e $qtaP_{x_2}$ (che ha appena letto ed è 70). Conclude che **100 è diverso da 70**.
- **Istante t_{14}:** T_1 fa il *commit* e probabilmente invia una mail di allarme all'amministratore segnalando che il database è corrotto.

Le Conseguenze

A differenza della "Perdita di Aggiornamento" (Lost Update) dove il dato veniva distrutto sul serio, qui **il magazzino reale è a posto**.

Il danno, tuttavia, è logico: il software T_1 crede che la transazione del cliente abbia modificato solo x_2 e non x_1 (l'aggiornamento a x_1 è il "fantasma").

La Teoria del Controllo della Concorrenza

Per studiare matematicamente come gestire gli accessi simultanei, l'intera complessità di una transazione viene ridotta all'osso.

Nella teoria del controllo, **una transazione coincide esclusivamente con la sua sequenza temporale di letture (Read - r) e scritture (Write - w)** sugli oggetti del database. (in pratica una transazione non è altro che la "telecronaca" essenziale dei suoi passaggi sul database)

Vengono **omessi** di proposito:

- I comandi di apertura e chiusura ($bot, commit, rollback$).
- Le operazioni di calcolo in memoria RAM (es. $A = A - 50$).

Esempio di notazione teorica:

$$T_1: r_1(X), w_1(X), r_1(Y), r_1(Z), w_1(Z)$$

$$T_2: r_2(X), r_2(Y), r_2(Z), w_2(Y)$$

(Dove il pedice indica il numero della transazione e la lettera tra parentesi l'oggetto letto o scritto).

Schedule e Scheduler

Poiché le transazioni avvengono in concorrenza, i loro passaggi ($_r$ e $_w$) si incrociano nel tempo. Per definire l'ordine esatto in cui il database esegue questi incroci, usiamo due concetti fondamentali:

- **Schedule (o Schedulazione):** È l'elenco cronologico di tutte le operazioni di lettura e scrittura mischiate tra loro. La regola d'oro di uno schedule è che **deve sempre preservare l'ordine interno** di ciascuna transazione (se in T_1 leggo prima X e poi Y, nello schedule globale $r_1(X)$ dovrà sempre apparire prima di $r_1(Y)$).
    - *Esempi di schedule mischiati per T_1 e T_2:*
        - $S_1: r_1(X), r_2(X), w_1(X), r_1(Y), r_1(Z), w_1(Z), r_2(Z), r_2(Y), w_2(Y)$
        - $S_2: r_2(X), r_1(X), w_1(X), r_1(Y), r_2(Z), r_2(Y), w_2(Y), r_1(Z), w_1(Z)$
- **Scheduler:** È il componente software del DBMS che gestisce il traffico. Il suo compito è accettare, rifiutare o riordinare le operazioni in arrivo per creare uno schedule sicuro che non generi anomalie.

Classificazione degli Schedule

Per capire se uno schedule è sicuro, il DBMS utilizza una classificazione precisa.

*(Nota preliminare - **Commit Proiezione**: per analizzare uno schedule, si ignorano tutte le transazioni che sono andate in abort, tenendo conto solo di quelle andate a buon fine).*

Schedule Seriale (Sicuro ma Lento)

Le transazioni vengono eseguite **una dopo l'altra**, senza alcuna sovrapposizione.

- *Esempio*

$$(S_3): r_1(X), w_1(X), r_1(Y), r_1(Z), w_1(Z), \mathbf{r_2(X)}, \mathbf{r_2(Y)}, \mathbf{r_2(Z)}, \mathbf{w_2(Y)}$$

(Tutta la transazione T_1 finisce, poi inizia T_2. Zero anomalie, ma pessime prestazioni).

Schedule Serializzabile (Il Santo Graal)

È uno schedule non seriale (quindi con operazioni mischiate e veloci) che però **produce lo stesso identico risultato finale** di uno schedule seriale.

- *Esempio*

$$(S_4): r_1(X), w_1(X), \mathbf{r_2(X)}, \mathbf{r_2(Y)}, \mathbf{r_2(Z)}, \mathbf{w_2(Y)}, r_1(Y), r_1(Z), w_1(Z)$$

- *Perché è serializzabile?* Anche se le operazioni sono alternate, notiamo che l'oggetto X viene usato e modificato solo da T_1 all'inizio. Dopo aver scritto X, T_1 si "mette in pausa", lascia lavorare T_2 su altri dati, e poi riprende. Il risultato finale non subisce corruzioni, è esattamente come se fossero state eseguite in fila indiana.

L'Obiettivo del Gestore della Concorrenza: Lo scopo primario della teoria è trovare *classi di schedule serializzabili*. Il DBMS deve essere in grado di riconoscere e produrre questi schedule sicuri in modo rapido, con un "costo" di calcolo bassissimo.

4. Tecniche pratiche per il Controllo di Concorrenza

Nella pratica reale, i DBMS usano tecniche specifiche per "forzare" le transazioni a comportarsi in modo serializzabile. Si dividono in due grandi famiglie filosofiche:

A. Metodi Pessimistici (o Conservativi)

Partono dal presupposto che **le anomalie accadranno sicuramente**. Pertanto, agiscono in via preventiva: se un'operazione rischia di creare un conflitto, lo scheduler la "ritarda" o la mette in attesa.

I due strumenti principali sono:

- **Metodi basati su Lock:** Uso di "lucchetti" temporanei sui dati. Se un dato è bloccato da T_1, T_2 deve aspettare.
- **Metodi basati su Timestamp:** Ad ogni transazione viene assegnata un'etichetta temporale esatta per imporre un ordine cronologico di anzianità.

B. Metodi Ottimistici

Partono dal presupposto opposto: in certi ambienti (es. dove si fanno tantissime letture e pochissime scritture), **i conflitti sono rari**.

Piuttosto che perdere tempo a mettere lucchetti preventivi, il DBMS lascia che tutte le transazioni scorrano veloci in sovrapposizione. L'unico controllo viene fatto **alla fine, subito prima del commit**. Se il sistema si accorge che si è generata un'anomalia, fa fallire (abort) la transazione e la fa ripartire.

METODI BASATI SUI LOCK

La filosofia alla base di questa tecnica conservativa è semplice: se voglio usare un oggetto del database (una tabella, un record, o anche un singolo campo), devo prima "chiuderlo a chiave" in modo che nessun altro possa fargli danni finché non ho finito.

Esistono due tipi di lucchetti:

- **Read Lock (Lucchetto in Lettura):** È un blocco **condiviso**. Se chiedo di leggere un dato, il database me lo concede. Se un altro utente chiede di leggerlo nello stesso momento, il database lo concede anche a lui. *La lettura non fa danni, quindi più persone possono leggere contemporaneamente.*
- **Write Lock (Lucchetto in Scrittura):** È un blocco **esclusivo**. Se voglio modificare un dato, devo avere l'accesso unico e assoluto. Se c'è un Write Lock attivo, nessun altro può leggere o scrivere quel dato finché non lo rilascio.

Se una transazione deve prima leggere e poi scrivere, può chiedere una **Lock Escalation**: parte con un lucchetto in lettura e, appena prima di scrivere, chiede al sistema di "potenziarlo" a lucchetto in scrittura.

Alla fine delle operazioni, la transazione deve obbligatoriamente restituire la chiave tramite il comando **Unlock**.

LOCK MANAGER

Lo Scheduler possederà un componente particolare, detto lock manager, che riceve le varie richieste di lock dalle transazioni e le accoglie o rifiuta, sulla base della tavola dei conflitti.

Quando riceve una richiesta, consulta la **Tavola dei Conflitti** per decidere se dire "Sì" o "No". Un oggetto può trovarsi in tre stati: Libero (free), Bloccato in lettura (r_locked) o Bloccato in scrittura (w_locked).

Ecco le regole d'oro tratte dall'immagine:

1. **Su un oggetto FREE:** Qualsiasi richiesta (read o write) viene accettata (Sì).
2. **Su un oggetto R_LOCKED:** Se chiedi un read_lock, ti viene detto di **Sì** (lettura condivisa). Se chiedi un write_lock, ti viene detto di **No**, perché per scrivere devi essere l'unico sulla risorsa. Devi aspettare in fila.
3. **Su un oggetto W_LOCKED:** Qualsiasi richiesta (read o write) riceve un **No** secco. Chi sta scrivendo non può essere disturbato.

Nota sull'Unlock: Rilasciare un Write Lock riporta l'oggetto a free. Rilasciare un Read Lock, invece, dipende: se c'erano 3 utenti che stavano leggendo, togliere un lucchetto lascia l'oggetto in r_locked per gli altri due. Diventa free solo quando l'ultimo utente se ne va.

2 PHASE LOCKING (2PL)

La Granularità del Lock

Il database non blocca tutto il sistema per una singola operazione. La **granularità** (la precisione del blocco) può variare a seconda della necessità:

- **Intero File/Tabella:** Blocca tutti i dati (massima sicurezza, ma blocca tutti gli altri utenti).
- **Pagina di memoria:** Blocca un blocco fisico di dati sul disco.
- **Record (Riga):** Blocca solo la riga specifica (es. i dati dell'utente Mario Rossi).
- **Singolo Campo:** Blocca solo un attributo (es. solo la colonna "Quantità" di Mario Rossi).

Fase 1: Crescente (Acquisizione)

La transazione richiede e accumula tutti i lucchetti (read_lock e write_lock) di cui ha bisogno sui vari oggetti. I lucchetti aumentano, ma non ne viene rilasciato nessuno.

Fase 2: Decrescente (Rilascio)

La transazione, avendo finito le sue operazioni, inizia a restituire i lucchetti tramite i comandi di unlock.

La Regola d'Oro del 2PL:

Una transazione, dopo aver rilasciato anche solo il suo primissimo lucchetto, **non può MAI più acquisirne di nuovi**. Non si torna indietro alla fase crescente.

Esempio pratico

Se la Transazione 1 (T_1) prende il lock in scrittura sulla quantità di un prodotto, la Transazione 2 (T_2) **deve aspettare in fila**. Quando T_2 finalmente ottiene il suo turno, T_1 avrà già finito tutto l'aggiornamento. Risultato? T_2 leggerà il valore corretto e aggiornato (es. 50 unità), evitando del tutto l'anomalia della *Perdita di Aggiornamento*.

Lo Strict 2PL

Il 2PL classico è ottimo: previene la Perdita di Aggiornamento, l'Aggiornamento Fantasma e la Lettura Inconsistente. **Ma ha un difetto: non ferma la Lettura Sporca (Dirty Read).**

Perché? Nel 2PL classico, una transazione potrebbe finire di scrivere un dato, rilasciare il lucchetto (fase decrescente) ma... fare abort poco dopo! Nel frattempo, un'altra transazione potrebbe aver letto quel dato appena sbloccato, che però era destinato a sparire col rollback.

Per tappare questa falla nasce il **2PL Ristretto (Strict 2PL)**. Aggiunge una regola drastica e potentissima:

- **Nessun lucchetto esclusivo può essere rilasciato durante l'esecuzione.** Tutti i lock vengono tenuti stretti e rilasciati **solo ed esclusivamente** nell'istante in cui viene dichiarato il commit (successo definitivo) o l'abort (fallimento totale). In questo modo, nessuno può sbirciare dati temporanei.

Tutta questa sicurezza genera il problema numero uno dei database concorrenti: il **Deadlock (o Stallo)**.

Il Deadlock è una situazione di blocco totale in cui due o più transazioni si tengono in ostaggio a vicenda.

- **L'esempio classico:** * La transazione T_1 ha bloccato l'oggetto **A** e le serve l'oggetto **B** per finire.
    - La transazione T_2 ha bloccato l'oggetto **B** e le serve l'oggetto **A** per finire.

METODI BASATI SU TIMESTAMP (TS)

A differenza dei protocolli basati su Lock, che gestiscono i conflitti mettendo le transazioni in attesa, il *Timestamp Ordering* garantisce la serializzabilità imponendo un ordine cronologico rigoroso, senza prevedere meccanismi di attesa (wait).

Ogni volta che una transazione nasce, il database le assegna un **Timestamp (τ)**, ovvero un "timbro temporale" (un'etichetta con l'istante esatto di inizio).

- Una transazione nata alle 10:00 ha un τ più piccolo (è **più vecchia**).
- Una transazione nata alle 10:05 ha un τ più grande (è **più giovane**).

Il Metodo "TS Mono-versione" (Come funziona in pratica)

Per far rispettare queste regole, il database appiccica su **ogni singolo oggetto** X (es. il record di un magazzino) due "etichette" che tengono traccia di chi lo ha toccato per ultimo:

- $RTM(X)$ **(Read Timestamp):** È il τ dell'ultima (ovvero la più "giovane") transazione che ha **letto** l'oggetto.
- $WTM(X)$ **(Write Timestamp):** È il τ dell'ultima (ovvero la più "giovane") transazione che ha **scritto/modificato** l'oggetto.

Quando una transazione Tcon timestamp $\tau(T)$richiede un'operazione, il sistema applica le seguenti regole:

- **Operazione di Lettura** $read(T, X)$**:**
  - *Condizione di Abort:* Se $\tau(T) < WTM(X)$, la transazione Tviene **abortita** (e successivamente riavviata con un nuovo timestamp). Questo perché Ttenterebbe di leggere un dato già sovrascritto da una transazione più giovane, violando la causalità temporale.
  - *Esecuzione:* Se la condizione di abort non si verifica, la lettura è consentita. Successivamente, il sistema aggiorna il metadato: $RTM(X) = \max(RTM(X), \tau(T))$.
- **Operazione di Scrittura** $write(T, X)$**:**
  - *Condizione di Abort:* Se $\tau(T) < RTM(X)$oppure $\tau(T) < WTM(X)$, la transazione Tviene **abortita**. Una scrittura in questo contesto invaliderebbe i dati già letti o scritti da transazioni più giovani, compromettendo la consistenza della serializzazione.
  - *Esecuzione:* Se la condizione di abort non si verifica, la scrittura è consentita. Il sistema aggiorna il metadato impostando $WTM(X) = \tau(T)$.

(Nota: In caso di abort, la transazione viene riavviata con un nuovo timestamp, solitamente maggiore di quelli correnti, per garantire la terminazione del sistema).

APPROCCI OTTIMISTICI

In contesti applicativi in cui la probabilità di conflitto d'accesso su dati condivisi è statisticamente trascurabile (es. sistemi *Read-Intensive*), le metodologie pessimistiche (Locking e Timestamp) introducono un *overhead* prestazionale non giustificato. In tali scenari si preferiscono i **protocolli di controllo di concorrenza ottimistici**, i quali evitano meccanismi di sincronizzazione preventiva.

Le transazioni sono libere di operare (letture/scritture) senza alcuna politica di blocco o ritardo. Solo all'atto del commit, viene effettuato un controllo per stabilire se sono stati riscontrati eventuali conflitti, e nel caso, viene effettuato il rollback delle azioni delle transazioni e la relativa riesecuzione (restarting).

Le Tre Fasi dell'Approccio Ottimistico

Il ciclo di vita di una transazione gestita tramite protocolli ottimistici si articola in tre fasi distinte:

1. **Fase di Lettura:** La transazione preleva i valori degli oggetti di interesse dalla base di dati e li memorizza in uno spazio di lavoro locale (variabili locali della transazione). Qualsiasi operazione o aggiornamento matematico viene eseguito esclusivamente su queste copie temporanee. Lo stato effettivo del database non viene alterato in questa fase.
2. **Fase di Validazione:** Prima del *commit*, lo *Scheduler* verifica che lo schedule garantisca la serializzabilità delle transazioni prima di aggiornare il database fisico.
3. **Fase di Scrittura:** Se la validazione ha esito positivo, gli aggiornamenti calcolati nello spazio di lavoro locale vengono propagati (scritti definitivamente) sugli oggetti della base di dati.

Nota sull'efficienza del Rollback: Nell'approccio ottimistico, il *rollback* di una transazione che fallisce la fase di validazione è un'operazione computazionalmente "leggera". Questo accade perché la transazione non ha ancora interagito con il disco: annullarla significa semplicemente svuotare un'area di memoria temporanea (le variabili locali), senza dover ripristinare blocchi fisici di dati nella memoria secondaria.

Gestione della Concorrenza nello Standard SQL

Nei sistemi relazionali, lo standard SQL permette al Database Administrator (DBA) o allo sviluppatore di calibrare la rigidità del controllo di concorrenza definendo il **Livello di Isolamento** della transazione.

In SQL è possibile differenziare transazioni che effettuano solo letture da quelle che effettuano letture/scritture. Per queste ultime viene usato il 2PL stretto mentre per le prime è possibile stabilire il cosiddetto livello di isolamento richiesto, tra i seguenti:

- **READ UNCOMMITTED:** È il livello minimo di isolamento. La transazione non richiede *lock* in lettura e ignora quelli altrui, esponendo all'anomalia della **Lettura Sporca (Dirty Read)**. È infatti possibile accedere a dati modificati da altre transazioni prima del loro *commit*, rischiando di leggere valori soggetti a un successivo *abort*.
- **READ COMMITTED:** Garantisce che la transazione legga esclusivamente dati confermati da un *commit* definitivo. Tuttavia, non vengono mantenuti i *lock* in lettura per l'intera durata della transazione. Questo livello espone quindi all'anomalia della **Lettura Inconsistente (Non-Repeatable Read)**. Di conseguenza, se un oggetto viene letto due volte in istanti temporali diversi, il valore restituito potrebbe variare a causa di aggiornamenti intermedi effettuati da altre transazioni.
- **REPEATABLE READ:** Estende le garanzie del livello precedente assicurando la "ripetibilità" della lettura. Una volta che un record viene letto, il sistema garantisce che, per l'intera durata della transazione, nessuna operazione esterna potrà modificarlo. Se la transazione interroga il medesimo record più volte, otterrà sempre il medesimo risultato, prevenendo così l'anomalia della Lettura Inconsistente.
- **SERIALIZABLE:** È il massimo livello di isolamento. Il DBMS impone vincoli stringenti per garantire la produzione di uno schedule strettamente serializzabile, eliminando in via definitiva ogni forma di anomalia (inclusi i cosiddetti *Aggiornamenti Fantasma*).

Controllo di Affidabilità (*Recovery*)

Il **Controllo di Affidabilità** costituisce il modulo responsabile della resilienza fisica e logica del sistema.

L'obiettivo principale è il ripristino (*recovery*) del database dopo guasti o attacchi.

Il sottosistema di affidabilità garantisce due proprietà ACID:

1. **Atomicità:** annulla le transazioni interrotte per evitare scritture parziali.
2. **Persistenza:** assicura che le transazioni completate (*commit*) siano registrate permanentemente, riscrivendole se necessario al riavvio.

Gerarchia delle Memorie e Persistenza Dei Dati

La progettazione del gestore dell'affidabilità è strettamente legata alle caratteristiche fisiche dello *storage* utilizzato dal DBMS. Si distinguono tre livelli logici di memoria:

- **Memoria Centrale (Storage Primario):** Caratterizzata da tempi di accesso estremamente ridotti (elevata velocità di lettura/scrittura), ha una capacità limitata ed è **volatile**. In caso di mancanza di alimentazione elettrica o di crash del sistema, il suo intero contenuto va perduto.
- **Memoria di Massa (Storage Secondario):** Supporti come HDD o SSD, lenti ma capienti e **persistenti**.
- **Memoria Stabile:** Astrazione ideale immune da guasti, approssimata tecnicamente tramite ridondanza (RAID, replicazione, backup).

Il Gestore dell'Affidabilità e la Criticità delle Scritture

Per adempiere al suo mandato, il **Gestore dell'Affidabilità** deve garantire che gli effetti di tutte le transazioni che hanno effettuato il *commit* siano memorizzati in maniera permanente, rendendo sempre possibile il ripristino del contenuto corretto della base di dati a valle di un guasto.

La problematica cruciale in questo processo risiede nel fatto che **le operazioni di scrittura delle transazioni non sono azioni atomiche**. Si presenta infatti il seguente limite strutturale:

- Gli effetti di una transazione che ha effettuato il *commit* nella memoria primaria (volatile) potrebbero non essere riportati immediatamente sulla memoria secondaria (persistente). Di conseguenza, in caso di crash improvviso prima del trasferimento fisico dei dati, la persistenza della transazione verrebbe compromessa se non adeguatamente protetta e ricostruita dal modulo di recovery.

Concetti Base del Recovery e il File di Log

Il gestore dell'affidabilità è il componente che amministra l'esecuzione dei comandi transazionali fondamentali (begin transaction, commit, rollback o abort) e coordina tutte le procedure di ripristino in seguito a guasti. Lo strumento cardine per svolgere questo compito è il **File di Log**.

- **File di Log:** È un file salvato su *memoria stabile* che registra, nel loro esatto ordine di esecuzione, tutte le operazioni svolte dalle transazioni.
- **Funzione:** Agisce come un "diario di bordo" del DBMS. In qualsiasi istante e a seguito di qualsiasi malfunzionamento, leggendo il log è possibile ricostruire lo stato corretto e consistente della base di dati.

Strumenti di Sincronizzazione: Dump e Checkpoint

Per ottimizzare le operazioni di ripristino e garantire la sicurezza dei dati, il sistema si avvale di due operazioni principali:

- **Dump (Backup):** Corrisponde alla copia di sicurezza dell'intera base di dati su memoria stabile. Viene solitamente pianificata nel tempo ed eseguita quando il sistema non è operativo (*offline*). È vitale per ricostruire il database nel caso in cui la memoria secondaria (non stabile) venga danneggiata fisicamente.
- **Checkpoint:** È un'operazione che serve a "fare il punto della situazione", sincronizzando il contenuto effettivo della base di dati con il file di log. Semplifica enormemente il ripristino, poiché registra le transazioni attive in un dato momento e garantisce che quelle concluse siano salvate.

Le Fasi del Checkpoint

Il checkpoint è un'operazione complessa che richiede il blocco temporaneo del sistema. Si articola in 4 fasi sequenziali:

1. **Sospensione:** Si sospende l'accettazione di qualsiasi nuova richiesta da parte delle transazioni.
2. **Scrittura (Force):** Si trasferiscono fisicamente in memoria di massa (utilizzando le primitive *force* del buffer manager) tutte le "pagine sporche" (modificate) che appartengono a transazioni già andate in *commit*.
3. **Registrazione:** Si scrive sul file di log, in modo sincrono (tramite *force*), un record speciale di checkpoint che contiene gli identificatori di tutte le transazioni attualmente attive.
4. **Ripresa:** Si sblocca il sistema e riprende la normale accettazione delle operazioni.

Risultato del Checkpoint: A valle di questa procedura, si ha la certezza assoluta che i dati di tutte le transazioni concluse siano effettivamente al sicuro in memoria di massa, e si ha una fotografia esatta delle sole transazioni rimaste "in corso".

Anatomia del Log di Sistema

Il file di log memorizza due grandi categorie di record: i **record di sistema** e i **record di transazione**.

1. Record di Sistema

Tengono traccia degli eventi di gestione globale del DBMS:

- **Dump (DP):** Contiene il timestamp (ts) dell'ultimo backup effettuato e alcuni dettagli tecnici (es. file e dispositivi di archiviazione coinvolti). Formato logico: DP(ts).

- **Checkpoint (CK):** Contiene l'insieme delle transazioni attive (T) in un dato istante (ts). Formato logico: CK(ts, T).

2. Record di Transazione

Tengono traccia delle singole azioni sui dati. Seguono la struttura logica **Op (T, ts, O, BI, AI)**.

Parametro	Significato	Descrizione e Valori
Op	**Operazione**	L'azione svolta: begin (**B**), update (**U**), delete (**D**), insert (**I**), commit (**C**), abort (**A**).
T	**Transazione**	L'identificativo univoco della transazione (es. T1, T2).
ts	**Timestamp**	L'istante di tempo in cui è avvenuta l'azione.
O	**Oggetto**	L'identificativo del dato o del record della base di dati coinvolto.
BI	**Before-Image**	Il valore dell'oggetto *prima* di subire la modifica.
AI	**After-Image**	Il valore dell'oggetto *dopo* la modifica.

Esempio Pratico

Per semplicità di lettura, se non si è interessati al momento temporale esatto, il parametro ts può essere omesso (rappresentato con un trattino -), poiché l'ordine consecutivo dei record nel log è sufficiente a indicare la sequenza temporale.

```
DP
B(T1, -, -, -)
U(T1, -, qtaP, 100, 90)
U(T1, -, qtaC, NULL, 10)
C(T1, -, -, -)
B(T2, -, -, -)
CK(T2)
U(T2, -, qtaP, 90, 70)
U(T2, -, qtaC, NULL, 20) *
C(T2, -, -, -)
B(T3, -, -, -)
U(T3, -, qtaP, 100, 90)
U(T3, -, qtaC, NULL, 10)
C(T3, -, -, -)
...
```

Il Ripristino e le Operazioni di Undo / Redo

Il momento cruciale che determina il destino di una transazione e le conseguenti azioni di ripristino è la scrittura (in modo sincrono) del **record di commit** nel file di log. In caso di guasto, il Gestore dell'Affidabilità interviene in due modi contrapposti, a seconda di quando il crash si è verificato rispetto al commit:

- **Undo (Disfacimento):** Se il guasto avviene *prima* della registrazione del commit nel log, la transazione è considerata fallita. Se parte delle modifiche era già stata scritta nella base di dati (in memoria secondaria), il sistema deve annullare queste operazioni leggendo i valori originali (*before-image*) dal log.
- **Redo (Rifacimento):** Se il guasto avviene *dopo* il commit ma prima che i dati vengano trasferiti permanentemente dal buffer (memoria primaria) alla memoria secondaria, la transazione è considerata valida. Il sistema riapplicherà le modifiche leggendo i nuovi valori (*after-image*) dal log per garantirne la persistenza.

Regole Fondamentali di Scrittura sul Log

Per far sì che le operazioni di *Undo* e *Redo* funzionino correttamente e che la base di dati non venga mai corrotta, il sistema impone due regole ferree per la scrittura sul log:

1. **Write-Ahead (Scrittura Anticipata):** Ogni transazione deve scrivere il relativo record sul file di log *prima* di effettuare la corrispondente modifica effettiva sulla base di dati.
    - *Perché è necessaria:* Assicura che il valore precedente alla modifica sia al sicuro sul log, rendendo sempre possibile l'operazione di **Undo**.
2. **Commit Precedenza:** Ogni transazione scrive sul log tutti i relativi record prima di effettuare il commit.
    - *Perché è necessaria:* Assicura che tutti i nuovi valori calcolati dalla transazione siano registrati in modo persistente, garantendo che l'operazione di **Redo** possa andare a buon fine anche se le pagine modificate non sono ancora state riversate sul disco

Modalità di Scrittura nella Base di Dati

Il modo in cui le modifiche vengono trasferite dal buffer alla memoria di massa determina quali operazioni di ripristino saranno necessarie in caso di guasto. Esistono tre politiche principali:

- **Modalità Immediata:** Ogni scrittura sul log è seguita immediatamente dalla scrittura fisica sulla base di dati.

    - *Effetto sul recovery:* Comporta solo operazioni di **Undo** (per le transazioni non committate). Evita le operazioni di Redo, poiché i dati delle transazioni concluse sono già su disco.
- **Modalità Differita:** Le scritture fisiche sulla base di dati avvengono solo *dopo* la memorizzazione del record di commit nel log.
    - *Effetto sul recovery:* Comporta solo operazioni di **Redo** (per riscrivere i dati persi nel buffer). Evita le operazioni di Undo, poiché le transazioni interrotte non hanno mai toccato la memoria secondaria.
- **Modalità Mista:** È la soluzione più comune e flessibile; le scritture possono avvenire in modo sia immediato sia differito a discrezione del buffer manager.
    - *Effetto sul recovery:* Richiede sia operazioni di **Undo** che di **Redo**.

Sintesi del Recovery: L'azione di ripristino consiste nel partire dall'ultimo *Dump* (backup) utile e, analizzando il file di *Log* assieme all'ultimo *Checkpoint*, ricostruire lo stato corretto applicando selettivamente *Undo* o *Redo*.

Tipi di Guasto e Procedure di Ripresa

I malfunzionamenti si dividono in due categorie principali, che innescano procedure di ripristino differenti:

1. Guasti Soft (Ripresa a Caldo / *Warm Restart*)

Includono errori di programma, cadute di tensione o crash del sistema operativo.

- **Danni:** Si perde esclusivamente il contenuto della memoria centrale (volatile). La memoria secondaria (database) e la memoria stabile (log) rimangono intatte.
- **Azione:** Il DBMS esegue una *ripresa a caldo*, ricalcolando lo stato partendo dai dati salvati su disco e dal log.

2. Guasti Hard (Ripresa a Freddo / *Cold Restart*)

Includono i danni fisici o la corruzione dei dispositivi di memoria di massa (es. rottura di un Hard Disk).

- **Danni:** Si perde sia la memoria centrale che la memoria secondaria. Sopravvive solo la memoria stabile (dove risiedono i backup e il log).
- **Azione:** Il DBMS esegue una *ripresa a freddo*, ripristinando prima l'ultimo dump disponibile e riapplicando poi l'intero file di log.

Fasi Universali del Ripristino (Modello Fail-Stop)

Indipendentemente dalla gravità del guasto, la procedura segue sempre tre fasi rigorose:

1. **Arresto:** Si forza l'arresto completo di tutte le transazioni attive sul sistema.
2. **Ripristino di Sistema:** Viene ripristinato il corretto funzionamento dell'hardware e del sistema operativo.
3. **Recovery:** Il DBMS avvia la procedura di ripristino vero e proprio (caldo o freddo) per riportare il database a uno stato consistente.

La Ripresa a Caldo (*Warm Restart*)

La ripresa a caldo viene attivata in seguito a un guasto *soft* (perdita della sola memoria volatile). L'obiettivo è riportare il database a uno stato consistente leggendo il file di log. La procedura si articola in tre fasi precise:

Fase 1: Costruzione degli Insiemi (Analisi a ritroso)

Il sistema scorre il log **all'indietro** (dal momento del crash verso il passato) fino a incontrare l'ultimo record di Checkpoint. In base a questo, classifica le transazioni in due insiemi:

- **Insieme UNDO (Da disfare):** Contiene le transazioni che, al momento del guasto, non avevano ancora effettuato il *commit* (ma potrebbero aver già "sporcato" il database) o che avevano esplicitamente registrato un *abort*.
- **Insieme REDO (Da rifare):** Contiene le transazioni che hanno completato con successo il *commit* prima del guasto, ma i cui dati potrebbero non essere stati ancora trasferiti fisicamente su disco.

Fase 2: Disfacimento (Scansione a ritroso)

Continuando a scorrere il log **all'indietro** (fino all'operazione più vecchia Begin tra tutte le transazioni incluse negli insiemi Undo/Redo), il sistema annulla gli effetti delle transazioni dell'insieme **UNDO**.

Fase 3: Rifacimento (Scansione in avanti)

Una volta raggiunta l'azione più vecchia, il sistema inverte la marcia e ripercorre il log **in avanti** (verso il crash), riapplicando le azioni delle transazioni appartenenti all'insieme **REDO**.

Le azioni di compensazione applicate variano in base al tipo di operazione originale:

Operazione Originale	Azione di UNDO (Disfacimento)	Azione di REDO (Rifacimento)
Update (U)	Si ripristina il valore originale	Si scrive il nuovo valore (*After-Image*, AI).

Operazione Originale	Azione di UNDO (Disfacimento)	Azione di REDO (Rifacimento)
	(*Before-Image*, BI).	
Insert (I)	Si elimina fisicamente l'oggetto (Delete).	Si reinserisce l'oggetto.
Delete (D)	Si reinserisce l'oggetto precedentemente eliminato.	Si elimina nuovamente l'oggetto.

L'Esercizio Riformulato e Spiegato Passo-Passo

Ecco come si applica l'algoritmo alla luce di quanto abbiamo appena visto.

```
Log: DP, B(T1), B(T2), B(T3), I(T1,O1,A1), D(T2,O2,B2), B(T4), U(T4,O3,B3,A3), U(T1,O4,B4,A4),
C(T2), CK(T1,T3,T4), B(T5), U(T5,O5,B5,A5), A(T3), CK(T1,T4,T5), B(T6), A(T4), U(T6,O6,B6,A6),
U(T6,O3,B7,A7), C(T6), crash
```

Passo 1: Trovare lo Spartiacque

Leggiamo il log partendo dalla fine (dal crash) andando all'indietro. Il primo Checkpoint che incontriamo è **CK(T1, T4, T5)**.

Questo ci dice che, in quel preciso momento, le uniche transazioni in sospeso nel sistema erano T1, T4 e T5. Tutto il resto (T2, T3) era già un capitolo chiuso.

Passo 2: Smistamento (Creazione degli insiemi)

Ora guardiamo cosa succede *dopo* quel Checkpoint fino al momento del crash:

- T1 e T5 non fanno nulla di conclusivo (nessun Commit). $\rightarrow$ Vanno in **UNDO**.
- T4 fa un Abort. $\rightarrow$ Va in **UNDO**.
- Nasce T6 e riesce a fare Commit prima del crash. $\rightarrow$ Va in **REDO** (è l'unica!).

Risultato:

- **Insieme UNDO:** {T1, T4, T5}
- **Insieme REDO:** {T6}

Passo 3: Fase di UNDO (Scansione all'indietro)

Ora dobbiamo cancellare gli effetti delle transazioni in UNDO. Troviamo la più vecchia tra queste (T1) e partiamo dal crash andando a ritroso, annullando le loro modifiche ripristinando il *Before-Image*:

1. Incontriamo U(T5,O5,B5,A5) $\rightarrow$ Annulliamo, ripristinando **O5 = B5**
2. Incontriamo U(T1,O4,B4,A4) $\rightarrow$ Annulliamo, ripristinando **O4 = B4**
3. Incontriamo U(T4,O3,B3,A3) $\rightarrow$ Annulliamo, ripristinando **O3 = B3**
4. Incontriamo I(T1,O1,A1) $\rightarrow$ Annulliamo un inserimento cancellando il dato: **Delete(O1)**

Passo 4: Fase di REDO (Scansione in avanti)

Le scritture interrotte sono state pulite. Ora invertiamo la marcia: partiamo dall'azione più vecchia e andiamo verso la fine del log (verso il crash), riapplicando **solo** le azioni di T6, usando l'*After-Image*:

1. Incontriamo U(T6,O6,B6,A6) $\rightarrow$ Riapplichiamo la modifica: scriviamo **O6 = A6**
2. Incontriamo U(T6,O3,B7,A7) $\rightarrow$ Riapplichiamo la modifica: scriviamo **O3 = A7**

A questo punto il database è perfettamente coerente: le operazioni a metà sono state rimosse e l'unica operazione conclusa correttamente di recente (T6) è stata salvata per sempre.

La Ripresa a Freddo (*Cold Restart*)

La ripresa a freddo interviene quando si verifica un guasto *hard* (o un guasto soft critico) che provoca danni fisici o logici alla memoria secondaria (il database vero e proprio). Essendo andati persi i dati su disco, la memoria stabile (dove risiedono i backup e il log) è l'unica ancora di salvezza.

La tecnica universale prevede l'esecuzione sequenziale di tre passi:

1. **Ripristino del Dump:** Si formatta/prepara il nuovo disco e si ripristinano tutti i dati a partire dall'ultimo backup disponibile (individuato tramite l'ultimo record DP nel log).
2. **Replay del Log:** Si ricaricano i dati persi eseguendo in avanti (redo totale) tutte le operazioni registrate sul file di log dalla data del dump fino all'istante immediatamente precedente al guasto. Questo allinea il database allo stato esatto in cui si trovava un attimo prima del crash.
3. **Esecuzione della Ripresa a Caldo:** Poiché il replay del log ha ricreato esattamente lo scenario pre-crash (ripristinando sia le transazioni committate che quelle attive ma incomplete), si esegue una normale Ripresa a Caldo (come vista sopra) per fare "pulizia", annullando le transazioni interrotte e garantendo l'assoluta consistenza della base di dati.

Dai Sistemi Centralizzati ai DBMS Distribuiti

L'evoluzione delle basi di dati ha portato al passaggio dai sistemi tradizionali a quelli distribuiti per rispondere a esigenze di scalabilità e affidabilità.

- **Sistemi Centralizzati:** Sono caratterizzati da un unico database locale (fisicamente in un solo luogo) controllato da un singolo DBMS.
- **Sistemi Distribuiti:** Permettono l'accesso a dati che sono fisicamente dislocati su più nodi (server) remoti, collegati tra loro da una rete informatica.

Definizioni Fondamentali

1. **Database Distribuito (DDB):** È una collezione di dati condivisi e *logicamente correlati*, ma *fisicamente distribuiti* su una rete di calcolatori. Dal punto di vista logico, per l'utente, il database è percepito come un'entità unica.
2. **DBMS Distribuito (DDBMS):** È il sistema software che si occupa di gestire questa base di dati sparsa sulla rete. Il suo compito principale è rendere la distribuzione **completamente trasparente** all'utente finale (l'utente o l'applicazione interroga il database senza doversi preoccupare di *dove* si trovino fisicamente i dati).

Il Concetto di "Frammento" e l'Autonomia Locale

Il segreto del funzionamento di un database distribuito risiede nella scomposizione dei dati:

- L'unico database logico è suddiviso in un insieme di sotto-parti chiamate **frammenti**.
- Ogni frammento è memorizzato su uno o più server della rete (nodi).
- Ogni server è dotato di un proprio **DBMS locale** che controlla i frammenti lì salvati.

Questa struttura garantisce **autonomia locale**: ogni server è in grado di elaborare in modo indipendente le query degli utenti che richiedono l'accesso ai dati fisicamente presenti su quel server (agendo di fatto come un database centralizzato). Al contempo, il server è in grado di collaborare con il DDBMS per recuperare o elaborare i dati memorizzati sugli altri nodi della rete quando la query lo richiede.

Tipi di Database Distribuiti

A seconda di come vengono scelti e configurati i DBMS sui vari server, si distinguono:

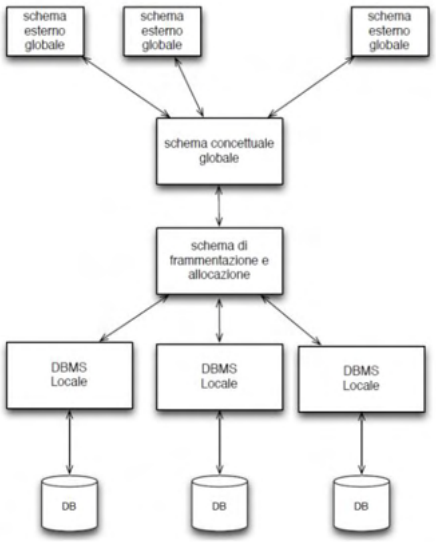
- **Database Omogenei:** Su tutti i nodi (server) della rete viene utilizzato esattamente lo stesso software DBMS (es. tutti usano Oracle o tutti usano PostgreSQL).
- **Database Eterogenei:** I server della rete utilizzano DBMS di produttori o tipologie diverse, costringendo il sistema a uno sforzo maggiore per "tradurre" e far comunicare i nodi tra loro.
- **Sistemi Multidatabase:** È un caso estremo di eterogeneità in cui ogni server mantiene la sua completa e totale autonomia operativa, comportandosi quasi come un'entità indipendente che accetta di collaborare in una federazione.

VANTAGGI (PRO):

- **Economicità e Scalabilità:** Si possono usare più server con hardware meno costoso standard, aggiungendo nodi man mano che il sistema cresce, piuttosto che comprare un singolo mainframe costosissimo.
- **Affidabilità e Disponibilità:** Se un nodo (server) si guasta, il resto del database continua a funzionare. I dati (se replicati) rimangono accessibili altrove.
- **Prestazioni:** Le query possono essere processate in parallelo su più macchine e gli utenti possono accedere ai nodi geograficamente più vicini a loro, riducendo i tempi di attesa.
- **Integrazione:** Permette di unire sotto un'unica vista logica sistemi che prima erano separati.

SVANTAGGI (CONTRO):

- **Complessità Software/Hardware:** Gestire la concorrenza, i guasti, le transazioni distribuite e il recovery su una rete è infinitamente più complesso rispetto a un sistema singolo.
- **Mancanza di Standard:** Spesso è difficile far comunicare in modo efficiente nodi eterogenei a causa della mancanza di protocolli standard universali sul mercato.



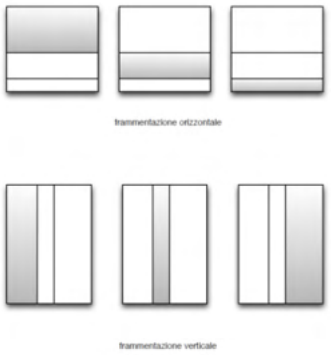
1. **Schemi Esterni Globali:** Offrono viste personalizzate agli utenti, nascondendo la complessità della rete per far apparire il database come un'unica entità.

2. **Schema Concettuale Globale:** Nucleo logico che descrive l'intera base di dati (tabelle e relazioni) in modo unitario, ignorando la sua distribuzione fisica.

3. **Schema di Frammentazione e Allocazione:** Mappa del DDBMS che specifica come i dati sono partizionati (frammentazione) e su quali nodi sono memorizzati (allocazione).

4. **DBMS Locali e DB:** Nodi fisici che gestiscono i propri dati tramite database tradizionali (ANSI/SPARC), eseguendo le istruzioni ricevute dai livelli superiori.

Tipologie di Frammentazione



1. Frammentazione Orizzontale

- **Definizione visiva (prima riga dell'immagine):** Immagina di tagliare la tabella orizzontalmente. Stai dividendo la tabella per **righe** (tuple).
- **Operazione Algebrica:** Si ottiene tramite l'operatore relazionale di **Selezione** ($_{\sigma}$). Viene applicata una condizione logica $_{\chi}$ agli attributi della relazione originaria $_{r}$.
    - *Formula:* $\sigma_{\chi}(r)$
- **Ricostruzione:** Poiché i frammenti contengono righe diverse della stessa struttura (stesse colonne), per ricostruire la tabella originale si utilizza l'operatore di **Unione insiemistica** ($_{\cup}$).
    - *Formula:* $r_1 \cup r_2 \cup \ldots \cup r_m = r$

2. Frammentazione Verticale

- **Definizione visiva (seconda riga dell'immagine):** Immagina di tagliare la tabella verticalmente. Stai dividendo la tabella per **colonne** (attributi).
- **Operazione Algebrica:** Si ottiene tramite l'operatore relazionale di **Proiezione** ($_{\pi}$). Si estrae un sottoinsieme specifico di attributi $_{A}$ dalla relazione originaria $_{r}$.
    - *Formula:* $\pi_{A}(r)$
- **Ricostruzione:** Poiché i frammenti contengono dati appartenenti a colonne diverse, per riunirli correttamente associando i dati alla riga giusta, si utilizza

l'operatore di **Join naturale** ($\bowtie$). *Nota: per fare il join, la chiave primaria deve essere presente in tutti i frammenti verticali.*

- *Formula:* $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_m = r$

Regole di Correttezza della Frammentazione

Non si può tagliare una tabella in modo casuale. Affinché la frammentazione di una relazione r in frammenti $r_1, r_2, \ldots, r_m$ sia formalmente corretta, deve rispettare tre proprietà ferree:

1. **Completezza:** Nessun dato deve essere perso. Ogni singola tupla (o attributo) presente nella relazione originale r deve potersi trovare in almeno uno dei frammenti r_i.
2. **Ricostruibilità:** Deve essere sempre possibile, applicando operazioni matematiche standard (come Unione o Join), prendere tutti i frammenti r_i e riassemblare esattamente la relazione r di partenza, mantenendo intatte tutte le sue dipendenze e legami funzionali.
3. **Disgiunzione:** I frammenti non devono sovrapporsi. Se un dato (non chiave) appare in un frammento, non deve apparire in nessun altro. Questo previene la ridondanza non voluta a livello logico.

Allocazione sui Server e Trasparenza

Una volta che i dati sono stati "tagliati" nei vari frammenti, bisogna decidere **dove** salvarli fisicamente nella rete di computer. Questa operazione si chiama allocazione.

- **Allocazione Non Ridondante:** Ogni frammento viene salvato su un solo e unico server della rete. Minimizza lo spazio occupato, ma espone al rischio: se il server cade, quel frammento è inaccessibile.
- **Allocazione Ridondante:** Lo stesso frammento (o copia esatta) viene salvato su più server diversi. Aumenta lo spazio occupato e la complessità degli aggiornamenti, ma garantisce alta disponibilità dei dati in caso di guasti.

Il fine ultimo: La Trasparenza

Tutto questo complesso lavoro di scomposizione e posizionamento ha uno scopo: deve rimanere "invisibile" all'utente.

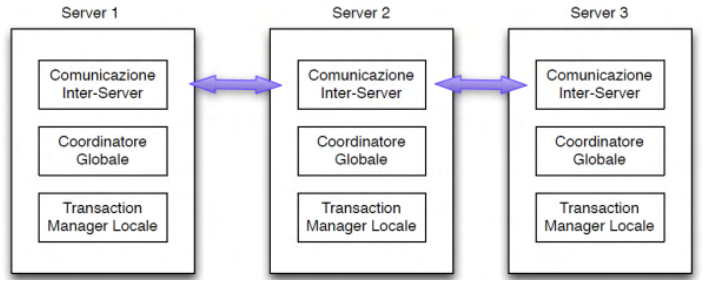
- **Trasparenza di Frammentazione:** L'utente o il programmatore fa le sue query sulla tabella come se fosse unita; non sa, e non deve sapere, che in realtà la tabella è "tagliata" in frammenti.

- **Trasparenza di Allocazione:** L'utente non sa su quale specifico server (o città, o nazione) siano fisicamente salvati i dati che sta richiamando. Il DDBMS gestisce tutto il routing in background.

Gestione delle Transazioni nei DDBMS (Database Distribuiti)

In un sistema distribuito, l'esecuzione di una singola operazione (chiamata **transazione globale**) spesso richiede l'attivazione simultanea di operazioni più piccole su più server diversi della rete (chiamate **sottotransazioni locali**).

L'obiettivo fondamentale del DDBMS è garantire che le **proprietà ACID** (Atomicità, Consistenza, Isolamento, Persistenza) siano rispettate non solo per ogni singola sottotransazione locale, ma per l'intera transazione globale nel suo complesso.



Per fare questo, l'architettura si dota di:

- **Coordinatore Globale** su ogni server che ha il compito di coordinare l'esecuzione sia delle transazioni globali sia di quelle locali sul sito (gestite attraverso un Transaction Manager Locale).
- **Comunicazione Inter-Server:** Un'infrastruttura di rete sicura per scambiare messaggi tra i vari nodi.

La Verifica delle Proprietà ACID

1. Consistenza e Atomicità (in assenza di guasti)

I vincoli di integrità dei dati (Consistenza) sono gestiti localmente. Allo stesso modo i meccanismi usati nei sistemi centralizzati per garantire che una transazione si atomica continuano a funzionare anche a livello globale.

2. Isolamento (Controllo della Concorrenza)

L'isolamento previene conflitti tra transazioni simultanee. In un sistema distribuito vale un principio fondamentale:

Se lo schedule (l'ordine di esecuzione) delle transazioni in ogni singolo server locale è serializzabile, allora anche lo schedule globale è serializzabile.

Come si ottiene?

- Se ogni server locale usa il protocollo **2PL Ristretto** (Strict Two-Phase Locking).
- Se il sistema usa l'**ordinamento tramite Timestamp** e i timestamp vengono assegnati in maniera globale e univoca a tutte le sottotransazioni.

3. Persistenza e la Gestione dei Guasti

La vera criticità nei sistemi distribuiti riguarda la **Persistenza** (e di riflesso l'Atomicità). Se si verifica un guasto, bisogna evitare scenari disastrosi in cui una metà del database pensa che la transazione sia completata e l'altra metà l'abbia annullata.

L'ambiente di rete introduce nuove e insidiose tipologie di fallimento:

- Perdita di messaggi.
- Caduta improvvisa della connessione.
- Crash fisico di un singolo server.
- Partizionamento della rete (un guasto hardware divide la rete a metà, isolando due gruppi di server).

Il Protocollo Two-Phase Commit (2PC)

Per garantire l'**Atomicità globale anche in presenza di fallimenti**, è necessario che il sistema abbia la certezza matematica che tutte le sottotransazioni terminino allo stesso modo (tutte *Commit* o tutte *Abort*).

Il meccanismo standard per ottenere questo è il protocollo **Commit a Due Fasi (Two-Phase Commit - 2PC)**.

Ruoli nel 2PC

- **Coordinatore:** Il server che ha preso in carico la transazione globale (di solito quello che ha ricevuto la query dall'utente). Conosce l'identità di tutti gli altri nodi coinvolti.
- **Partecipanti:** Gli altri server locali che devono eseguire fisicamente le sottotransazioni. Ognuno conosce chi è il Coordinatore.

Importante: Sia il Coordinatore sia i Partecipanti utilizzano i propri **File di Log locali** per registrare le loro dichiarazioni, in modo da poter resistere a un crash improvviso.

Come funziona il 2PC (L'analogia del "Matrimonio")

Il protocollo si divide in due fasi distinte e funziona esattamente come la celebrazione di un matrimonio: affinché la transazione sia finalizzata, il "sì" deve essere unanime.

Fase 1: La Fase di Voting (La Promessa)

Il Coordinatore chiede a tutti i Partecipanti: *"Siete pronti a eseguire il commit della vostra parte di transazione?"*

A questo punto ogni Partecipante verifica le proprie condizioni locali e vota:

- **Se un partecipante vota ABORT:** Ha il potere di abortire la propria parte in modo unilaterale. La transazione fallirà globalmente.

- **Se un partecipante vota COMMIT:** Fa una promessa formale (che registra sul proprio Log). Da questo momento **perde la sua autonomia**. Non può più annullare la transazione né completarla, ma *deve attendere* ordini dal Coordinatore.
- Se un partecipante non risponde entro un tempo massimo (Timeout), il Coordinatore assume che il voto sia "Abort".

Fase 2: La Fase di Decisione (La Pronuncia)

Il Coordinatore raccoglie tutti i voti.

- **Global Commit:** Se *TUTTI* i Partecipanti hanno risposto "Sì" (Commit) in tempo utile, il Coordinatore registra la decisione globale nel suo Log e ordina a tutti i nodi di procedere con il *Commit* effettivo.
- **Global Abort:** Se *anche un solo* Partecipante ha votato "No" (o è andato in Timeout), il Coordinatore decide per l'annullamento globale. Invia a tutti i nodi il comando di eseguire l'*Abort* (Rollback).

Tutti i partecipanti sono obbligati ad attenersi rigidamente alla Decisione Globale.

Criticità del 2PC

Sebbene molto sicuro, il protocollo presenta due problemi principali:

1. **Deadlock (Stallo):** I partecipanti che hanno votato "Commit" restano bloccati in attesa della decisione del Coordinatore. Se non ricevono istruzioni, non possono rilasciare i *lock* sulle loro tabelle.
2. **Caduta del Coordinatore (SPOF):** Se il Coordinatore subisce un crash o si disconnette esattamente tra la Fase 1 e la Fase 2, i Partecipanti che hanno votato "Commit" rimangono "appesi" in un limbo, congelando risorse vitali nel database locale senza sapere come procedere.

La Replicazione dei Dati (Modello Asimmetrico)

Consiste nel distribuire copie identiche di un database su più server. Il principio base è che **ogni modifica** apportata a un dato deve essere replicata su tutte le copie esistenti.

L'Architettura Master-Slave

Il sistema è gerarchico e si divide in due ruoli:

- **Server Master:** È il nodo principale. Riceve le modifiche dagli utenti e "comanda" la propagazione degli aggiornamenti.
- **Server Slave:** Sono i nodi secondari che ospitano le copie. Si limitano ad aggiornare i propri dati per restare perfettamente allineati al Master.

Modalità di Sincronizzazione

L'allineamento tra Master e Slave avviene in due modi:

- **Sincrona (Tempo reale):** L'aggiornamento sugli Slave avviene *immediatamente* insieme alla modifica sul Master (utilizzando protocolli sicuri come il 2PC).
- **Asincrona (Differita):** L'aggiornamento sugli Slave avviene *periodicamente* (ogni tot minuti, ore o giorni).

Vantaggi Chiave

- **Alta Affidabilità (Fault Tolerance):** Avendo copie multiple, se un server si guasta il sistema continua a funzionare e il *recovery* a caldo è immediato.
- **Prestazioni e Scalabilità:** Si effettua un bilanciamento del carico (*load balancing*): le richieste vengono smistate su più server, evitando di sovraccaricare una singola macchina e permettendo l'accesso a un numero molto più alto di utenti.